

# REPUBLIQUE DU BURUNDI



## COURS DE PROGRAMMATION WEB I

**Faculté d'Informatique de Gestion**

**Deuxième année de baccalauréat**

**A/A : 2025-2026**

**Titulaire :**

**Msc. Jackson NIGARURA**

|                                                                    |    |
|--------------------------------------------------------------------|----|
| CHAPITRE I : COMMENT CREER DES SITES WEB ? .....                   | 1  |
| I.1 Comment fonctionnent les sites web ? .....                     | 1  |
| I.2 HTML et CSS : deux langages pour créer un site web.....        | 1  |
| Le rôle de HTML et CSS .....                                       | 1  |
| I.3 L'éditeur de texte.....                                        | 2  |
| I.4 Les navigateurs .....                                          | 3  |
| I.5 Créer une page web avec l'éditeur .....                        | 4  |
| I.6 Les balises et leurs attributs .....                           | 5  |
| Les balises.....                                                   | 5  |
| Les attributs.....                                                 | 6  |
| I.7 Structure de base d'une page HTML5.....                        | 6  |
| Le doctype.....                                                    | 7  |
| La balise <html> .....                                             | 7  |
| L'en-tête <head> et le corps <body>.....                           | 7  |
| I.8 Les commentaires .....                                         | 9  |
| CHAPITRE II COMMENT ORGANISER SON TEXTE .....                      | 10 |
| II.1 Les paragraphes.....                                          | 10 |
| II.2 Les titres .....                                              | 13 |
| II.3 La mise en valeur .....                                       | 15 |
| Mettre un peu en valeur .....                                      | 16 |
| Mettre bien en valeur .....                                        | 16 |
| Marquer le texte .....                                             | 17 |
| Mettre en exposant ou en indice .....                              | 18 |
| Les citations .....                                                | 19 |
| Les acronymes.....                                                 | 20 |
| II.4 Les listes à puces .....                                      | 21 |
| Liste non ordonnée.....                                            | 21 |
| Liste ordonnée.....                                                | 22 |
| CHAPITRE III : COMMENT CREER DES LIENS ET INSERER DES IMAGES ..... | 24 |
| III.1 Comment créer Un lien vers un autre site.....                | 24 |
| III.2 Comment créer un lien vers une autre page de son site .....  | 24 |
| III.2.1 Deux pages situées dans un même dossier.....               | 25 |
| III.2.2 Deux pages situées dans des dossiers différents .....      | 25 |
| III.3 Un lien vers une ancre.....                                  | 26 |
| III.4 Cas pratiques d'utilisation des liens .....                  | 28 |
| 1. Un lien qui affiche une info bulle au survol.....               | 28 |
| 2. Un lien qui ouvre une nouvelle fenêtre .....                    | 28 |
| 3. Un lien pour envoyer un e-mail .....                            | 29 |
| 4. Un lien pour télécharger un fichier .....                       | 29 |
| III.5 Les différents formats d'images .....                        | 29 |
| Insertion d'une image .....                                        | 31 |
| Ajouter une infobulle .....                                        | 32 |
| III.6 Les figures.....                                             | 33 |
| Création d'une figure.....                                         | 34 |
| Bien comprendre le rôle des figures .....                          | 34 |
| CHAPITRE IV METTRE EN PLACE LE CSS.....                            | 36 |
| IV 1. Petit rappel : à quoi sert CSS ? .....                       | 36 |
| IV 2. Où écrit-on le CSS ? .....                                   | 37 |
| Dans un fichier .css (recommandé).....                             | 37 |
| Dans l'en-tête <head> du fichier HTML .....                        | 39 |
| Directement dans les balises (non recommandé).....                 | 39 |

|                                                         |    |
|---------------------------------------------------------|----|
| IV.3 Appliquer un style : sélectionner une balise ..... | 40 |
| IV. 4. Des commentaires dans du CSS .....               | 42 |
| IV 5. Appliquer un style : class et id.....             | 42 |
| IV. 6. Les balises universelles.....                    | 44 |
| IV. 7. Appliquer un style : les sélecteurs avancés..... | 44 |
| CHAPITRE V FORMATAGE DU TEXTE .....                     | 47 |
| V.1 La taille du texte.....                             | 47 |
| V. 2. La police.....                                    | 48 |
| V.3 Comment mettre en Italique, gras, souligné.....     | 49 |
| V.4 L'alignement du texte .....                         | 51 |
| V.5 Les flottants.....                                  | 51 |
| Faire flotter une image .....                           | 52 |
| Stopper un flottant.....                                | 52 |
| V.6 Couleur du texte .....                              | 53 |
| Indiquer le nom de la couleur .....                     | 53 |
| La notation hexadécimale .....                          | 54 |
| La méthode RGB .....                                    | 54 |
| V.7 Couleur de fond.....                                | 55 |
| Le CSS et l'héritage.....                               | 55 |
| Exemple d'héritage avec la balise <mark> .....          | 56 |
| V.8 Images de fond.....                                 | 56 |
| Appliquer une image de fond.....                        | 56 |
| Options disponibles pour l'image de fond.....           | 57 |
| Combiner les propriétés .....                           | 58 |
| Plusieurs images de fond.....                           | 58 |
| V.9 La transparence .....                               | 59 |
| La propriété opacity .....                              | 59 |
| La notation RGBa .....                                  | 59 |
| CHAPITRE VI LES BORDURES ET LES OMBRES .....            | 61 |
| VI.1 Bordures standard .....                            | 61 |
| En haut, à droite, à gauche, en bas... ..               | 61 |
| VI.2 Bordures arrondies.....                            | 62 |
| VI.3 Les ombres.....                                    | 63 |
| box-shadow : les ombres des boîtes.....                 | 63 |
| text-shadow : l'ombre du texte .....                    | 63 |
| CHAPITRE VII CREATION D'APPARENCES DYNAMIQUES .....     | 64 |
| VII.1 Au survol.....                                    | 64 |
| VII.2 Au clic et lors de la sélection.....              | 64 |
| :active : au moment du clic .....                       | 64 |
| :focus : lorsque l'élément est sélectionné .....        | 65 |
| VII.3 Lorsque le lien a déjà été visité .....           | 65 |
| CHAPITRE VIII STRUCTURER SA PAGE.....                   | 66 |
| VIII.1 Les balises structurantes de HTML5.....          | 66 |
| <header> : l'en-tête.....                               | 66 |
| <footer> : le pied de page .....                        | 66 |
| <nav> : principaux liens de navigation.....             | 66 |
| <section> : une section de page .....                   | 67 |
| <aside> : informations complémentaires .....            | 68 |
| <article> : un article indépendant .....                | 68 |
| Résumé.....                                             | 68 |
| VIII.2 Exemple concret d'utilisation des balises.....   | 69 |
| VIII.3 Assurer la compatibilité avec IE .....           | 70 |

|                                                                |     |
|----------------------------------------------------------------|-----|
| CHAPITRE IX : LE MODELE DE BOITE .....                         | 72  |
| IX.1 Les balises de type block et inline .....                 | 72  |
| IX.2 Les dimensions .....                                      | 73  |
| IX.3 Les marges .....                                          | 74  |
| CHAPITRE X : Le positionnement en CSS .....                    | 76  |
| X.1 Le positionnement flottant .....                           | 76  |
| En haut, à droite, à gauche, en bas... Et on recommence !..... | 78  |
| Centrer des blocs.....                                         | 79  |
| <i>Quand ça dépasse...</i> .....                               | 79  |
| overflow : couper un bloc .....                                | 79  |
| word-wrap : couper les textes trop larges .....                | 80  |
| X.2 Transformez vos éléments avec display.....                 | 81  |
| X.3 Le positionnement inline-block.....                        | 81  |
| X.4 Les positionnements absolu, fixe et relatif.....           | 83  |
| Le positionnement absolu .....                                 | 83  |
| Le positionnement fixe.....                                    | 84  |
| Le positionnement relatif .....                                | 85  |
| CHAPITRE XI : LES TABLEAUX ET LES FORMULAIRES.....             | 86  |
| XI.1 Un tableau simple .....                                   | 86  |
| XI.2 Un tableau structuré .....                                | 89  |
| XI.3 Créer un formulaire.....                                  | 92  |
| <i>Les zones de saisie basiques</i> .....                      | 94  |
| Zone de texte monoligne.....                                   | 94  |
| Les libellés .....                                             | 94  |
| Quelques attributs supplémentaires .....                       | 95  |
| Zone de mot de passe .....                                     | 95  |
| Zone de texte multiligne .....                                 | 96  |
| <i>Les zones de saisie enrichies</i> .....                     | 97  |
| E-mail.....                                                    | 97  |
| Une URL .....                                                  | 97  |
| Numéro de téléphone .....                                      | 97  |
| Nombre.....                                                    | 98  |
| Un curseur.....                                                | 98  |
| Couleur.....                                                   | 98  |
| Date .....                                                     | 98  |
| Recherche.....                                                 | 99  |
| <i>Les éléments d'options</i> .....                            | 99  |
| Les cases à cocher .....                                       | 99  |
| Les zones d'options .....                                      | 100 |
| Les listes déroulantes .....                                   | 101 |
| <i>Finaliser et envoyer le formulaire</i> .....                | 102 |
| Regrouper les champs .....                                     | 102 |
| Sélectionner automatiquement un champ .....                    | 103 |
| Rendre un champ obligatoire .....                              | 103 |
| Le bouton d'envoi.....                                         | 104 |
| CHAPITRE XII : LA VIDEO ET L'AUDIO.....                        | 105 |
| XII.1 Les formats audio et vidéo .....                         | 105 |
| Les formats audio.....                                         | 105 |
| Les formats vidéo.....                                         | 105 |
| XII.2 Insertion d'un élément audio.....                        | 107 |
| XII.3 Insertion d'une vidéo .....                              | 108 |
| Envoyez votre site sur le web .....                            | 110 |

|                                     |     |
|-------------------------------------|-----|
| <i>Le nom de domaine</i> .....      | 110 |
| Réserver un nom de domaine .....    | 110 |
| <i>L'hébergeur</i> .....            | 110 |
| Le rôle de l'hébergeur.....         | 110 |
| Trouver un hébergeur.....           | 111 |
| <i>Utiliser un client FTP</i> ..... | 112 |
| Installer un client FTP.....        | 112 |
| Configurer le client FTP .....      | 113 |
| Transférer les fichiers.....        | 114 |

# CHAPITRE I : COMMENT CREER DES SITES WEB ?

## I.1 Comment fonctionnent les sites web ?

De nombreuses personnes confondent (à tort) Internet et le Web. Il faut savoir que le Web fait partie d'Internet. Internet est donc un grand ensemble qui comprend, entre autres : le Web, les e-mails, la messagerie instantanée, etc.

Les langages HTML et CSS sont à la base du fonctionnement de tous les sites web. Quand vous visitez un site avec votre navigateur, il faut savoir que derrière des rouages s'activent pour permettre au site web de s'afficher. L'ordinateur se base sur ce qu'on lui a expliqué en HTML et CSS pour savoir ce qu'il doit afficher.

HTML et CSS sont deux "langues" qu'il faut savoir parler pour créer des sites web. C'est le navigateur web qui fera la traduction entre ces langages informatiques et ce que vous verrez s'afficher à l'écran.

## I.2 HTML et CSS : deux langages pour créer un site web

Pour créer un site web, on doit donner des instructions à l'ordinateur. Il ne suffit pas simplement de taper le texte qu'il y aura dans son site (comme on le ferait dans un traitement de texte Word par exemple), il faut aussi indiquer où placer ce texte, insérer des images, faire des liens entre les pages, etc.

### Le rôle de HTML et CSS

Pour expliquer à l'ordinateur ce que vous voulez faire, il va falloir utiliser un langage qu'il comprend. Et c'est là que les choses se compliquent, parce qu'il va falloir apprendre deux langages !

Pourquoi avoir créé deux langages ?

S'il y a deux langages c'est au contraire pour faciliter les choses. Nous allons avoir affaire à deux langages qui se complètent car ils ont des rôles différents :

- **HTML** (HyperText Markup Language) : il a fait son apparition dès 1991 lors du lancement du Web. Son rôle est de gérer et organiser le contenu. C'est donc en HTML que vous écrirez ce que vous souhaitez que la page affiche : du texte, des liens, des images... Vous direz par exemple : "Ceci est mon titre, ceci est mon menu, voici le texte principal de la page, voici une image à afficher, etc."
- **CSS** (Cascading Style Sheets, aussi appelées Feuilles de style) : le rôle du CSS est de gérer l'apparence de la page web (agencement, positionnement, décoration, couleur, taille du texte...). Ce langage est venu compléter le HTML en 1996.

Vous avez peut-être aussi entendu parler du langage XHTML. Il s'agit d'une variante du HTML qui se veut plus rigoureuse et qui est donc un peu plus délicate à manipuler.

Pour faire simple, le HTML est apparu le premier en 1991. Début 2000, le W3C(Le **World Wide Web Consortium**, abrégé par le sigle **W3C**, est un organisme de normalisation à but non-lucratif, fondé en octobre 1994 comme un consortium chargé de promouvoir la compatibilité des technologies du World Wide Web telles que HTML, XHTML, XML, RDF, SPARQL, CSS, PNG, SVG et SOAP.) a lancé le XHTML en indiquant que ce serait l'avenir... mais le XHTML n'a pas percé comme on l'espérait. Retour aux sources en 2009 : le W3C abandonne le XHTML et décide de revenir au HTML pour le faire évoluer.

Il y a beaucoup de confusion autour de ces langages, alors qu'ils se ressemblent beaucoup. Aucun n'est vraiment meilleur que l'autre, il s'agit de deux façons de faire différentes. Dans ce cours, nous allons travailler sur la dernière version de HTML (HTML5) qui est aujourd'hui le langage d'avenir que tout le monde est incité à utiliser.

Vous pouvez très bien créer un site web uniquement en HTML, mais celui-ci ne sera pas très beau : l'information apparaîtra "brute". C'est pour cela que le langage CSS vient toujours le compléter.

Le HTML définit le contenu (comme vous pouvez le voir, c'est brut de décoffrage !). Le CSS permet, lui, d'arranger le contenu et de définir la présentation : couleur, image de fond, marges, taille du texte...

Comme vous vous en doutez, le CSS a besoin d'une page HTML pour fonctionner. C'est pour cela que nous allons d'abord apprendre les bases du HTML avant de nous occuper de la décoration en CSS.

Vos premières pages ne seront donc pas les plus esthétiques, mais qu'importe ! Ca ne durera pas longtemps.

### 1.3 L'éditeur de texte

Il existe de nombreux logiciels dédiés à la création de sites web. Bloc-notes peut être suffisant! Il y a cependant des logiciels plus puissants aujourd'hui et personne n'utilise vraiment Bloc-Notes. On peut classer ces logiciels de création de site web en deux catégories :

- Les **WYSIWYG** (What You See Is What You Get - Ce Que Vous Voyez Est Ce Que Vous Obtenez) : ce sont des programmes qui se veulent très faciles d'emploi, ils permettent de créer des sites web sans apprendre de langage particulier. Parmi les plus connus d'entre eux : Nvu, Microsoft Expression Web, Dreamweaver... ! Leurs principaux outils, souvent d'assez mauvaise qualité. Un bon créateur de site web doit tôt ou tard connaître HTML et CSS, c'est pourquoi je ne recommande pas l'usage de ces outils.

- Les **éditeurs de texte** : ce sont des programmes dédiés à l'écriture de code. On peut en général les utiliser pour de multiples langages, pas seulement HTML et CSS. Ils se révèlent être de puissants alliés pour les créateurs de sites web !

Vous l'aurez compris, je vais vous inviter à utiliser un éditeur de texte dans ce cours.

## Sous Windows

Il existe une grande quantité d'éditeurs de texte, on ne pourra pas les voir tous. Néanmoins, on va se pencher sur Notepad++, l'un des plus utilisés d'entre eux sous Windows. Ce logiciel est simple, en français et gratuit.

D'autres éditeurs disponibles sous Windows existent. Si Notepad++ ne vous convient pas, vous pouvez essayer :

- [jEdit](#)
- [PSPad](#)
- [ConTEXT](#)
- ... et bien d'autres si vous recherchez "Editeur de texte" sur le Web.

## 1.4 Les navigateurs

### Pourquoi le navigateur est important

Le navigateur est le programme qui nous permet de voir les sites web.

Le travail du navigateur est de lire le code HTML et CSS pour afficher un résultat visuel à l'écran. Si votre code CSS dit "Les titres sont en rouge", alors le navigateur affichera les titres en rouge. Le rôle du navigateur est donc essentiel !

On ne dirait pas, mais un navigateur est un programme extrêmement complexe. Comprendre le code HTML et CSS n'est en effet pas une mince affaire. Le principal problème, vous vous en rendrez vite compte, c'est que **les navigateurs n'affichent pas tous les sites exactement de la même façon** ! Il faudra vous y faire et prendre l'habitude de vérifier régulièrement que votre site fonctionne correctement sur la plupart des navigateurs.

Il existe de nombreux navigateurs différents. Voici les principaux à connaître :

| Navigateur                                                                                                  | OS                      | Commentaires                                             |
|-------------------------------------------------------------------------------------------------------------|-------------------------|----------------------------------------------------------|
| <b>Google Chrome</b><br> | Windows<br>Mac<br>Linux | Le navigateur de Google, simple d'emploi et très rapide. |



| Navigateur                                                                                                    | OS                      | Commentaires                                                                            |
|---------------------------------------------------------------------------------------------------------------|-------------------------|-----------------------------------------------------------------------------------------|
| <b>Mozilla Firefox</b><br>   | Windows<br>Mac<br>Linux | Le navigateur de la fondation Mozilla, célèbre et réputé.                               |
| <b>Internet Explorer</b><br> | Windows                 | Le navigateur de Microsoft, qui équipe tous les PC Windows.                             |
| <b>Safari</b><br>            | Windows<br>Mac          | Le navigateur d'Apple, qui équipe tous les Mac.                                         |
| <b>Opera</b><br>             | Windows<br>Mac<br>Linux | L'éternel outsider.<br>Il est moins utilisé mais propose de nombreuses fonctionnalités. |

**Il est conseillé d'installer plusieurs navigateurs sur son ordinateur** pour s'assurer que son site fonctionne correctement sur chacun d'eux.

## I.5 Créer une page web avec l'éditeur

Dans ce point, nous allons commencer à pratiquer ! Nous allons découvrir les bases du langage HTML et enregistrer notre toute première page web !

On va faire un petit essai.

Vous pouvez écrire les quelques phrases ; le but est d'écrire quelque chose. Maintenant, enregistrons ce fichier. Pour ça, c'est très simple : comme dans tous les programmes, vous avez un menu Fichier / Enregistrer. Une boîte de dialogue vous demande où enregistrer le fichier et sous quel nom. Enregistrez-le où vous voulez. Donnez au fichier le nom que vous voulez, en terminant par .html, par exemple : **test.html**.

Je vous recommande de créer un nouveau dossier dans vos documents qui contiendra les fichiers de votre site. Pour ma part j'ai créé un dossier test dans lequel j'ai mis mon fichier **test.html**.

Ouvrez maintenant l'explorateur de fichiers dans le dossier où vous avez enregistré votre page. Vous y verrez le fichier que vous venez de créer.

L'apparence du fichier dépend de votre navigateur web par défaut. Double-cliquez simplement sur ce fichier et... votre navigateur s'ouvre en affichant le texte que vous avez écrit !

**Le texte s'affiche sur la même ligne alors qu'on avait demandé à l'écrire sur 2 lignes différentes. Que se passe-t-il ?**

En fait, pour créer une page web il ne suffit pas simplement de taper du texte comme on vient de le faire. En plus de ce texte, il faut aussi écrire ce qu'on appelle des **balises**, qui vont donner des instructions à l'ordinateur comme "aller à la ligne", "afficher une image", etc.

## 1.6 Les balises et leurs attributs

Il ne suffit pas "simplement" d'écrire du texte dans l'éditeur, il faut aussi donner des instructions à l'ordinateur. En HTML, on passe pour cela par des **balises**.

### Les balises

Les pages HTML sont remplies de ce qu'on appelle des balises. Celles-ci sont invisibles à l'écran pour vos visiteurs, mais elles permettent à l'ordinateur de comprendre ce qu'il doit afficher. Les balises se repèrent facilement. Elles sont entourées de "chevrons", c'est-à-dire des symboles < et >, comme ceci : <balise>

À quoi est-ce qu'elles servent ? Elles indiquent la nature du texte autour d'elles. Elles veulent dire par exemple : "Ceci est le titre de la page", "Ceci est une image", "Ceci est un paragraphe de texte", etc.

On distingue deux types de balises : les balises en paires et les balises orphelines.

### Les balises en paires

Elles s'ouvrent, contiennent du texte, et se ferment plus loin. Voici à quoi elles ressemblent :

**Code : HTML -**

```
<titre>Ceci est un titre</titre>
```

|                                                                                                                                                                                                                           |                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| On distingue une balise ouvrante (<titre>) et une balise fermante (</titre>) qui indique que le titre se termine. Cela signifie pour l'ordinateur que tout ce qui n'est pas entre ces deux balises... n'est pas un titre. | Ceci n'est pas un titre <titre>Ceci est un titre</titre> Ceci n'est pas un titre |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|

### Les balises orphelines

Ce sont des balises qui servent le plus souvent à insérer un élément à un endroit précis (par exemple une image). Il n'est pas nécessaire de délimiter le début et la fin de l'image, on veut juste dire à l'ordinateur "Insère une image ici". Une balise orpheline s'écrit comme ceci :

**Code : HTML -**

```
<image />
```

Notez que le / de fin n'est pas obligatoire. On pourrait écrire seulement <image>. Néanmoins, pour ne pas confondre avec le premier type de balise, les webmasters recommandent de rajouter ce / (slash) à

la fin de la balise. Vous me verrez donc mettre un / aux balises orphelines et je vous recommande de faire de même, c'est une bonne pratique.

## Les attributs

Les attributs sont un peu les options des balises. Ils viennent les compléter pour donner des informations supplémentaires. L'attribut se place après le nom de la balise ouvrante et a le plus souvent une valeur, comme ceci :

```
1 <balise attribut="valeur">
```

A quoi ça sert ? Prenons la balise `<img />` que nous venons de voir. Seule, elle ne sert pas à grand-chose. On pourrait rajouter un attribut qui indique le nom de l'image à afficher :

```
1 <img nom="photo.jpg" />
```

L'ordinateur comprend alors qu'il doit afficher l'image contenue dans le fichier photo.jpg. Dans le cas d'une balise fonctionnant "par paire", on ne met les attributs que dans la balise ouvrante et pas dans la balise fermante. Par exemple, ce code indique que la citation est de Neil Armstrong et qu'elle date du 21 Juillet 1969 :

```
1 <citation auteur="Neil Armstrong" date="21/07/1969">
```

```
2 C'est un petit pas pour l'homme un bond de géant pour l'humanité.
```

```
3 </citation>
```

Toutes les balises que nous venons de voir sont fictives. Les vraies balises ont des noms en anglais, nous allons les découvrir dans la suite de ce cours.

## 1.7 Structure de base d'une page HTML5

Reprenons ce code correspondant à la base d'une page web en HTML5 :

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    <meta charset="utf-8/" >
```

```
    <title>Titre</title>
```

```
  </head>
```

```
  <body>
```

```
  </body>
```

```
</html>
```

Vous noterez que les balises s'ouvrent et se ferment dans un ordre précis. Par exemple, la balise `<html>` est la première que l'on ouvre, et c'est aussi la dernière que l'on ferme (tout à la fin du code, avec `</html>`). Les balises doivent être fermées dans le sens inverse de leur ouverture. Un exemple :

- `<html><body></body></html>` : **correct**. Une balise qui est ouverte à l'intérieur d'une autre doit aussi être fermée à l'intérieur.
- `<html><body></html></body>` : **incorrect**, les balises s'entremêlent.

## Le doctype

```
<!DOCTYPE html>
```

La toute première ligne s'appelle le doctype. Elle est indispensable car c'est elle qui indique qu'il s'agit bien d'une page web HTML.

Ce n'est pas vraiment une balise comme les autres (elle commence par un point d'exclamation), vous pouvez considérer que c'est un peu l'exception qui confirme la règle.

Cette ligne du doctype était autrefois incroyablement complexe. Il était impossible de la retenir de tête.

Pour XHTML 1.0, il fallait écrire :

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

Pour HTML5, il a été décidé de la simplifier, pour le plus grand bonheur des webmasters. Quand vous voyez une balise doctype courte (`<!DOCTYPE html>`), cela signifie que la page est écrite en HTML5.

## La balise `<html>`

```
<html>
```

```
2
```

```
3
```

```
4 </html>
```

C'est la balise principale du code. Elle englobe tout le contenu de votre page. Comme vous pouvez le voir, la balise fermante `</html>` se trouve tout à la fin du code !

## L'en-tête `<head>` et le corps `<body>`

Une page web est constituée de 2 parties :

- L'en-tête `<head>` : cette section donne quelques informations générales sur la page, comme son titre, l'encodage (pour la gestion des caractères spéciaux), etc. Cette section est généralement assez courte. Les informations que l'en-tête contient ne sont pas affichées sur la page, ce sont simplement des informations générales à destination de l'ordinateur. Elles sont cependant très importantes !

- Le corps `<body>` : c'est là que se trouve la partie principale de la page. Tout ce que nous écrirons ici sera affiché à l'écran. C'est à l'intérieur du corps que nous écrirons la majeure partie de notre code.

Pour le moment, le corps est vide (nous y reviendrons plus loin). Intéressons-nous par contre aux deux balises contenues dans l'en-tête...

## L'encodage (charset)

```
1 <meta charset="utf-8 /">
```

UTF-8 Unicode Transformation Format-8

Cette balise indique l'encodage utilisé dans votre fichier **.html**. Sans rentrer dans les détails, car cela pourrait vite devenir compliqué, l'encodage indique la façon dont le fichier est enregistré. C'est lui qui détermine comment les caractères spéciaux vont s'afficher (accents, idéogrammes chinois et japonais, symboles arabes, etc.).

Il y a plusieurs techniques d'encodage aux noms bizarres utilisées en fonction des langues : ISO-8859-1, OEM 775, Windows-1253... Un seul cependant devrait être utilisé aujourd'hui autant que possible : UTF-8. Cette méthode d'encodage permet d'afficher sans aucun problème pratiquement tous les symboles de toutes les langues de notre planète ! C'est pour cela que j'ai indiqué utf-8 dans cette balise.

Il faut aussi que votre fichier soit bien enregistré en UTF-8. C'est le cas le plus souvent sous Linux par défaut, mais sous Windows il faut généralement le dire au logiciel.

Sous Notepad++, allez dans le menu Encodage > Encoder en UTF-8 (sans BOM) pour que votre fichier soit enregistré en UTF-8 dès le début. Cela ne s'applique qu'au fichier actuellement ouvert. Pour ne pas avoir à le faire pour chaque nouveau fichier, je vous conseille d'aller dans le menu Paramétrage > Préférences, onglet Nouveau document/Dossier. Sélectionnez UTF-8 sans BOM dans la liste.

Si vous avez un problème d'affichage des accents plus tard sur votre page web, c'est qu'il y a un problème avec l'encodage. Vérifiez que la balise indique bien UTF-8 et que votre fichier est enregistré en UTF-8 (votre éditeur de texte est capable de vous le dire, Notepad++ le fait dans le menu Encodage).

## Le titre principal de la page

```
1 <title>
```

C'est le titre de votre page, probablement l'élément le plus important ! Toute page doit avoir un titre qui décrit ce qu'elle contient. Il est conseillé que le titre soit assez court (moins de 100 caractères en général).

Le titre ne s'affiche pas dans votre page mais en haut de celle-ci (souvent dans l'onglet du navigateur). Enregistrez votre page web et ouvrez-la dans votre navigateur. Vous verrez que le titre s'affiche dans l'onglet.

## I.8 Les commentaires

Nous avons appris à créer notre première vraie page HTML dans ce chapitre. Avant de terminer, j'aimerais vous présenter le principe des commentaires.

Un **commentaire** en HTML est un texte qui sert simplement de mémo. Il n'est pas affiché, il n'est pas lu par l'ordinateur, ça ne change rien à l'affichage de la page.

Ça sert pour vous et les personnes qui liront le code source de votre page. Vous pouvez utiliser les commentaires pour laisser des indications sur le fonctionnement de votre page.

Quel intérêt ? Cela vous permettra de vous souvenir comment fonctionne votre page si vous revenez sur votre code source après un long moment d'absence.

### Insérer un commentaire

Un commentaire est une balise HTML avec une forme bien spéciale :

```
1  <!-- Ceci est un commentaire -->
```

Vous pouvez le mettre où vous voulez au sein de votre code source : il n'a aucun impact sur votre page, mais vous pouvez vous en servir pour vous aider à vous repérer dans votre code source (surtout s'il est long).

```
<!DOCTYPE html>
<html>
  <head>
    <!-- En-tête
de la page -->
    <meta
charset="utf-8" />

    <title>Titre</title>
  </head>
```

|  |                                                                                           |
|--|-------------------------------------------------------------------------------------------|
|  | <pre> &lt;body&gt;     &lt;!-- Corps de la page --&gt; &lt;/body&gt; &lt;/html&gt; </pre> |
|--|-------------------------------------------------------------------------------------------|

### **Tout le monde peut voir vos commentaires... et tout votre code HTML !**

Terminons par une remarque importante : **tout le monde peut voir le code HTML de votre page** une fois celle-ci mise en ligne sur le Web. Il suffit de faire un clic droit sur la page et de sélectionner "Afficher le code source de la page" (l'intitulé peut changer selon votre navigateur).

## **CHAPITRE II COMMENT ORGANISER SON TEXTE**

Nous allons voir comment rédiger le contenu de notre page web dans ce chapitre. Comme nous l'avons vu, il ne faudra pas faire ça n'importe comment : il ne faut pas oublier qu'une page HTML est composée de balises. Ces balises indiquent à l'ordinateur le sens du texte : ceci est un paragraphe, ceci est un titre, etc.

Nous allons découvrir de nombreuses balises HTML dans ce chapitre. Certaines existent depuis la toute première version de HTML, d'autres ont été introduites plus récemment dans HTML5. Nous allons voir successivement dans ce chapitre :

- Comment rédiger des paragraphes.
- Comment structurer sa page avec les titres.
- Comment donner de l'importance à certains mots de son texte.
- Comment organiser les informations sous forme de liste à puces.

### **II.1 Les paragraphes**

La plupart du temps, lorsqu'on écrit du texte dans une page web, on le fait à l'intérieur de paragraphes. Le langage HTML propose justement la balise `<p>` pour délimiter les paragraphes.

```
1 <p>Bonjour et bienvenue sur mon site !</p>
```

- `<p>` signifie "Début du paragraphe"
- `</p>` signifie "Fin du paragraphe"

Comme indiqué dans le chapitre précédent, on écrit le contenu de notre site web entre les balises `<body></body>`. Il nous suffit donc de mettre notre paragraphe entre ces deux balises, et nous aurons enfin notre première vraie page web avec du texte !

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Paragraphe</title>
  </head>
  <body>
    <p>Bonjour et bienvenue sur mon site !</p>
  </body>
</html>
```

Nous allons voir maintenant quelque chose d'un peu particulier en HTML : le saut de lignes. Ça a l'air simple, mais pourtant ça ne fonctionne pas vraiment comme dans un traitement de texte habituel.

## Sauter une ligne

En HTML, si vous appuyez sur la touche Entrée, ça ne crée pas une nouvelle ligne comme vous en avez l'habitude. Essayez donc ce code :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Essais de sauts de ligne</title>
  </head>

  <body>
    <p>
      Bonjour et bienvenue sur mon site !
      Ceci est mon premier test, alors soyez indulgents s'il
vous plaît, j'apprends petit à petit comment ça marche.
      Pour l'instant c'est un peu vide, mais revenez dans 2-3
jours quand j'aurai appris un peu plus de choses.
    </p>
  </body>
```



```
</html>
```

Tout est sur la même ligne alors qu'on est pourtant allé à la ligne dans notre code ! Taper frénétiquement sur la touche Entrée dans l'éditeur de texte ne sert donc strictement à rien. Il y a pourtant bien un moyen de faire des sauts de ligne en HTML. En fait, si vous voulez écrire un deuxième paragraphe, il vous suffit d'utiliser une deuxième balise <p>. Votre code HTML devrait donc être au final rempli de balises de paragraphe ! Un exemple :

```
<html>
  <head>
    <meta charset="utf-8" />
    <title>Paragraphes</title>
  </head>

  <body>
    <p>
      Bonjour et bienvenue sur mon site !
      Ceci est mon premier test, alors soyez indulgents s'il
vous plaît, j'apprends petit à petit comment ça marche.
    </p>

    <p>
      Pour l'instant c'est un peu vide, mais revenez dans 2-3
jours quand j'aurai appris un peu plus de choses, je vous assure que
vous allez être surpris !
    </p>
  </body>
</html>
```

Il existe une balise "Aller à la ligne" !

C'est une balise orpheline qui sert juste à indiquer qu'on doit aller à la ligne : <br />. Vous devez obligatoirement la mettre **à l'intérieur d'un paragraphe**.

Voici comment l'utiliser dans un code :

```
<html>
  <head>
    <meta charset="utf-8" />
```

```

        <title>Sauts de ligne</title>
    </head>
    <body>
        <p>
            Bonjour et bienvenue sur mon site !<br />
            Ceci est mon premier test, alors soyez indulgents s'il
vous plaît, j'apprends petit à petit comment ça marche.
        </p>
        <p>
            Pour l'instant c'est un peu vide, mais revenez dans 2-3
jours quand j'aurai appris un peu plus de choses, je vous assure que
vous allez être surpris !
        </p>
    </body>
</html>

```

Vous pouvez théoriquement mettre plusieurs balises `<br />` d'affilée pour faire plusieurs sauts de lignes, mais on considère que c'est une mauvaise pratique qui rend le code délicat à maintenir. Pour décaler un texte avec plus de précision, on utilisera le CSS, ce langage qui vient compléter le HTML et dont on parlera un peu plus loin.

Donc c'est compris ?

- `<p> </p>` : pour organiser son texte en paragraphes.
- `<br />` : pour aller à la ligne.

## II.2 Les titres

Lorsque le contenu de votre page va s'étoffer avec de nombreux paragraphes, cela va devenir difficile pour vos visiteurs de se repérer. C'est là que les titres deviennent utiles.

En HTML, on a le droit d'utiliser 6 niveaux de titres différents. On a donc 6 balises de titre différentes :

- `<h1> </h1>` : signifie "titre très important". En général, on s'en sert pour afficher le titre de la page au début de celle-ci.
- `<h2> </h2>` : signifie "titre important".
- `<h3> </h3>` : pareil, c'est un titre un peu moins important (on peut dire un "sous-titre" si vous voulez).
- `<h4> </h4>` : titre encore moins important.
- `<h5> </h5>` : titre pas important.

- `<h6>` `</h6>` : titre vraiment, mais alors là vraiment pas important du tout.

Attention : ne confondez pas avec la balise `<title>` ! La balise `<title>` affiche le titre de la page dans la barre de titre du navigateur comme nous l'avons vu. Les titres `<h1>` et compagnie eux, servent à créer des titres qui seront affichés dans la page web.

Ne vous laissez pas impressionner par toutes ces balises. En fait, 6 niveaux de titres, c'est beaucoup. Dans la pratique, personnellement, je n'utilise que les balises `<h1>`, `<h2>` et `<h3>`, et très rarement les autres (je n'ai pas souvent besoin de 6 niveaux de titres différents). Votre navigateur affiche le titre très important en très gros, le titre un peu moins important en un peu moins gros, etc.

Ne choisissez pas votre balise de titre en fonction de la taille qu'elle procure au texte ! Il faut impérativement bien structurer sa page en commençant par un titre de niveau 1 (`<h1>`), puis un titre de niveau 2 (`<h2>`), etc. Il ne devrait pas y avoir de sous-titre sans titre principal ! Si vous voulez modifier la taille du texte, sachez que nous apprendrons à faire ceci en CSS un peu plus tard.

Essayez de faire une page web avec des titres pour voir ce que ça donne :

```
<html>
  <head>
    <meta charset="utf-8" />
    <title>Niveaux de titres</title>
  </head>
  <body>
    <h1>Titre super important</h1>
    <h2>Titre important</h2>
    <h3>Titre un peu moins important (sous-titre)</h3>
    <h4>Titre pas trop important</h4>
    <h5>Titre pas important</h5>
    <h6>Titre vraiment pas important du tout</h6>
  </body>
</html>
```

Voici un exemple d'utilisation des titres dans une page web :

```
<html>
<head>
  <meta charset="utf-8" />
  <title>Présentation de mon Site</title>
```

```

</head>
<body>
  <h1>Bienvenue sur mon Site!</h1>
  <p>
    Bonjour et bienvenue sur mon site : Mon premier site.
  </p>
  <h2>Des cours pour débutants</h2>
  <p>
    Ce site va être un bon début pour commencer à acquérir des
    connaissances en matière de la programmation Web!
  </p>
  <p>
    Vous pourrez ainsi apprendre, sans rien y connaître
    auparavant, à créer un site web, à programmer, .... !
  </p>
  <h2>Une communauté active</h2>
  <p>
    N'importe qui peut apprendre à créer un site web à l'aide de
    ce dernier.
  </p>
</body>
</html>

```

Et si je veux centrer mon titre, l'écrire en rouge et le souligner ?

Nous ferons tout cela lorsque nous apprendrons le CSS (dès la deuxième partie du cours). Il faut savoir que `<h1>` ne signifie pas "Times New Roman, taille 16 pt", mais "Titre important".

Grâce au langage CSS, vous pourrez dire "Je veux que mes titres importants soient centrés, rouges et soulignés". Pour le moment, en HTML, nous ne faisons que structurer notre page. **Nous rédigeons le contenu avant de nous amuser à le mettre en forme.**

## II.3 La mise en valeur

Au sein de vos paragraphes, certains mots sont parfois plus importants que d'autres et vous aimeriez les faire ressortir. HTML vous propose différents moyens de mettre en valeur le texte de votre page.

## Mettre un peu en valeur

Pour mettre un peu en valeur votre texte, vous devez utiliser la balise `<em>` `</em>`(Emphasis). Son utilisation est très simple : entourez les mots à mettre en valeur par ces balises, et c'est bon ! Je reprends un peu l'exemple de tout à l'heure, et j'y mets quelques mots en évidence :

```
<html>
<head>
  <meta charset="utf-8" />
  <title>Emphase</title>
</head>
<body>
  <p>
    Bonjour et bienvenue sur mon site !<br />

    Ceci est mon premier test, alors <em>soyez indulgents</em>
    s'il vous plaît, j'apprends petit à petit comment ça marche.
  </p>
</body>
</html>
```

Comme vous pouvez le voir, utiliser la balise `<em>` a pour conséquence de mettre le texte en italique. En fait, c'est le navigateur qui choisit comment afficher les mots. On lui dit que les mots sont assez importants et pour faire ressortir cette information, il change l'apparence du texte en utilisant l'italique.

## Mettre bien en valeur

Pour mettre un texte bien en valeur, on utilise la balise `<strong>` qui signifie "fort", ou "important" si vous préférez. Elle s'utilise exactement de la même manière que `<em>` :

```
<html>
<head>
  <meta charset="utf-8" />
  <title>Forte emphase</title>
</head>
<body>
  <p>
    Bonjour et bienvenue sur mon site !<br />
    Ceci est mon premier test, alors <strong>soyez
```

```
indulgents</strong> s'il vous plaît, j'apprends petit à petit
comment ça marche.
</p>
</body>
</html>
```

Vous voyez sûrement le texte s'affiche en gras. Là encore, le gras n'est qu'une conséquence. Le navigateur a choisi d'afficher en gras les mots importants pour les faire plus ressortir. La balise `<strong>` ne signifie pas "mettre en gras" mais "important". On pourra décider plus tard en CSS d'afficher les mots "importants" d'une autre façon que le gras si on le souhaite.

## Marquer le texte

La balise `<mark>` permet de faire ressortir visuellement une portion de texte. Le texte n'est pas forcément considéré comme important mais on veut qu'il se distingue bien du reste du texte. Cela peut être utile pour faire ressortir un texte pertinent après une recherche sur votre site par exemple.

```
<html>
<head>
  <meta charset="utf-8" />
  <title>Marquage du texte</title>
</head>
<body>
  <p>
    Bonjour et bienvenue sur mon site !<br />
    Ceci est mon premier test, alors <mark>soyez
indulgents</mark> s'il vous plaît, j'apprends petit à petit comment
ça marche.
  </p>
</body>
</html>
```

Par défaut, `<mark>` a pour effet de surligner le texte. On pourra changer l'affichage en CSS (décider de surligner dans une autre couleur, décider d'encadrer le texte, etc.). C'est le même principe que ce qu'on disait pour les balises précédentes : elles indiquent le sens des mots et non pas comment ceux-ci doivent s'afficher.

## N'oubliez pas : HTML pour le fond, CSS pour la forme

Je vais peut-être vous sembler un peu lourd mais il est très important qu'on se comprenne bien, car les débutants font souvent la même grosse erreur à ce stade. Ils ont vu les balises `<em>`, `<strong>`, `<mark>`... et ils se disent qu'ils ont découvert comment mettre en italique, en gras et comment surligner du texte en HTML !".

Et pourtant... ce n'est pas à ça que servent ces balises ! Quand on utilise `<strong>` le texte apparaît en gras, donc c'est pour mettre en gras.", et pourtant, c'est une erreur de croire que cette balise sert à ça. Le rôle des balises est d'indiquer le sens du texte. Ainsi, `<strong>` indique à l'ordinateur "Ce texte est important". C'est tout.

Et pour montrer que le texte est important, l'ordinateur décide de le mettre en gras (mais il pourrait aussi bien l'écrire en rouge !). La plupart des navigateurs affichent les textes importants en gras, mais rien ne les y oblige.

De nombreux programmes analysent les codes source des pages web, à commencer par les robots de moteurs de recherche. Ces robots parcourent le web en lisant le code HTML de tous les sites. C'est le cas des robots de Google et de Bing par exemple. Les mots-clés "importants" ont tendance à avoir plus de valeur à leurs yeux, donc si quelqu'un fait une recherche sur ces mots il a plus de chances de tomber sur votre site.

Bien entendu c'est une explication grossière, et il ne faut pas croire qu'utiliser la balise `<strong>` à tout-va améliorera votre référencement. Il faut simplement faire confiance aux ordinateurs : ils comprennent ce qu'un texte "important" veut dire et peuvent se servir de cette information.

### ***Quelques balises plus rares (mais utiles !)***

Je sais que vous avez appris pas mal de nouvelles balises dans ce chapitre, mais c'est nécessaire si vous voulez réussir votre site web.

Contrairement à ce qu'il paraît, c'est facile à retenir et ça vient très rapidement avec un peu de pratique. Si les balises que je vous ai montrées jusqu'ici sont très fréquemment utilisées (paragraphes, titres, mise en valeur...), celles que je vais vous présenter ci-dessous sont un peu plus rarement utilisées. Comme elles sont plus rares, je ne vous oblige pas à les retenir par cœur.

Je ne vous présente pas toutes les balises qui existent dont on pourrait se servir sur notre texte, ça serait trop long, et des fois la différence est minime.

Nous allons voir les balises suivantes :

- Mettre en exposant ou en indice
- Les citations
- Les acronymes

### **Mettre en exposant ou en indice**

Pour mettre en exposant certains mots ou caractères, on utilise la balise `<sup>`.

Les mots seront alors affichés en hauteur. Par exemple :

## Code : HTML

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr" >
  <head>
    <title>Un peu d'histoire...</title>
    <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
  </head>
  <body>
    <h1>Un peu d'histoire...</h1>
    <p>
      <em>Le saviez-vous ?</em><br />
      Le langage XHTML est le langage qui nous permet de créer des sites web en ce début de
      XXI<sup>ème</sup> siècle.<br />
      Avant, vers la fin du XX<sup>ème</sup>, on utilisait le langage HTML, qui disparaît
      aujourd'hui peu à peu au profit de XHTML
    </p>
  </body>
</html>
```

Pour mettre en indice (c'est-à-dire pour écrire le texte en bas de la ligne en petit), on utilise cette fois la balise **<sub>**.

Bon, à partir de maintenant je ne vous donnerai pas toujours le code entier de la page (c'est un peu lourd à chaque fois), là je vous donne juste la partie du code qui nous intéresse. Ici, j'utilise la mise en indice pour écrire une formule mathématique :

```
<p>
  x<sub>n</sub> = x<sub>n - 1</sub> - 2x<sub>n-2</sub>
</p>
```

## Les citations

On peut faire 2 types de citations :

- Des citations courtes, dans un paragraphe : les citations courtes s'effectuent à l'intérieur d'un paragraphe. On encadre la citation par la balise **<q>**, qui fonctionne comme **<em>**, **<strong>**, **<sup>** etc... Voici un exemple de citation courte au milieu d'un paragraphe :

### Code : HTML

```
<p>Vous souvenez-vous de la phrase célèbre qu'a prononcé Neil Armstrong en posant le
premier pied sur la Lune ? <q>C'est un petit pas pour l'Homme, un grand pas pour
l'Humanité</q>. C'était un certain 20 Juillet 1969...</p>
```

- La citation, sur un navigateur récent comme Mozilla Firefox, est entourée de guillemets. C'est logique me direz-vous, mais IE lui ne fait rien de particulier (essayez sur Firefox et IE, vous allez voir qu'il y a une différence !). Sous IE donc, la balise **<q>** n'a aucun effet particulier. Mais ne vous en faites pas, vous pourrez en CSS décider de la façon dont vous voulez afficher les citations courtes (en couleur, en italique etc...)
- Des citations longues, hors d'un paragraphe : si vous avez besoin d'effectuer une citation assez longue, vous devrez utiliser la balise **<blockquote>**. Vous devez mettre des balises de paragraphe **<p>** à l'intérieur du **blockquote** comme ceci :



```

<!DOCTYPE html>
<html >
  <head>
    <title>Bienvenue sur mon site !</title>
    <meta charset=utf-8" />
  </head>
  <body>
    <h1>Le Corbeau et le Renard</h1>
    <p>Voici ce qui est sans aucun doute une des plus connues fables de <em>La Fontaine</em>
: </p>
    <blockquote>
      <p>
        Maître Corbeau, sur un arbre perché,<br />
        Tenait en son bec un fromage.<br />
        Maître Renard, par l'odeur alléché,<br />
        Lui tint à peu près ce langage :<br />
        "Hé ! bonjour, Monsieur du Corbeau.<br />
        Que vous êtes joli ! que vous me semblez beau !<br />
        Sans mentir, si votre ramage<br />
        Se rapporte à votre plumage,<br />
        Vous êtes le Phénix des hôtes de ces bois."<br />
        A ces mots le Corbeau ne se sent pas de joie ;<br />
        Et pour montrer sa belle voix,<br />
        Il ouvre un large bec, laisse tomber sa proie.<br />
        Le Renard s'en saisit, et dit : "Mon bon Monsieur,<br />
        Apprenez que tout flatteur<br />
        Vit aux dépens de celui qui l'écoute :<br />
        Cette leçon vaut bien un fromage, sans doute. "<br />
        Le Corbeau, honteux et confus,<br />
        Jura, mais un peu tard, qu'on ne l'y prendrait plus.
      </p>
    </blockquote>
  </body>
</html>

```

- Vous remarquez que les citations longues sont mises légèrement en décalé sur la droite.

Les balises `<q>` et `<blockquote>` servent toutes deux à effectuer une citation.  
En HTML, on distingue 2 types de balises :

- Les balises de type inline : ce sont des balises que l'on utilise à l'intérieur d'un paragraphe. C'est le cas notamment des balises `<strong>`, `<em>`, `<q>` etc...
- Les balises de type bloc : ces balises servent à structurer la page en plusieurs "blocs". La première qu'on a appris, c'est la balise `<p>`, mais il y a aussi les balises de titre `<h1>`, `<h2>`, `<blockquote>`... Vous ne devez pas mettre de titre ou de citation longue à l'intérieur d'un paragraphe. Chacune de ces balises constitue un bloc séparé.

## Les acronymes

Un acronyme est une suite d'initiales utilisée généralement pour raccourcir certains noms, comme par exemple XHTML, FBI, CSA etc...

Le problème des acronymes, c'est qu'on ne sait pas toujours ce qu'ils signifient. Pour que le visiteur sache de quoi il s'agit, vous devez entourer votre acronyme de la balise `<acronym>`

La nouveauté ici, c'est que vous allez devoir utiliser un attribut (title) pour expliquer ce que signifie l'acronyme.

Comme un exemple vaut toujours mieux qu'un long discours :

### **Code : HTML**

```
<p>Le <acronym title="eXtensible HyperText Markup Language">XHTML</acronym> est le langage qui permet de créer un site web. </p>
```

Comme vous pouvez le voir, si vous pointez la souris sur XHTML, la description de l'acronyme apparaît.

Vous noterez encore une petite différence entre IE et Firefox : Firefox souligne en pointillés l'acronyme, tandis qu'IE... ne fait rien

## **II.4 Les listes à puces**

Les listes à puces nous permettent souvent de mieux structurer notre texte et d'ordonner nos informations.

Nous allons découvrir ici deux types de listes à puces :

- Les listes non ordonnées
- Les listes ordonnées

### **Liste non ordonnée**

Une liste non ordonnée ressemble à ceci :

- Sciences de la communication
- Gestion et Administration
- Informatique de Gestion

C'est un système qui nous permet de faire une liste d'éléments, sans notion d'ordre (il n'y a pas de "premier" ni de "dernier"). Créer une liste à puces non ordonnée est très simple. Il suffit d'utiliser la balise `<ul>` que l'on referme un peu plus loin avec un `</ul>`.

Commencez donc à taper ceci :

```
<ul></ul>
```

Et maintenant, voilà ce qu'on va faire : on va écrire chacun des éléments de la liste entre 2 balises `<li></li>`. Toutes ces balises doivent se trouver entre `<ul>` et `</ul>`.

Vous allez comprendre de suite avec cet exemple :

```
<ul>
<li> Sciences de la communication</li>
<li> Gestion et Administration</li>
<li> Informatique de Gestion</li>
</ul>
```

Notez que la liste doit être placée à l'intérieur de `<body></body>`. Je ne mets pas tout le code de la page à partir de maintenant pour rester lisible.

Retenez donc ces deux balises :

- `<ul></ul>` délimite toute la liste.
- `<li></li>` délimite un élément de la liste (une puce).

Vous pouvez mettre autant d'éléments que vous voulez dans la liste à puces, vous n'êtes pas limités à 3 éléments bien entendu.

Et voilà, vous savez créer une liste à puce non ordonnée ! Pas si dur une fois qu'on a compris comment imbriquer les balises.

Pour ceux qui ont besoin de faire des listes complexes, sachez que vous pouvez imbriquer des listes à puces (créer une liste à puces dans une liste à puces). Si vous voulez faire ça, ouvrez une seconde balise `<ul>` à l'intérieur d'un élément `<li></li>`.

Si vous fermez les balises dans le bon ordre, vous n'aurez pas de problème. Attention néanmoins, cette technique est un peu compliquée à maîtriser.

## Liste ordonnée

Une liste ordonnée fonctionne de la même façon, seule une balise change : il faut remplacer `<ul></ul>` par `<ol></ol>`.

À l'intérieur de la liste, on ne change rien : on utilise toujours des balises `<li></li>` pour délimiter les éléments.

L'ordre dans lequel vous mettez les éléments de la liste est important. Le premier `<li></li>` sera l'élément n°1, le second sera le n°2 etc...

Comme c'est particulièrement intuitif, je vous laisse admirer la simplicité de cet exemple :

```
<h1>Ma journée</h1>
<ol>
  <li>Je me lève</li>
  <li>Je mange et je bois</li>
  <li>Je retourne me coucher</li>
</ol>
```

Par rapport à l'exemple précédent, tout ce qu'on a eu à changer est donc la balise `<ol>`. Pour information, il existe un troisième type de liste, beaucoup plus rare : la liste de définitions. Elle

fait intervenir les balises <dl> (pour délimiter la liste), <dd> (pour délimiter un terme) et <dt> (pour délimiter la définition de ce terme).

## CHAPITRE III : COMMENT CREER DES LIENS ET INSERER DES IMAGES

Un site web est composé de plusieurs pages. Comment faire pour aller d'une page vers une autre ? Dans ce chapitre, nous allons justement apprendre à créer des liens entre nos pages. Un lien est un texte sur lequel on peut cliquer pour se rendre sur une autre page. On peut faire un lien d'une page a.html vers une page b.html, mais on peut aussi faire un lien vers un autre site (ex. : <http://www.yahoo.com>). Dans les 2 cas, nous allons voir que le fonctionnement est le même.

### III.1 Comment créer Un lien vers un autre site

Il est facile de reconnaître les liens sur une page : ils sont écrits d'une façon différente (par défaut en bleu souligné) et un curseur en forme de main apparaît lorsqu'on pointe dessus. Pour faire un lien, la balise que nous allons utiliser est très simple à retenir : `<a>`. Il faut cependant lui ajouter un attribut, `href`, pour indiquer vers quelle page on souhaite amener. Voici un lien qui amène vers le Site de Google, situé à l'adresse <http://www.google.com> :

```
<a href="http://www.google.com"> Site de Google </a>
```

Nous allons placer ce lien au sein d'un paragraphe. Voici donc comment reproduire l'exemple de l'image précédente :

```
<p>Bonjour.      Souhaitez-vous      visiter      le      <ahref="http://www.
www.google.com ">Site de Google </a> ?<br />
C'est un bon site !</p>
```

Par défaut, le lien s'affiche en bleu souligné. Si vous avez déjà visité la page, le lien s'affiche en violet. Nous verrons comment changer cette apparence lorsque nous étudierons le CSS.

Si vous voulez faire un lien vers un autre site, il suffit donc de copier son adresse (on parle d'URL) en `http://`. Notez que certains liens commencent parfois par `https://` (sites sécurisés) ou d'autres préfixes (`ftp://...`).

Les liens que nous venons de voir sont appelés **liens absolus**, car on indique l'adresse complète. Nous allons maintenant voir que l'on peut écrire les liens d'une façon un peu différente, ce qui va nous être utile pour faire des liens entre les pages de notre site.

### III.2 Comment créer un lien vers une autre page de son site

Nous venons d'apprendre à créer des liens vers des sites déjà existants. Mais on peut créer des liens entre les différentes pages d'un site, non ?

En effet, pour le moment vous êtes en train de créer votre site sur votre ordinateur. Vous êtes le seul à pouvoir le voir, et il n'a pas encore "d'adresse web" qui commence en http:// comme la plupart des sites. Heureusement, cela ne va pas nous empêcher de travailler.

### III.2.1 Deux pages situées dans un même dossier

Pour commencer, nous allons créer 2 fichiers correspondant à 2 pages HTML différentes. Appelons-les page1.html et page2.html. Nous aurons donc ces 2 fichiers sur notre disque **dans le même dossier**. Voici le code que nous allons utiliser dans nos fichiers page1.html et page2.html.

#### page1.html

```
<p>Bonjour. Souhaitez-vous visiter <a href="page2.html">la page  
2</a> ?</p>
```

#### page2.html

La page 2 (page d'arrivée) affichera simplement un message pour indiquer que l'on est bien arrivé sur la page 2 :

```
<h1>Bienvenue sur la page 2 !</h1>
```

### III.2.2 Deux pages situées dans des dossiers différents

Les choses se corsent un petit peu si les pages sont situées dans des dossiers différents. Idéalement, elles ne devraient pas être trop loin l'une de l'autre (dans un sous-dossier par exemple). Imaginons que page2.html se trouve dans un sous-dossier appelé contenu :

Dans ce cas de figure, il va falloir faire un lien comme ceci :

#### Code : HTML

```
<a href="contenu/page2.html">
```

S'il y avait plusieurs sous-dossiers, on écrirait ceci :

```
<a href="contenu/autredossier/page2.html">
```

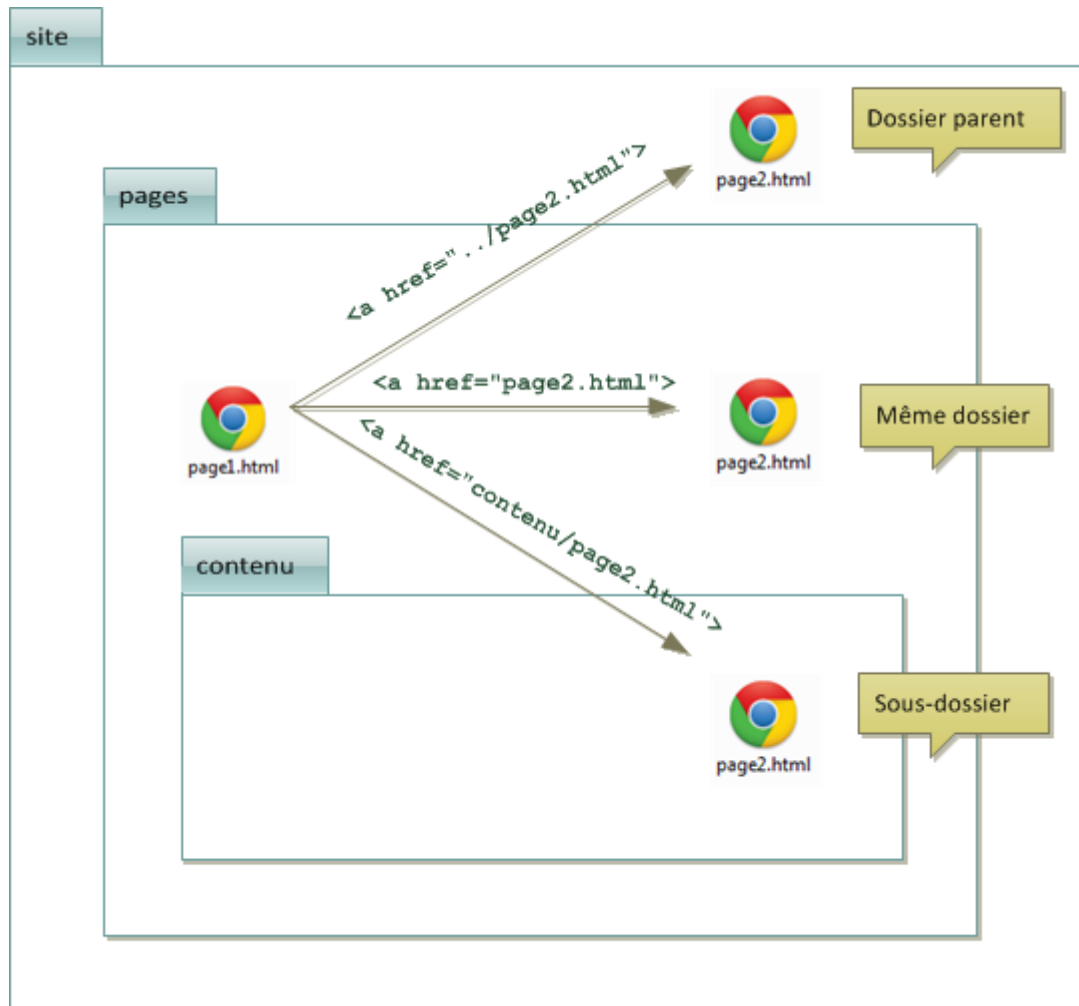
Et si le fichier ne se trouve pas dans un sous-dossier mais dans un dossier "parent", on fait comment ?

Si votre fichier cible est placé dans un dossier qui se trouve "avant" dans l'arborescence, il faut écrire deux points comme ceci :

```
<a href="../page2.html">
```

## Résumé en images

Les liens relatifs ne sont pas bien compliqués à utiliser une fois qu'on a compris le principe. Il suffit de regarder dans quel "niveau de dossier" se trouve votre fichier cible pour savoir comment écrire votre lien :



### III.3 Un lien vers une ancre

Une **ancre** est une sorte de point de repère que vous pouvez mettre dans vos grosses pages HTML. En effet, si votre page est très grande il peut être utile de faire un lien amenant plus bas dans la même page pour que le visiteur puisse sauter directement à la partie qui l'intéresse. Pour créer une ancre, il suffit de rajouter l'attribut id à une balise qui va alors servir de repère. Ce peut être n'importe quelle balise, un titre par exemple.

Utilisez l'attribut id pour donner un nom à l'ancre. Cela nous servira ensuite pour faire un lien vers cette ancre. Par exemple :

```
<h2 id="mon_ancre">Titre</h2>
```

Ensuite, il suffit de faire un lien comme d'habitude, mais cette fois l'attribut href contiendra un dièse (#) suivi du nom de l'ancre. Exemple :

## Code : HTML

```
<a href="#mon_ancre">Aller vers l'ancre</a>
```

Normalement, si vous cliquez sur le lien, cela vous amènera plus bas dans la même page (à condition que la page comporte suffisamment de texte pour que les barres de défilement se déplacent automatiquement).

Voici un exemple de page comportant beaucoup de texte et utilisant les ancres:

```
<h1>Ma grande page</h1>
<p>
  Aller directement à la partie traitant de :<br />
  <a href="#cuisine">La cuisine</a><br />
  <a href="#rollers">Les rollers</a><br />
  <a href="#arc">Le tir à l'arc</a><br />
</p>
<h2 id="cuisine">La cuisine</h2>
<p>... (beaucoup de texte) ...</p>
<h2 id="rollers">Les rollers</h2>
<p>... (beaucoup de texte) ...</p>
<h2 id="arc">Le tir à l'arc</h2>
<p>... (beaucoup de texte) ...</p>
```

S'il ne se passe rien quand vous cliquez sur les liens, c'est qu'il n'y a pas assez de texte. Dans ce cas, vous pouvez soit rajouter du blabla dans la page pour qu'il y ait (encore) plus de texte, ou bien réduire la taille de la fenêtre de votre navigateur pour faire apparaître les barres de défilement sur le côté.

L'attribut id sert à donner un nom "unique" à une balise, pour s'en servir de repère. Ici, on s'en sert pour faire un lien vers une ancre, mais en CSS cela nous sera très utile pour "repérer" une balise précise, vous verrez.

Evitez cependant de créer des id avec des espaces ou des caractères spéciaux, utilisez simplement des lettres et chiffres dans la mesure du possible pour que cela soit reconnu par tous les navigateurs.

## Lien vers une ancre située dans une autre page

L'idée, c'est de faire un lien qui ouvre une nouvelle page ET qui amène directement à une ancre située plus bas sur cette page.

En pratique c'est assez simple à faire : il suffit de taper le nom de la page, suivi d'un dièse (#), suivi du nom de l'ancre.

Par exemple : `<a href="ancres.html#rollers">...` vous amènera sur la page `ancres.html`, directement au niveau de l'ancre appelée "rollers".



Voici une page qui contient 3 liens, chacun amenant vers une des ancres de la page de l'exemple précédent :

```
<h1>Le Mégamix</h1>
<p>
  Rendez-vous quelque part sur une autre page :<br />
  <a href="ancres.html#cuisine">La cuisine</a><br />
  <a href="ancres.html#rollers">Les rollers</a><br />
  <a href="ancres.html#arc">Le tir à l'arc</a><br />
</p>
```

### III.4 Cas pratiques d'utilisation des liens

#### 1. Un lien qui affiche une info bulle au survol

Vous pouvez utiliser l'attribut `title` qui affiche une bulle d'aide lorsqu'on pointe sur le lien. Cet attribut est facultatif.

La bulle d'aide peut être utile pour informer le visiteur avant même qu'il n'ait cliqué sur le lien.

##### Code : HTML

```
<p>Bonjour. Souhaitez-vous visiter le <a
href="http://www.google.com" title="Bon site pour les
chercheurs">Site de Google </a> ?</p>
```

#### 2. Un lien qui ouvre une nouvelle fenêtre

Il est possible de "forcer" l'ouverture d'un lien dans une nouvelle fenêtre. Pour cela, on rajoutera `target="_blank"` à la balise `<a>` :

##### Code : HTML

```
<p>Bonjour. Souhaitez-vous visiter le <a href="http://www.
google.com" target="_blank">Site de Google</a> ?<br />
Le site s'affichera dans une autre fenêtre.</p>
```

Selon la configuration du navigateur, la page s'affichera dans une nouvelle fenêtre ou un nouvel onglet.

Vous ne pouvez pas choisir entre l'ouverture d'une nouvelle fenêtre ou d'un nouvel onglet.

Notez cependant qu'il est déconseillé d'abuser de cette technique car elle perturbe la navigation. Le visiteur lui-même peut décider s'il veut ouvrir le lien dans une nouvelle fenêtre. Il fera Maj + Clic sur le lien pour ouvrir dans une nouvelle fenêtre, ou Ctrl + Clic pour ouvrir dans un nouvel onglet.

### 3. Un lien pour envoyer un e-mail

Si vous voulez que vos visiteurs puissent vous envoyer un mail, vous pouvez utiliser des liens de type "mailto". Rien ne change au niveau de la balise, vous devez simplement modifier la valeur de l'attribut href comme ceci :

```
<p><a href="mailto:votrenom@bidule.com">Envoyez-moi un e-mail  
!</a></p>
```

Il suffit donc de faire commencer le lien par "mailto:" et d'écrire l'adresse e-mail où on peut vous contacter. Si vous cliquez sur le lien, un nouveau message vide s'ouvre, prêt à être envoyé à votre adresse e-mail.

### 4. Un lien pour télécharger un fichier

Beaucoup d'entre vous se demandent comment ça se passe pour le téléchargement d'un fichier... En fait, il faut faire exactement comme si vous faisiez un lien vers une page web, mais en indiquant cette fois le nom du fichier à télécharger. Par exemple, supposez que vous vouliez faire télécharger monfichier.zip. Placez simplement ce fichier dans le même dossier que votre page web (ou dans un sous-dossier) et faites un lien vers ce fichier :

#### Code : HTML

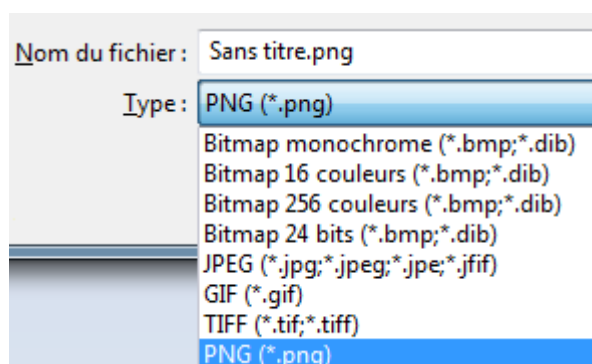
```
<p><a href="monfichier.zip">Télécharger le fichier</a></p>
```

## III.5 Les différents formats d'images

Il existe différents formats d'image que l'on peut utiliser sur des sites web, et on ne doit pas les choisir au hasard. En effet, les images sont parfois volumineuses à télécharger, ce qui ralentit le temps de chargement de la page (beaucoup plus que le texte !).

Quand vous avez une image "entre les mains", vous avez la possibilité de l'enregistrer dans plusieurs "formats" différents. Le poids (en Ko, voire en Mo) de l'image sera plus ou moins élevé selon le format choisi, et la qualité de l'image va changer.

Par exemple, le logiciel de dessin Paint (même si c'est loin d'être le meilleur) vous propose de choisir entre plusieurs formats lorsque vous enregistrez une image :



Certains formats sont plus adaptés que d'autres selon l'image (photo, dessin, image animée...). Notre but ici est de faire le tour des différents formats que l'on utilise sur le web, pour que vous les connaissiez et sachiez choisir celui qui convient le mieux à votre image. Rassurez-vous, il n'y a pas beaucoup de formats différents, ça ne sera donc pas bien long.

Toutes les images diffusées sur Internet ont un point commun : elles sont **compressées**. Cela veut dire que l'ordinateur fait des calculs pour qu'elles soient moins lourdes et donc plus rapides à charger.

## **Le JPEG**

Les images au format JPEG (Joint Photographic Expert Group) sont très répandues sur le Web. Ce format est conçu pour réduire la taille des photos, qui peuvent comporter plus de 16 millions de couleurs différentes.

Les images JPEG sont enregistrées avec l'extension .jpg ou .jpeg.

Notez que le JPEG détériore un peu la qualité de l'image d'une façon généralement imperceptible. C'est ce qui le rend si efficace pour réduire le poids des photos.

Quand il s'agit d'une photo, on ne peut généralement pas détecter la perte de qualité. Par contre, si ce n'est pas une photo, vous risquez de voir l'image un peu se détériorer. Dans ce cas, il vaut mieux utiliser le format PNG.

## **Le PNG**

Le format PNG (Portable Network Graphics) est le plus récent de tous. Ce format est adapté à la plupart des graphiques (je serais tenté de dire "à tout ce qui n'est pas une photo"). Le PNG a deux gros avantages : il peut être rendu transparent et il n'altère pas la qualité de l'image. Le PNG a été inventé pour concurrencer un autre format, le GIF, à l'époque où il fallait payer pour pouvoir utiliser des GIF. Depuis, le PNG a bien évolué et c'est devenu le format le plus puissant pour enregistrer la plupart des images.

Le PNG existe en 2 versions, en fonction du nombre de couleurs que doit comporter l'image :

- PNG 8 bits : 256 couleurs
- PNG 24 bits : 16 millions de couleurs (autant qu'une image JPEG)

Une vieille version d'Internet Explorer (IE6) ne peut pas afficher correctement les images PNG 24 bits transparentes. Ce navigateur tend à être de moins en moins utilisé, mais gardez quand même cette information en tête.

Au fait, si le PNG 24 bits peut afficher autant de couleurs qu'une image JPEG, et qu'en plus il peut être rendu transparent sans modifier la qualité de l'image... quel est l'intérêt du JPEG ?

La compression du JPEG est plus puissante sur les photos. Une photo enregistrée en JPEG se chargera toujours beaucoup plus vite que si elle était enregistrée en PNG. C'est conseillé donc de toujours réserver le format JPEG aux photos.

## Le GIF

C'est un format assez vieux, qui a été néanmoins très utilisé (et qui reste très utilisé par habitude). Aujourd'hui, le PNG est globalement bien meilleur que le GIF : les images sont le plus souvent plus légères et la transparence est de meilleure qualité. Je vous recommande donc d'utiliser le PNG autant que possible.

Le format GIF est limité à 256 couleurs (alors que le PNG peut aller jusqu'à plusieurs millions de couleurs).

Il existe un format adapté à chaque image

Si on résume, voici quel format adopter en fonction de l'image que vous avez :

- **Une photo** : utilisez un JPEG.
- **N'importe quel graphique avec peu de couleurs** (moins de 256) : utilisez un PNG 8 bits, ou éventuellement un GIF.
- **N'importe quel graphique avec beaucoup de couleurs** : utilisez un PNG 24 bits.
- **Une image animée** : utilisez un GIF animé.

## Bannissez les autres formats

Les autres formats non cités ici, comme le format BITMAP (\*.bmp) sont à bannir car bien souvent ils ne sont pas compressés, donc trop gros. Ils ne sont pas du tout adaptés au web. On peut en mettre sur son site, mais le chargement sera vraiment extrêmement long !

## Choisissez bien le nom de votre image

Si vous voulez éviter des problèmes, prenez l'habitude d'enregistrer vos fichiers avec des noms en minuscules, sans espaces ni accents. Par exemple : mon\_image.png.

Vous pouvez remplacer les espaces par le caractère "underscore" comme je l'ai fait : \_ .

## Insertion d'une image

Quelle est la fameuse balise qui va nous permettre d'insérer une image ? Il s'agit de... `<img />`  
C'est une balise de type orpheline (comme `<br />`). Cela veut dire qu'on n'a pas besoin de l'écrire en deux exemplaires comme la plupart des autres balises que nous avons vues jusqu'ici. En effet, nous n'avons pas besoin de délimiter une portion de texte, nous voulons juste insérer une image à un endroit précis.

La balise doit être accompagnée de 2 attributs obligatoires :

- **src** : il permet d'indiquer où se trouve l'image que l'on veut insérer. Vous pouvez soit mettre un chemin en absolu (ex. : `http://www.site.com/fleur.png`), soit mettre le chemin en relatif (ce qu'on fait le plus souvent). Ainsi, si votre image est dans un sous-dossier images vous devrez taper : `src="images/fleur.png"`
- **alt** : cela signifie "texte alternatif". On doit **toujours** indiquer un texte alternatif à l'image, c'est-à-dire un court texte qui décrit ce que contient l'image. Ce texte sera affiché à la place de l'image si celle-ci ne peut pas être téléchargée (ça arrive), ou sur les navigateurs de personnes handicapées (non-voyants) qui ne peuvent malheureusement pas "voir" l'image. Cela aide aussi les robots des moteurs de recherche pour les recherches d'images. Pour la fleur, on mettrait par exemple : `alt="Une fleur"`.

Les images doivent se trouver obligatoirement à l'intérieur d'un paragraphe (`<p></p>`). Voici un exemple d'insertion d'image :

```
<p>
    Voici une photo que j'ai prise lors de mes dernières vacances à
    la montagne :<br />
    
</p>
```

Bref, l'insertion d'image est quelque chose de très facile pour peu qu'on sache indiquer où se trouve. Evitez à tout prix les accents, majuscules et espaces dans vos noms de fichiers et de dossiers. Voici un chemin qui va poser problème :

"Images du site/Image toute bête.jpg"

Il faudrait supprimer les espaces (ou les remplacer par le symbole "\_"), supprimer les accents et tout mettre en minuscules comme ceci :

"images\_du\_site/image\_toute\_bete.jpg"

Sachez donc que si votre image ne s'affiche pas, c'est très certainement parce que le chemin est incorrect ! Simplifiez au maximum vos noms de fichiers et de dossiers et tout ira bien.

## Ajouter une infobulle

L'attribut fait pour afficher une bulle d'aide est le même que pour les liens : il s'agit de `title`. Cet attribut est facultatif (contrairement à `alt`).

```
<p>
    Voici une photo que j'ai prise lors de mes dernières vacances à
    la montagne :<br />
    
```

</p>

Survolez la photo avec la souris pour voir l'infobulle apparaître.

## Images réactives

Comme le texte, une image peut très bien servir de lien. Au lieu de mettre du texte entre les balises <a></a>, vous pouvez très bien mettre une balise <img /> !

**Par exemple :**

**Code : HTML**

```
<p>
1 Vous souhaitez vous rendre vers un endroit magique ? Rien de plus simple, cliquez sur l'image ci-
2 dessous :<br />
3 <a href="http://www.siteduzero.com"></a>
</p>
```

L'ennui, c'est qu'un cadre bleu (ou violet) bien moche s'affiche autour de votre image cliquable. Heureusement, on va pouvoir "enlever" ce cadre grâce à CSS.

## III.6 Les figures

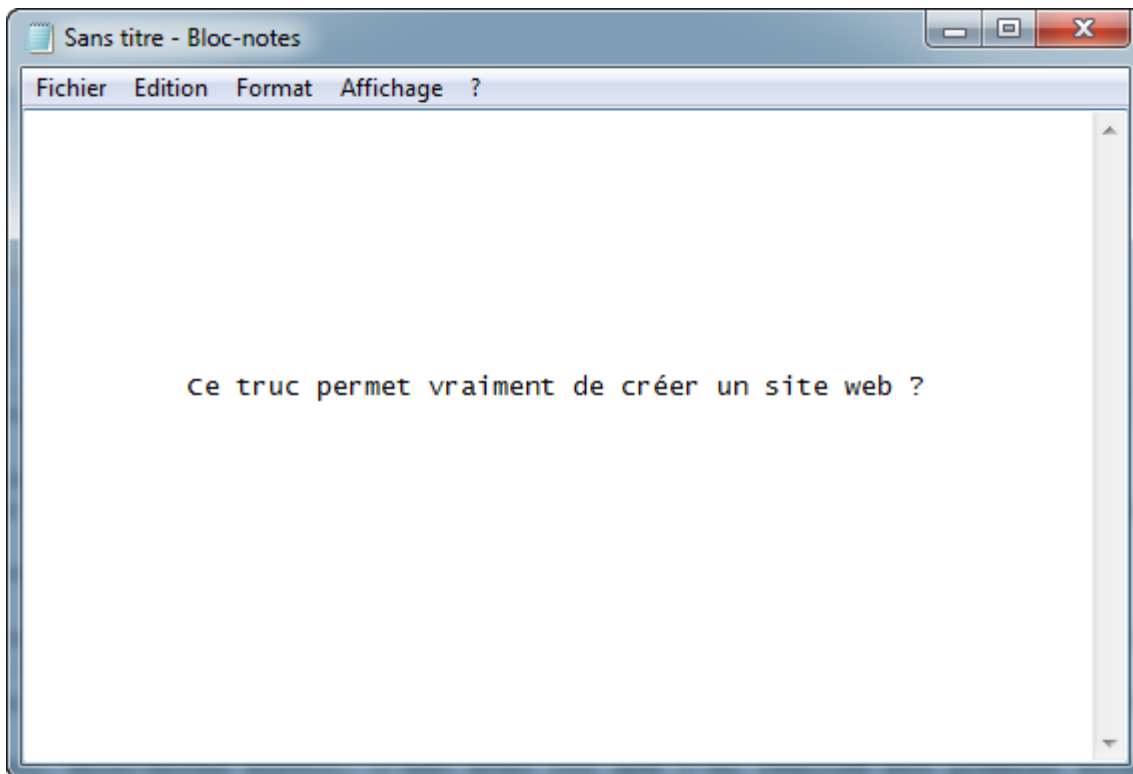
Ce sont des éléments qui viennent enrichir le texte pour compléter les informations de la page.

Les figures peuvent être de différents types :

- Images
- Codes source
- Citations
- ...

Bref, tout ce qui vient illustrer le texte est une figure. Nous allons ici nous intéresser aux images, mais contrairement à ce qu'on pourrait croire, les figures ne sont pas forcément des images : un code source aussi illustre le texte.

## Création d'une figure



Le logiciel Bloc-Notes

En HTML5, on dispose de la balise `<figure>`. Voici comment on pourrait l'utiliser :

```
<figure>
    
</figure>
```

Une figure est le plus souvent accompagnée d'une légende. Pour ajouter une légende, utilisez la balise `<figcaption>` à l'intérieur de la balise `<figure>`, comme ceci :

```
<figure>
    
    <figcaption>Le logiciel Bloc-Notes</figcaption>
</figure>
```

### Bien comprendre le rôle des figures

Si vous faites de votre image une figure, l'image peut être située en-dehors d'un paragraphe.

```
<p>Connaissez-vous le logiciel Bloc-Notes ? On peut faire des sites
web avec !</p>
```

```
<figure>
    
    <figcaption>Le logiciel Bloc-Notes</figcaption>
```

</figure>

Quand doit-on placer une image dans un paragraphe et quand doit-on la placer dans une figure ?

Tout dépend de ce que votre image apporte au texte :

- Si elle n'apporte aucune information (c'est juste une illustration pour décorer) : placez l'image dans un paragraphe.
- Si elle apporte une information : placez l'image dans une figure.

La balise <figure> a un rôle avant tout sémantique. Cela veut dire qu'elle indique à l'ordinateur que l'image a du sens et qu'elle est importante pour la bonne compréhension du texte. Cela peut permettre à un programme de récupérer toutes les figures du texte et de les référencer dans une table des figures par exemple.

Enfin, sachez qu'une figure peut très bien comporter plusieurs images. Voici un cas où cela se justifie :

**Code : HTML -**

```
<figure>
  
  
  
  <figcaption>Logos des différents navigateurs</figcaption>
</figure>
```



## CHAPITRE IV METTRE EN PLACE LE CSS

Le CSS comporte beaucoup de propriétés, qui sont un peu l'équivalent des balises en HTML. Heureusement, personne ne vous demande de tout retenir par cœur.

Dans ce premier chapitre sur le CSS, nous allons voir la théorie sur le CSS : qu'est-ce que c'est ? À quoi ça ressemble ? Où est-ce qu'on écrit du code CSS ? Cette théorie n'est pas bien compliquée, mais vous devez obligatoirement la connaître car c'est la base du CSS.

### IV 1. Petit rappel : à quoi sert CSS ?

CSS ? C'est lui qui vous permet de choisir la couleur de votre texte.

Lui qui vous permet de sélectionner la police utilisée sur votre site.

Lui encore qui permet de définir la taille du texte, les bordures, le fond...

Et aussi, c'est lui qui permet de faire la mise en page de votre site. Vous pourrez dire : je veux que mon menu soit à gauche et occupe telle largeur, que l'en-tête de mon site soit calé en haut et qu'il soit toujours visible, etc.

### CSS : des débuts difficiles

Il faut savoir qu'au tout début du Web, CSS n'existait pas. En fait, au départ, il n'y avait que le langage HTML.

Le HTML est né en 1991, et CSS est né en 1996. Alors, vous vous dites sûrement : comment faisait-on la mise en forme de 1991 à 1996 ? Eh bien, uniquement en HTML ! Il y avait en effet des balises HTML dédiée à la mise en forme. `<font color="#aab1c3">` par exemple, permettait de définir la couleur du texte.

Cependant, les pages HTML commençaient à devenir assez complexes. Il y avait de plus en plus de balises et c'était un joyeux mélange entre le fond et la forme, qui rendait la mise à jour des pages web de plus en plus complexe. C'est pour cela que l'on a créé le langage CSS. Cependant, le CSS n'a pas été adopté par les webmasters immédiatement, loin de là. Il fallait se défaire de certaines mauvaises habitudes, et ça a pris du temps. Encore aujourd'hui on peut trouver des sites web avec des vieilles balises HTML de mise en forme qui n'existent plus, comme `<font>` !

### CSS : le support des navigateurs

Tout comme le HTML, le CSS a évolué. Il y a 4 versions importantes de CSS :

- CSS 1
- CSS 2.0
- CSS 2.1
- CSS 3

En fait, la version CSS 3 n'est pas encore totalement finalisée (ce n'est pas encore une version officielle). Cependant, elle est bien avancée et déjà bien supportée par de nombreux navigateurs aujourd'hui, ce qui fait qu'on peut déjà s'en servir.

Il serait dommage de passer à côté, car CSS 3 ajoute de nombreuses fonctionnalités à CSS (il y a le double de fonctionnalités par rapport à CSS 2.1 !). Nous nous baserons donc dans ce cours sur CSS 3, qui reprend et complète la plupart des fonctionnalités de CSS 2.1.

Ce sont les navigateurs web qui font le travail le plus complexe : ils doivent lire le code CSS et comprendre comment afficher la page.

Au début des années 2000, Internet Explorer était le navigateur le plus répandu, mais son support du CSS est resté longtemps assez médiocre .C'était la grande époque de la version 6 (IE6), hélas encore utilisée par une petite partie des internautes aujourd'hui (heureusement, cette proportion tend à diminuer).

Depuis, de nombreux navigateurs sont arrivés: Mozilla Firefox bien sûr, mais aussi Google Chrome. Cela a incité Microsoft à réagir, qui a publié (après une longue période d'inactivité) IE 7, puis IE 8 et IE 9. On parle déjà de IE 10.

**Que faut-il retenir de tout ça ?** Que les navigateurs ne connaissent pas toutes les propriétés CSS qui existent. Plus le navigateur est vieux, moins il connaît de fonctionnalités CSS. Je vais vous présenter dans ce cours un certain nombre de fonctionnalités de CSS qui ne fonctionnent pas forcément sur les navigateurs les plus vieux. Je ne peux pas l'éviter, c'est comme ça : aucun navigateur ne connaît parfaitement toutes les fonctionnalités CSS de toute façon ! Au pire, si le navigateur ne connaît pas une fonctionnalité de CSS, il l'ignore et ne met pas en forme, mais ça ne fait pas planter votre page, ce qui fait que celle-ci sera toujours lisible.

## IV 2. Où écrit-on le CSS ?

Vous avez le choix, car on peut écrire du code en langage CSS à 3 endroits différents :

- Dans un fichier .css (**méthode la plus recommandée**) ;
- Dans l'en-tête <head> du fichier HTML ;
- Directement dans les balises du fichier HTML via un attribut style (**méthode la moins recommandée**) ;

### Dans un fichier .css (recommandé)

Le plus souvent on écrit du code CSS dans un fichier spécial ayant l'extension .css (contrairement aux fichiers HTML qui ont l'extension .html). C'est la méthode la plus pratique et la plus flexible. Ça nous évite de tout mélanger dans un même fichier. On utilisera cette technique tout au long de la suite de ce cours.

Commençons à pratiquer dès maintenant ! On va partir du fichier HTML suivant :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <link rel="stylesheet" href="style.css" />
    <title>Premiers tests du CSS</title>
  </head>
  <body>
    <h1>Mon super site</h1>
    <p>Bonjour et bienvenue sur mon site !</p>
    <p>Pour le moment, mon site est un peu <em>vide</em>.
    Patientez encore un peu !</p>
  </body>
</html>
```

Vous noterez la ligne `<link rel="stylesheet" href="style.css" />` : c'est elle qui indique que ce fichier HTML va être associé à un fichier appelé `style.css` qui va gérer sa mise en forme.

Enregistrez ce fichier avec le nom que vous voulez (par exemple `page.html`). Pour le moment, rien d'extraordinaire à part la nouvelle balise que nous avons ajoutée.

Maintenant, créez un nouveau fichier vide dans votre éditeur de texte (par exemple Notepad++) et copiez-y ce bout de code CSS (rassurez-vous je vous expliquerai ce qu'il veut dire tout à l'heure) :

```
p
{
  color: blue;
}
```

Pour activer la coloration du code dans Notepad++, allez dans le menu Langage > CSS.

Enregistrez le fichier en lui donnant un nom qui se termine par `.css`, comme `style.css`. Placez ce fichier `.css` dans le même dossier que votre fichier `.html`.

Dans votre explorateur de fichiers, vous devriez les voir placés côte à côte. D'un côté le `.html`, de l'autre le `.css`

Ouvrez maintenant votre fichier `page.html` dans votre navigateur pour l'essayer, comme vous le faites d'habitude.

Il est inutile d'ouvrir directement le fichier `style.css` dans le navigateur. Il faut ouvrir le fichier `page.html` (il fera automatiquement appel au fichier `style.css`).

## Dans l'en-tête <head> du fichier HTML

Il existe une autre méthode pour utiliser du CSS dans ses fichiers HTML : cela consiste à insérer le code CSS directement dans une balise <style> à l'intérieur de l'en-tête <head>.

Voici comment on peut obtenir exactement le même résultat avec un seul fichier .html qui contient le code CSS :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8 /">
    <style>
      p
      {
        color: blue;
      }
    </style>
    <title>Premiers tests du CSS</title>
  </head>

  <body>
    <h1>Mon super site</h1>

    <p>Bonjour et bienvenue sur mon site !</p>
    <p>Pour le moment, mon site est un peu <em>vide</em>.
    Patientez encore un peu !</p>
  </body>
</html>
```

## Directement dans les balises (non recommandé)

Dernière méthode, à manipuler avec précaution : vous pouvez ajouter un attribut style à n'importe quelle balise. Vous insèrerez votre code CSS directement dans cet attribut :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Premiers tests du CSS</title>
  </head>

  <body>
    <h1>Mon super site</h1>

    <p style="color: blue;">Bonjour et bienvenue sur mon site
    !</p>
    <p>Pour le moment, mon site est un peu <em>vide</em>.
    Patientez encore un peu !</p>
  </body>
</html>
```

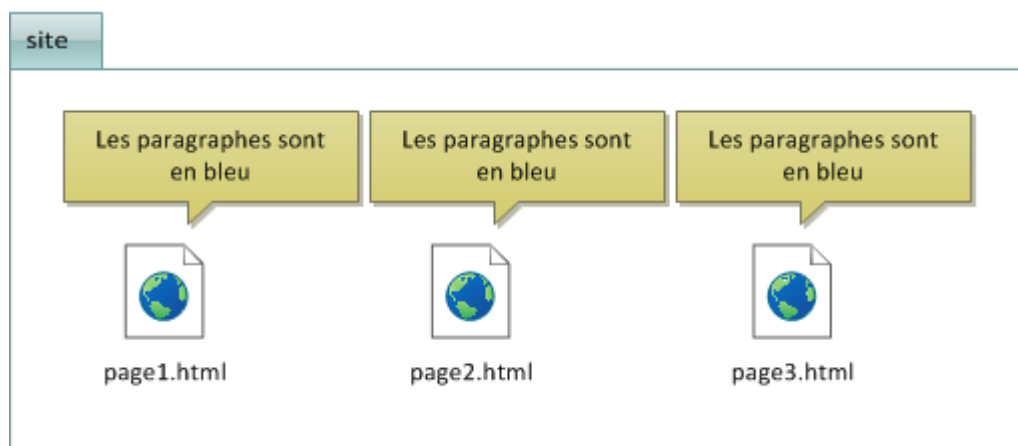
Cette fois, seule la balise du premier paragraphe qui contient le code CSS sera colorée en bleu ;

## Quelle méthode choisir ?

C'est recommandé fortement de prendre l'habitude de travailler avec la première méthode parce que c'est celle utilisée par la majorité des webmasters. Pourquoi ?

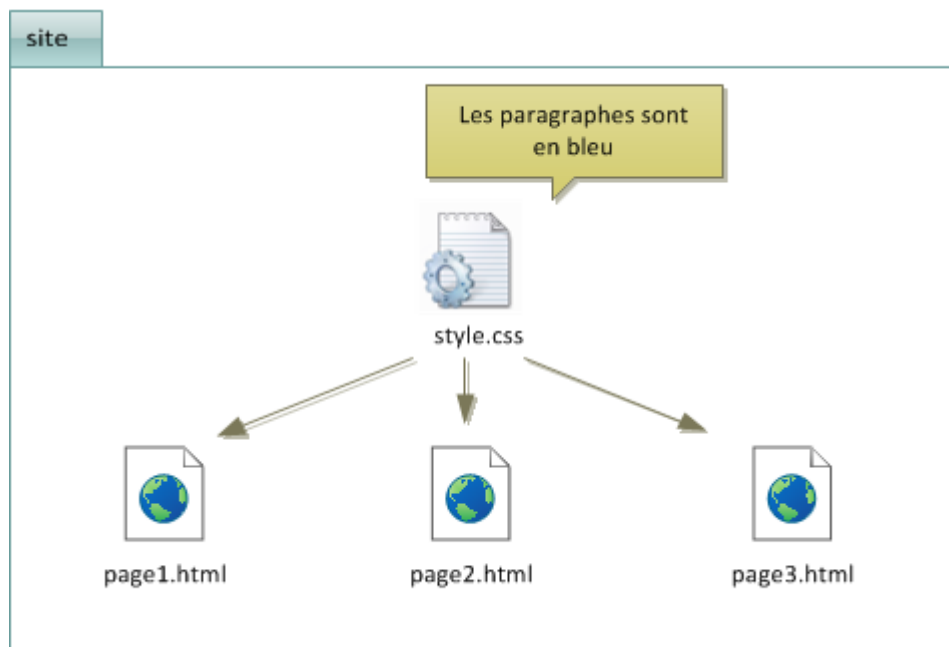
Pour le moment, vous faites vos tests dans un seul fichier HTML. Cependant, votre site sera plus tard constitué de plusieurs pages HTML.

Imaginez : si vous placez le code CSS directement dans le fichier HTML, il faudra copier ce code dans tous les fichiers HTML de votre site ! Et si demain vous changez d'avis, par exemple si vous voulez que vos paragraphes soient écrits en rouge et non en bleu, il faudra modifier chaque fichier HTML un à un !



Le code CSS est répété dans chaque fichier HTML : pas pratique !

Si vous travaillez avec un fichier CSS externe, vous n'aurez besoin d'écrire cette instruction qu'une seule fois pour tout votre site !



Le code CSS est indiqué une fois pour toutes dans un fichier CSS : c'est plus simple !

### IV.3 Appliquer un style : sélectionner une balise

Maintenant que nous savons où placer le code CSS, intéressons-nous à ce code de plus près.

```
p
{
    color: blue;
```

```
}
```

Dans un code CSS comme celui-ci, on trouve 3 éléments différents :

- **Des noms de balises** : on écrit les noms des balises dont on veut modifier l'apparence. Par exemple, si je veux modifier l'apparence de tous les paragraphes <p>, je dois écrire p.
- **Des propriétés CSS** : les "effets de style" de la page sont rangés dans des propriétés. Il y a par exemple la propriété color qui permet d'indiquer la couleur du texte, font-size qui permet d'indiquer la taille du texte, etc. Il y a beaucoup de propriétés).
- **Les valeurs** : à chaque propriété CSS on doit indiquer une valeur. Par exemple, pour la couleur, il faut indiquer le nom de la couleur. Pour la taille, il faut indiquer quelle taille on veut, etc.

Schématiquement, une feuille de style CSS ressemble donc à ça :

```
balise1
{
    propriete: valeur;
    propriete: valeur;
    propriete: valeur;
}
```

```
balise2
{
    propriete: valeur;
    propriete: valeur;
    propriete: valeur;
    propriete: valeur;
}
```

```
balise3
{
    propriete: valeur;
}
```

Vous repérez sur ce schéma les balises, propriétés et valeurs dont je viens de vous parler. Comme vous le voyez, on écrit le nom de la balise (par exemple h1), et on ouvre des accolades pour y mettre les propriétés et valeurs que l'on veut à l'intérieur. On peut mettre autant de propriétés que l'on veut à l'intérieur des accolades. Chaque propriété est suivie du symbole "deux-points" (:) puis de sa valeur correspondante. Enfin, chaque ligne se termine par un point-virgule (;).

Pour le moment, on va juste changer la couleur pour s'entraîner dans nos exemples. Le code CSS que nous avons utilisé jusqu'ici :

```
p
{
    color: blue;
}
```

Essayez de changer le nom de la balise affectée par le code CSS. Par exemple, si j'écris h1, c'est le titre qui sera écrit en bleu. Modifiez votre fichier style.css comme ceci :

```
h1
{
    color: blue;
}
```

Maintenant, ouvrez à nouveau votre page HTML (souvenez-vous, on ouvre la page HTML dans le navigateur, pas le fichier CSS !)

## Appliquer un style à plusieurs balises

```
h1
{
    color: blue;
}
em
{
    color: blue;
}
```

Il signifie que nos titres `<h1>` et nos textes importants `<em>` doivent s'afficher en bleu. Par contre, c'est un peu répétitif, vous ne trouvez pas ?

Heureusement, il existe un moyen en CSS d'aller plus vite si les 2 balises doivent avoir la même présentation. Il suffit de combiner la déclaration en séparant les noms des balises par une virgule comme ceci :

```
h1, em
{
color:blue;
}
```

Cela signifie : "Je veux que le texte de mes `<h1>` et `<em>` soit écrit en bleu". Vous pouvez indiquer autant de balises à la suite que vous le désirez.

## IV. 4. Des commentaires dans du CSS

Comme en HTML, il est possible de mettre des commentaires. Les commentaires ne seront pas affichés, ils servent simplement à indiquer des informations pour vous, par exemple pour vous y retrouver dans un long fichier CSS.

D'ailleurs, vous allez vous en rendre compte, en général le fichier HTML est assez petit, et la feuille CSS assez grande (si elle contient tous les éléments de style de votre site, c'est un peu normal). Notez qu'il est possible de créer plusieurs fichiers CSS pour son site si vous ressentez le besoin de séparer un peu votre code CSS (en fonction des différentes sections de votre site par exemple).

Donc, pour faire un commentaire, c'est facile ! Tapez `/*`, suivi de votre commentaire, puis `*/` pour terminer votre commentaire.

Vos commentaires peuvent être écrits sur une ou plusieurs lignes. Par exemple :

```
/*
style.css
-----
Par ...
*/
p
{
    color: blue; /* Les paragraphes seront bleus */
}
```

## IV 5. Appliquer un style : class et id

Ce qu'on a vu jusqu'ici a quand même un défaut : cela implique par exemple que TOUS les paragraphes soient écrits par exemple en bleu.

Comment faire pour que seulement certains paragraphes soient écrits d'une manière différente ? On pourrait placer notre code CSS dans un attribut style sur la balise que l'on vise (c'est la technique que je vous ai présentée un peu plus tôt), mais comme je vous l'ai dit ce n'est pas recommandé (il vaut mieux utiliser un fichier CSS externe).

Pour résoudre le problème, on peut utiliser ces attributs spéciaux qui fonctionnent sur toutes les balises :

- L'attribut class
- L'attribut id

Que les choses soient claires dès le début : les attributs class et id sont quasiment identiques. Il y a seulement une petite différence qu'on verra plus tard.

Pour le moment et pour faire simple, on ne va s'intéresser qu'à l'attribut class.

```
<h1 class=""> </h1>
<p class=""></p>
<img class="" />
```

Que met-on comme valeur à l'attribut class ?

En fait, vous devez écrire un nom qui sert à identifier la balise. Ce que vous voulez, tant que le nom commence par une lettre.

Par exemple, je vais donner la classe introduction à mon premier paragraphe :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <link rel="stylesheet" href="style.css" />
    <title>Premiers tests du CSS</title>
  </head>

  <body>
    <h1>Mon super site</h1>

    <p class="introduction">Bonjour et bienvenue sur mon site
!</p>
    <p>Pour le moment, mon site est un peu <em>vide</em>.
PatienteZ encore un peu !</p>
  </body>
</html>
```

Maintenant que c'est fait, votre paragraphe est identifié. Il a un nom : introduction. Vous allez pouvoir réutiliser ce nom dans le fichier CSS pour dire : "Je veux que seules les balises qui ont comme nom introduction soient affichées en bleu".

```
.introduction
{
  color: blue;
}
```

Testez le résultat : seul votre paragraphe appelé "introduction" va s'afficher en bleu !

Et l'attribut id alors ?

Lui, il fonctionne exactement de la même manière que class, à un détail près : il ne peut être utilisé qu'une fois dans le code.

En pratique, nous ne mettrons des id que sur des éléments qui sont uniques sur votre page, comme par exemple le logo :

```
1 
```

Si vous utilisez des id, dans le CSS il faudra faire précéder le nom de l'id par un dièse (#) :

```
#logo
{
```



```
/* Indiquez les propriétés CSS ici */  
}
```

Retenez que 2 balises peuvent avoir le même nom avec l'attribut class. Un nom d'id doit en revanche être unique dans la page HTML.

## IV. 6. Les balises universelles

Il arrivera parfois que vous ayez besoin d'appliquer une class (ou un id) à certains mots qui ne sont pas à l'origine entourés par des balises.

En effet, le problème de class, c'est qu'il s'agit d'un attribut. Vous ne pouvez donc en mettre que sur une balise. Par exemple, si je veux modifier uniquement "bienvenue" dans le paragraphe suivant :

```
<p>Bonjour et bienvenue sur mon site !</p>
```

En fait, on a inventé 2 balises dites universelles qui n'ont aucune signification particulière (elles n'indiquent pas que le mot est important par exemple). Il y a une petite (mais importante !) différence entre ces deux balises :

- `<span> </span>` : c'est une balise de type inline. C'est une balise que l'on place au sein d'un paragraphe de texte, pour sélectionner certains mots uniquement. Les balises `<strong>` et `<em>` sont de la même famille. Cette balise s'utilise donc au milieu d'un paragraphe, et c'est celle dont nous allons nous servir pour colorer "bienvenue".
- `<div> </div>` : c'est une balise de type block qui entoure un bloc de texte. Les balises de la même famille sont `<p>`, `<h1>`, etc. Ces balises ont quelque chose en commun : elles créent un nouveau "bloc" dans la page, et provoquent donc obligatoirement un retour à la ligne. `<div>` est une balise fréquemment utilisée dans la construction d'un design.

Pour le moment donc, nous allons utiliser plutôt la balise `<span>`. On la met autour de "bienvenue", on lui rajoute une classe (du nom qu'on veut), on crée le CSS et c'est fini

```
<p>Bonjour et <span class="salutations">bienvenue</span> sur mon  
site !</p>
```

```
.salutations  
{  
    color: blue;  
}
```

## IV. 7. Appliquer un style : les sélecteurs avancés

En CSS, le plus difficile est de savoir cibler le texte dont on veut changer la forme. Pour cibler (on dit "sélectionner") les éléments de la page à modifier, on utilise ce qu'on appelle des sélecteurs. Vous en avez déjà utilisé un peu plus tôt dans ce chapitre, résumons-les pour commencer.

### Les sélecteurs que vous connaissez déjà

Ces sélecteurs, que nous avons vus précédemment, sont de loin les plus couramment utilisés. Il faut les connaître par cœur. Commençons par la base de la base :

```
p  
{  
  
}  

```

... signifie "Je veux affecter tous les paragraphes". Après, c'est à vous de dire ce que vous faites à ces paragraphes (vous les écrivez en bleu par exemple).

Nous avons aussi vu :

```
h1, em
```

```
{
```

```
}
```

... qui signifie "Tous les titres et textes importants". Nous avons sélectionné deux balises d'un coup. Et enfin, nous avons vu comment sélectionner des balises précises à que nous avons donné un nom grâce aux attributs class et id :

```
.class
```

```
{
```

```
}
```

```
#id
```

```
{
```

```
}
```

## Les sélecteurs avancés

### \* : sélecteur universel

```
*
```

```
{
```

```
}
```

Sélectionne toutes les balises sans exception. On l'appelle le sélecteur universel.

### A B : une balise contenue dans une autre

```
h3 em
```

```
{
```

```
}
```

Sélectionne toutes les balises <em> situées à l'intérieur d'une balise <h3>.

Exemple de code HTML correspondant :

```
<h3>Titre avec <em>texte important</em></h3>
```

### A + B : une balise qui en suit une autre

```
h3 + p
```

```
{
```

```
}
```

Sélectionne la première balise <p> située après un titre <h3>.

Exemple :

```
<h3>Titre</h3>
```

```
<p>Paragraphe</p>
```

### A[attribut] : une balise qui possède un attribut

```
a[title]
```

```
{
```

```
}
```

Sélectionne tous les liens <a> qui possèdent un attribut title.

Exemple :

```
<a href="http://site.com" title="Infobulle">
```

**A[attribut="Valeur"] : une balise, un attribut et une valeur exacte**

```
a[title="Cliquez ici"]  
{  
}
```

Idem, mais l'attribut doit en plus avoir exactement pour valeur "Cliquez ici".

Exemple :

```
<a href="http://site.com" title="Cliquez ici">
```

**A[attribut\*="Valeur"] : une balise, un attribut et une valeur**

```
a[title*="ici"]  
{  
}
```

Idem, l'attribut doit cette fois contenir dans sa valeur le mot "ici" (peu importe sa position).

Exemple :

```
<a href="http://site.com" title="Quelque part par ici">
```

# CHAPITRE V FORMATAGE DU TEXTE

## V.1 La taille du texte

### *La taille*

Pour modifier la taille du texte, on utilise la propriété CSS font-size. Mais comment indiquer la taille du texte ? C'est là que ça se corse car plusieurs techniques vous sont proposées :

- Indiquer une **taille absolue** : en pixels, en centimètres ou millimètres. Cette méthode est très précise mais il est conseillé de ne l'utiliser que si c'est absolument nécessaire, car on risque parfois d'indiquer une taille trop petite pour certains lecteurs.
- Indiquer une **taille relative** : en pourcentage, "em" ou "ex", cette technique a l'avantage d'être plus souple. Elle s'adapte plus facilement aux préférences de taille des visiteurs.

### Une taille absolue

Pour indiquer une taille absolue, on utilise généralement les pixels. Pour avoir un texte de 16 pixels de hauteur, vous devez écrire :

```
font-size: 16px;
```

Voici un exemple d'utilisation (placez ce code dans votre fichier .css) :

```
p
{
    font-size: 14px; /* Paragraphes de 14 pixels */
}
h1
{
    font-size: 40px; /* Titres de 40 pixels */
}
```

Si vous le souhaitez, des tailles en centimètres ou millimètres sont aussi disponibles. Remplacez "px" par "cm" ou "mm". Ces unités sont cependant moins bien adaptées aux écrans.

### Une valeur relative

C'est la méthode recommandée car le texte s'adapte alors plus facilement aux préférences de tous les visiteurs.

Il y a plusieurs moyens d'indiquer une valeur relative. Vous pouvez par exemple écrire la taille avec des mots en anglais comme ceux-ci :

- xx-small : minuscule
- x-small : très petit
- small : petit
- medium : moyen
- large : grand
- x-large : très grand
- xx-large : euh... gigantesque.

Vous pouvez tester l'utilisation de ces valeurs dans votre code CSS :

```
p
{
    font-size: small;
}
h1
```

```
{
    font-size: large;
}
```

Cette technique a un défaut : il n'y a que 7 tailles disponibles (car il n'y a que 7 noms). Heureusement il existe d'autres moyens. La technique préférée consiste à indiquer la taille en "em". C'est une unité spécifique au CSS.

- Si vous écrivez 1em, le texte a une taille normale.
- Si vous voulez grossir le texte, vous pouvez inscrire une valeur supérieure à 1, comme 1.3em.
- Si vous voulez réduire le texte, inscrivez une valeur inférieure à 1, comme 0.8em.

Faites attention, il faut mettre un point à la place de la virgule pour les nombres décimaux. Vous devez donc écrire "1.4em" et non pas "1,4em" !

#### **Exemple :**

```
p
{
    font-size: 0.8em;
}
h1
{
    font-size: 1.3em;
}
```

D'autres unités sont disponibles. Vous pouvez essayer le "ex" (qui fonctionne sur le même principe que le em mais qui est plus petit de base) et le pourcentage (80%, 130%...)

## **V. 2. La police**

En effet, le problème c'est que, pour qu'une police s'affiche correctement, il faut que tous les internautes l'aient. Si un internaute n'a pas la même police que vous, son navigateur prendra une police par défaut (une police standard) qui n'aura peut-être rien à voir avec ce à quoi vous vous attendiez.

La bonne nouvelle, c'est que depuis CSS 3, il est possible de faire télécharger automatiquement une police au navigateur.

### **Modifier la police utilisée**

La propriété CSS qui permet d'indiquer la police à utiliser est font-family. Vous devez écrire le nom de la police comme ceci :

```
balise
{
    font-family: police;
}
```

Seulement, pour éviter qu'il n'y ait de problème si l'internaute n'a pas la même police que vous, on précise en général plusieurs noms de police, séparés par des virgules :

```
balise
{
    font-family: police1, police2, police3, police4;
}
```

Le navigateur essaiera d'abord de mettre la police1. S'il ne l'a pas, il essaiera de mettre la police2. S'il ne l'a pas, il essaiera la police3 et ainsi de suite.

En général, on indique en tout dernier serif, ce qui correspond à une police standard (qui ne se met que si aucune autre police n'a été trouvée).

Il existe aussi une autre police par défaut appelée sans-serif. Il y a une petite différence entre ces deux polices par défaut : serif possède des pattes de liaison en bas des lettres, tandis que sans-serif n'en possède pas. Oui c'est subtil.

Oui, mais quelles sont les polices les plus courantes qu'on a le "droit" d'utiliser me direz-vous ? Voici une liste de polices qui fonctionnent bien sur la plupart des navigateurs :

- Arial
- Arial Black
- Comic Sans MS
- Courier New
- Georgia
- Impact
- Times New Roman
- Trebuchet MS
- Verdana

```
p
{
    font-family: Impact, "Arial Black", Arial, Verdana, serif;
}
```

... cela signifie : "Mets la police Impact, ou, si elle n'y est pas, Arial Black, ou sinon Arial, ou sinon Verdana, ou si rien n'a marché mets une police standard (serif)".

En général, il est bien d'indiquer un choix de 3-4 polices (+ serif) afin de s'assurer qu'au moins l'une d'entre elles aura été trouvée sur l'ordinateur du visiteur.

Si le nom de la police comporte des espaces, je conseille de l'entourer de guillemets, comme je l'ai fait pour "Arial Black".

### V.3 Comment mettre en Italique, gras, souligné...

Il existe en CSS une série de propriétés de mises en forme classiques du texte. Nous allons découvrir ici la mise en gras, italique, souligné... et au passage nous verrons qu'il est même possible d'aller jusqu'à faire clignoter le texte !

#### Mettre en italique

Le CSS lui, permet de dire : "Je veux que ce texte soit en italique". Rien ne vous empêche par exemple de décider que tous vos titres seront en italique.

Concrètement, pour mettre en italique en CSS on utilise font-style, qui peut prendre 3 valeurs :

- italic : le texte sera mis en italique.
- oblique : le texte sera aussi mis en italique (en penchant les lettres).
- normal : le texte sera normal (par défaut). Cela vous permet d'annuler une mise en italique. Par exemple, si vous voulez que les textes entre <em> ne soient plus en italique, vous devrez écrire :

```
em
{
    font-style: normal;
}
```

Sur l'exemple suivant, je me sers par exemple de font-style pour mettre en italique tous mes titres <h2>

```
:  
h2  
{  
    font-style: italic;  
}
```

## Mettre en gras

Alors là, pareil, n'oubliez pas que ce n'est pas <strong> qui permet de mettre en gras (son rôle est d'indiquer que le texte est important, donc le navigateur l'affiche en gras). La mise en gras en CSS permet de mettre en gras par exemple les titres, certains paragraphes entiers, etc. C'est à vous de voir. La propriété CSS pour mettre en gras est font-weight, et prend les valeurs suivantes :

- bold : le texte sera en gras.
- normal : le texte sera écrit normalement (par défaut).

Voici par exemple comment écrire les titres en gras :

```
h1  
{  
    font-weight: bold;  
}
```

## Soulignement et autres décorations

Cette propriété CSS porte bien son nom : text-decoration. Elle permet entre autres de souligner le texte, mais pas seulement. Voici les différentes valeurs qu'elle peut prendre :

- underline : souligné.
- line-through : barré.
- overline : ligne au-dessus.
- blink : clignotant. Ne marche pas sur tous les navigateurs (Internet Explorer et Google Chrome notamment).
- none : normal (par défaut).

Ce CSS va vous permettre de tester les effets de text-decoration :

```
h1  
{  
    text-decoration: blink;  
}  
.souligne  
{  
    text-decoration: underline;  
}  
.barre  
{  
    text-decoration: line-through;  
}  
.ligne_dessus  
{  
    text-decoration: overline;  
}
```

## V.4 L'alignement du texte

Le langage CSS nous permet de faire tous les alignements que l'on connaît : à gauche, centré, à droite et justifié.

C'est tout simple. On utilise la propriété `text-align`, et on indique l'alignement désiré :

- `left` : le texte sera aligné à gauche (c'est le réglage par défaut).
- `center` : le texte sera centré.
- `right` : le texte sera aligné à droite.
- `justify` : le texte sera "justifié". Justifier le texte permet de faire en sorte qu'il prenne toute la largeur possible sans laisser d'espace blanc à la fin des lignes. Les textes des journaux, par exemple, sont tout le temps justifiés.

Regardez les différents alignements sur cet exemple :

```
h1
{
    text-align: center;
}
p
{
    text-align: justify;
}
.signature
{
    text-align: right;
}
```

Vous ne pouvez pas modifier l'alignement du texte d'une balise inline (comme `<span>`, `<a>`, `<em>`, `<strong>`...). L'alignement ne fonctionne que sur des balises de type block (`<p>`, `<div>`, `<h1>`, `<h2>`, ...), et c'est un peu logique quand on y pense : on ne peut pas modifier l'alignement de quelques mots au milieu d'un paragraphe !

C'est donc en général le paragraphe entier qu'il vous faudra aligner.

## V.5 Les flottants

Le CSS nous permet de faire flotter un élément autour du texte. On dit aussi qu'on fait un "habillage". Pour que vous voyiez bien de quoi on parle, voici ce que nous allons apprendre à faire :



Lorem ipsum dolor sit amet,  
consectetur adipiscing elit. Donec  
vitae lorem imperdiet lacus molestie  
molestie. Cum sociis natoque penatibus  
et magnis dis parturient montes, nascetur ridiculus  
mus. Donec eu purus. Phasellus metus lorem,  
blandit et, posuere quis, tincidunt vitae, ante.  
Vivamus consequat mauris a diam. Vivamus nibh  
erat, hendrerit nec, aliquet ut, hendrerit quis, nunc.  
Vestibulum et turpis et elit tempor euismod.

### Une image flottant à gauche



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec vitae lorem imperdiet lacus molestie molestie. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec eu purus. Phasellus metus lorem, blandit et, posuere quis, tincidunt vitae, ante. Vivamus consequat mauris a diam. Vivamus nibh erat, hendrerit nec, aliquet ut, hendrerit quis, nunc. Vestibulum et turpis et elit tempor euismod.



## Une image flottant à droite

La propriété magique qui fait flotter est... float ("flottant" en anglais). Cette propriété peut prendre 2 valeurs très simples :

- left : l'élément flottera à gauche.
- right : l'élément flottera... à droite ! Oui bravo.

L'utilisation des flottants est très simple :

1. Vous appliquez un float à une balise.
2. Puis vous continuez à écrire du texte à la suite normalement.

On peut aussi bien utiliser la propriété float sur des balises block que sur des balises inline. Il est courant de faire flotter une image pour qu'elle soit habillée par du texte, comme sur les exemples précédents.

## Faire flotter une image

Nous allons apprendre ici à faire flotter une image. Voici le code HTML que nous devons taper dans un premier temps :

```
<p> Ceci est un texte normal de paragraphe, écrit à la suite de l'image et qui l'habillera car l'image est flottante.</p>
```

Vous devez placer l'élément flottant en premier dans le code HTML. Si vous placez l'image après le paragraphe, l'effet ne fonctionnera pas.

Voici le seul bout de code CSS qu'on a besoin de taper, qui permet de faire flotter l'image à gauche :

```
.imageflottante  
{  
    float: left;  
}
```

Pour faire flotter l'image à droite, il suffit d'indiquer la valeur right et le tour est joué !

## Stopper un flottant

En gros, on aimerait pouvoir faire ça :



Ce texte entoure l'image. Bla  
bla bla bla bla bla bla

Texte écrit en-dessous

Il existe en fait une propriété CSS qui permet de dire : "Stop, ce texte doit être en-dessous du flottant et non plus à côté". C'est la propriété `clear` qui peut prendre ces trois valeurs :

- `left` : le texte se poursuit en-dessous après un `float: left`;
- `right` : le texte se poursuit en-dessous après un `float: right`;
- `both` : le texte se poursuit en-dessous, que ce soit après un `float: left`; ou après un `float: right`;

Pour simplifier, on va utiliser tout le temps le `clear: both`, qui marche après un flottant à gauche et après un flottant à droite (ça marche à tous les coups donc). Pour illustrer son fonctionnement, on va prendre ce code HTML :

```
<p></p>
<p>Ce texte est écrit à côté de l'image flottante.</p>
<p class="dessous">Ce texte est écrit sous l'image flottante.</p>
```

```
.imageflottante
{
    float: left;
}
.dessous
{
    clear: both;
}
```

## V.6 Couleur du texte

Vous connaissez déjà la propriété qui permet de modifier la couleur du texte : il s'agit de `color`. Nous allons nous intéresser aux différentes façons d'indiquer la couleur, car il y en a plusieurs.

### Indiquer le nom de la couleur

La méthode la plus simple et la plus pratique pour choisir une couleur est de taper son nom. Le seul défaut de cette méthode est qu'il n'existe que 16 couleurs dites "standard". Voici les 16 couleurs que vous pouvez utiliser en tapant simplement leur nom :

#### Couleur

white  
silver  
grey  
black  
red  
maroon  
lime  
green  
yellow  
olive  
blue  
navy  
fuchsia  
purple  
aqua

## Couleur

teal

Pour passer tous les titres en marron, on peut donc écrire :

```
h1
{
    color: maroon;
}
```

## La notation hexadécimale

16 couleurs, c'est quand même un peu limite quand on sait que la plupart des écrans peuvent en afficher 16 millions.

Heureusement, il existe plusieurs façons en CSS de choisir une couleur parmi toutes celles qui existent. La première qu'on va voir est la notation hexadécimale. Elle est couramment utilisée sur le Web, mais il existe aussi une autre méthode que nous verrons plus loin.

Un nom de couleur en hexadécimal, ça ressemble à ça : #FF5A28. Pour faire simple, c'est une combinaison de lettres et de chiffres qui indiquent une couleur.

On doit toujours commencer par écrire un dièse (#), suivi de 6 lettres ou chiffres allant de 0 à 9 et de A à F.

Ces lettres ou chiffres fonctionnent deux par deux. Les 2 premiers indiquent une quantité de rouge, les 2 suivants une quantité de vert, et les 2 derniers une quantité de bleu. En mélangeant ces quantités (qui sont les composantes Rouge-Vert-Bleu de la couleur) on peut obtenir la couleur qu'on veut.

Ainsi, #000000 correspond à la couleur noire et #FFFFFF à la couleur blanche.

Certains logiciels de dessin, comme Photoshop, Gimp et Paint .NET, vous indiquent les couleurs en hexadécimal. Il vous est alors facile de copier-coller le code hexadécimal d'une couleur dans votre fichier CSS.

Voici par exemple comment on fait pour appliquer la couleur blanche en hexadécimal sur les paragraphes :

```
p
{
    color: #FFFFFF;
}
```

Notez qu'il existe une notation raccourcie : on peut écrire une couleur avec seulement 3 caractères. Par exemple : #FA3 est équivalent à écrire #FFAA33.

## La méthode RGB

Que signifie RGB ? En anglais, Rouge-Vert-Bleu s'écrit Red-Green-Blue, ce qui s'abrège en "RGB". Comme pour la notation hexadécimale, on doit définir une quantité de rouge, de vert et de bleu pour choisir une couleur.

Oui, mais là vous allez voir que c'est beaucoup plus pratique et qu'avec un logiciel de dessin tout simple comme Paint, vous pouvez trouver la couleur que vous désirez. Voici la marche à suivre :

1. Lancez le logiciel Paint depuis le menu Démarrer.
2. Rendez-vous dans la section Modifier les couleurs :

Une fenêtre s'ouvre. Dans la zone qui apparaît à droite, faites bouger les curseurs pour sélectionner la couleur qui vous intéresse.

3. On relève les quantités de Rouge-Vert-Bleu correspondantes indiquées en bas à droite de la fenêtre (ici 240-96-204). Je recopie ces valeurs dans cet ordre dans le fichier CSS, comme ceci :

```
p
{
    color: rgb(240,96,204);
}
```

4. Comme vous avez pu le constater dans l'exemple, pour utiliser la méthode RGB il faut taper `rgb(Rouge, Vert, Bleu)` en remplaçant "Rouge, Vert, Bleu" par les nombres correspondants. Pour information, ces quantités sont toujours comprises entre 0 et 255.

## V.7 Couleur de fond

Pour indiquer une couleur de fond, on utilise la propriété CSS `background-color`. Elle s'utilise de la même manière que la propriété `color`, c'est-à-dire que vous pouvez taper le nom d'une couleur, l'écrire en notation hexadécimale ou encore utiliser la méthode RGB.

Pour indiquer la couleur de fond de la page web, il faut travailler sur la balise `<body>`. Eh oui, `<body>` correspond à toute la page web, c'est donc en modifiant sa couleur de fond que l'on changera la couleur de fond de la page web.

Regardez très attentivement ce fichier CSS :

```
body /* On travaille sur la balise body, donc sur TOUTE la page */
{
    background-color: black; /* Le fond de la page sera noir */
    color: white; /* Le texte de la page sera blanc */
}
```

On a demandé à ce que le texte de la balise `<body>` soit écrit en blanc, et tous les paragraphes `<p>` et titres `<h1>` ont pris cette couleur. Comment cela se fait-il ?

Ce phénomène s'appelle **l'héritage**.

## Le CSS et l'héritage

En CSS, si vous appliquez un style à une balise, toutes les balises qui se trouvent à l'intérieur prendront le même style.

C'est en fait simple à comprendre et intuitif. La balise `<body>`, vous le savez, contient entre autres les balises de paragraphe `<p>` et de titre `<h1>`.

Si j'applique une couleur de fond noire et une couleur de texte blanche à la balise `<body>`, tous mes titres et paragraphes auront eux aussi une couleur de fond noire et un texte de couleur blanche... C'est ce phénomène qui s'appelle l'héritage : on dit que les balises qui se trouvent à l'intérieur d'une autre balise "héritent" de ses propriétés.

C'est d'ailleurs de là que vient le nom "CSS", qui signifie "Cascading Style Sheets", c'est-à-dire "Feuilles de style en cascade". Les propriétés CSS sont héritées en cascade : si vous donnez un style à un élément, tous les sous-éléments auront le même style.

Si vous dites par la suite que vous voulez vos titres en rouge, ce style aura la priorité et vos titres seront donc en rouge. En revanche, si vous n'indiquez rien de particulier (comme on l'a fait tout à l'heure), alors vos titres hériteront de la couleur blanche.

Cela ne fonctionne pas uniquement pour la couleur, entendons-nous bien. Toutes les propriétés CSS seront héritées : vous pouvez par exemple demander une mise en gras dans la balise `<body>`, et tous vos titres et paragraphes seront en gras.

### Exemple d'héritage avec la balise `<mark>`

On a tendance à croire qu'on ne peut modifier que la couleur de fond de la page. C'est faux : vous pouvez changer le fond de n'importe quel élément : vos titres, vos paragraphes, certains mots... Dans ce cas, ils apparaîtront surlignés (comme si on avait mis un coup de marqueur dessus).

```
<h1>Qui a éteint la lumière ?</h1>
```

```
<p>Il fait tout noir sur ce site, c'est un peu  
<mark>inquiétant</mark> comme ambiance non vous trouvez pas ?</p>
```

Par défaut, le texte s'affiche sur un fond jaune. Vous pouvez changer ce comportement en CSS :

```
body
{
    background-color: black;
    color: white;
}
mark
{
    /* La couleur de fond est prioritaire à celle de toute la page */
    background-color: red;
}
```

Sur le texte de la balise `<mark>`, c'est la couleur de fond rouge qui s'applique. En effet, même si le fond de la page est noir, c'est la propriété CSS de l'élément le plus précis qui a la priorité.

Le même principe vaut pour toutes les balises HTML et toutes les propriétés CSS ! Si vous dites :

- Mes paragraphes ont une taille de 1.2em
- Mes textes importants (`<strong>`) ont une taille de 1.4em

... on pourrait penser qu'il y a un conflit. Le texte important fait partie d'un paragraphe, quelle taille lui donner ? 1.2em ou 1.4em ? Le CSS décide que c'est la déclaration la plus précise qui l'emporte : comme `<strong>` correspond à un élément plus précis que les paragraphes, le texte sera écrit en 1.4em.

## V.8 Images de fond

Dans les exemples qui suivent, je vais modifier l'image de fond de la page. Cependant, tout comme pour la couleur de fond, n'oubliez pas que l'image de fond ne s'applique pas forcément à la page entière. On peut aussi mettre une image de fond aux titres, paragraphes, etc.

### Appliquer une image de fond

La propriété permettant d'indiquer une image de fond est **background-image**. Comme valeur, on doit lui donner `url("nom_de_l_image.png")`. Par exemple :

```
body
{
    background-image: url("neige.png");
}
```

Bien entendu, votre fond n'est pas forcément en PNG, il peut aussi être en JPEG ou en GIF. L'adresse indiquant où se trouve l'image de fond peut être écrite en absolu (<http://...>) ou en relatif (fond.png).

Attention lorsque vous écrivez une adresse en relatif dans le fichier CSS ! L'adresse de l'image doit être indiquée par rapport au fichier .css et non pas par rapport au fichier .html. Je vous conseille de placer l'image de fond dans le même dossier que le fichier .css pour simplifier les choses (ou dans un sous-dossier).

## Options disponibles pour l'image de fond

On peut compléter la propriété background-image que nous venons de voir par plusieurs autres propriétés qui permettent de changer le comportement de l'image de fond.

### background-attachment : fixer le fond

La propriété CSS background-attachment permet de "fixer" le fond. L'effet obtenu est intéressant, car on voit alors le texte "glisser" par-dessus le fond. Deux valeurs sont disponibles :

- fixed : l'image de fond reste fixe.
- scroll : l'image de fond défile avec le texte (par défaut).

```
body
{
    background-image: url("neige.png");
    background-attachment: fixed; /* Le fond restera fixe */
}
```

### background-repeat : répétition du fond

Par défaut, l'image de fond est répétée en mosaïque. Vous pouvez changer cela avec la propriété background-repeat :

- no-repeat : le fond ne sera pas répété. L'image sera donc unique sur la page.
- repeat-x : le fond sera répété uniquement sur la première ligne, horizontalement.
- repeat-y : le fond sera répété uniquement sur la première colonne, verticalement.
- repeat : le fond sera répété en mosaïque (par défaut).

#### Exemple d'utilisation :

```
body
{
    background-image: url("soleil.png");
    background-repeat: no-repeat;
}
```

### background-position : position du fond

On peut indiquer où doit se trouver l'image de fond avec **background-position**. Cette propriété n'est intéressante que si elle est combinée avec background-repeat: no-repeat; (un fond qui ne se répète pas). Vous devez donner à background-position deux valeurs en pixels pour indiquer la position du fond par rapport au coin supérieur gauche de la page (ou du paragraphe si vous appliquez le fond à un paragraphe). Ainsi, si vous tapez :

```
background-position: 30px 50px;
```

... votre fond sera placé à 30 pixels de la gauche et à 50 pixels du haut. Il est aussi possible d'utiliser ces valeurs en anglais :

- top : en haut.

- bottom : en bas.
- left : à gauche.
- center : centré.
- right : à droite.

Il est possible de combiner ces mots. Par exemple, pour aligner une image en haut à droite, vous taperez :

```
background-position: top right;
```

Ainsi, si je veux afficher un soleil en image de fond, en un unique exemplaire (no-repeat), toujours visible (fixed) et positionné en haut à droite (top right), je vais écrire ceci :

```
body
{
    background-image: url("soleil.png");
    background-attachment: fixed; /* Le fond restera fixe */
    background-repeat: no-repeat; /* Le fond ne sera pas répété */
    background-position: top right; /* Le fond sera placé en haut à
droite */
}
```

## Combiner les propriétés

Si vous utilisez beaucoup de propriétés en rapport avec le fond (comme c'est le cas sur ce dernier exemple), vous pouvez utiliser une sorte de "super-propriété" appelée background qui peut prendre plusieurs valeurs combinées des propriétés vues précédemment : background-image, background-repeat, background-attachment et background-position.

On peut donc tout simplement écrire :

```
body
{
    background: url("soleil.png") fixed no-repeat top right;
}
```

C'est la première "super-propriété" que je vous montre, il y en aura d'autres. Il faut savoir que :

- L'ordre des valeurs n'a pas d'importance. Vous pouvez combiner les valeurs dans n'importe quel ordre.
- Vous n'êtes pas obligés de mettre toutes les valeurs. Ainsi, si vous ne voulez pas écrire fixed, vous pouvez l'enlever sans problème.

## Plusieurs images de fond

Depuis CSS3, il est possible de donner plusieurs images de fond à un élément. Pour cela, il suffit de séparer les déclarations par une virgule, comme ceci :

```
body
{
    background: url("soleil.png") fixed no-repeat top right,
url("neige.png") fixed;
}
```

La première image de cette liste sera placée par-dessus les autres. Attention donc, l'ordre de déclaration des images a son importance : si vous inversez le soleil et la neige dans le code CSS précédent, vous ne verrez plus le soleil .

À noter que les images de fond multiples fonctionnent sur tous les navigateurs, sauf sur les anciennes versions d'Internet Explorer, qui ne le reconnaît qu'à partir de la version 9 (IE9).

Une dernière chose avant d'en terminer avec les images de fond : dans tous ces exemples, on a appliqué un fond à la page entière (body). Mais cela ne doit pas vous faire oublier qu'on peut appliquer un fond à n'importe quel élément (un titre, un paragraphe, certains mots d'un paragraphe, etc.). Entraînez-vous à appliquer un fond à vos titres ou paragraphes.

## V.9 La transparence

Le CSS nous permet de jouer très facilement avec les niveaux de transparence des éléments ! Pour cela, nous allons utiliser des fonctionnalités de CSS3 : la propriété **opacity** et la notation RGB.

### La propriété opacity

La propriété opacity, très simple, permet d'indiquer le niveau d'opacité (c'est l'inverse de la transparence).

- Avec une valeur de 1, l'élément sera totalement opaque : c'est le comportement par défaut.
- Avec une valeur de 0, l'élément sera totalement transparent.

Il faut donc choisir une valeur comprise entre 0 et 1. Par exemple avec 0.6, votre élément sera opaque à 60%... et on verra donc à travers !

Voici comment on peut l'utiliser :

```
p
{
    opacity: 0.6;
}
```

Voici un exemple qui va nous permettre d'apprécier la transparence :

```
body
{
    background: url('neige.png');
}
p
{
    background-color: black;
    color: white;
    opacity: 0.3;
}
```

Notez que la transparence fonctionne sur tous les navigateurs récents, y compris Internet Explorer à partir de IE9.

Si vous appliquez la propriété opacity à un élément de la page, tout le contenu de cet élément sera rendu transparent (même les images, les autres blocs à l'intérieur, etc.). Si vous voulez juste rendre la couleur de fond transparente, utilisez plutôt la notation RGBa que l'on va découvrir.

### La notation RGBa

CSS3 nous propose une autre façon de jouer avec la transparence : la notation RGBa. Il s'agit en fait de la notation RGB que nous avons vue précédemment, mais avec un quatrième paramètre : le niveau de transparence (appelé "canal alpha"). De la même façon, avec une valeur de 1, le fond est complètement opaque. Avec une valeur inférieure à 1, il est transparent.

```
p
{
    background-color: rgba(255, 0, 0, 0.5); /* Fond rouge à moitié
transparent */
}
```



```
}
```

C'est aussi simple que cela. Vous pouvez obtenir exactement le même effet qu'avec opacity juste en jouant avec la notation RGBa.

Cette notation est connue de tous les navigateurs récents, y compris Internet Explorer (à partir de IE9). Pour les anciens navigateurs, il est recommandé d'indiquer la notation RGB classique en plus de RGBa.

Le fond ne sera alors pas transparent pour ces navigateurs, mais au moins il y aura bien une couleur de fond.

```
p
```

```
{
```

```
    background-color: rgb(255,0,0); /* Pour les anciens navigateurs
```

```
*/
```

```
    background-color: rgba(255,0,0,0.5); /* Pour les navigateurs
```

```
plus récents */
```

```
}
```

# CHAPITRE VI LES BORDURES ET LES OMBRES

## VI.1 Bordures standard

Le CSS vous offre un large choix de bordures pour décorer votre page. De nombreuses propriétés CSS vous permettent de modifier l'apparence de vos bordures : `border-width`, `border-color`, `border-style`...

Pour aller à l'essentiel, je vous propose ici d'utiliser directement la super-propriété `border` qui regroupe l'ensemble de ces propriétés. On va pouvoir combiner plusieurs valeurs.

Pour `border` on peut utiliser jusqu'à 3 valeurs pour modifier l'apparence de la bordure :

- **La largeur** : indiquez la largeur de votre bordure. Mettez une valeur en pixels (comme 2px).
- **La couleur** : c'est la couleur de votre bordure. Utilisez, comme on l'a appris, soit un nom de couleur ("black", "red"...), soit une valeur hexadécimale (#FF0000), soit une valeur rgb (rgb(198, 212, 37)).
- **Le type de bordure** : là, vous avez le choix. Votre bordure peut être un simple trait, ou des pointillés, ou encore des tirets etc... Voici les différentes valeurs disponibles :
  - none : pas de bordure (par défaut)
  - solid : un trait simple.
  - dotted : pointillés.
  - dashed : tirets.
  - double : bordure double.
  - groove : en relief.
  - ridge : autre effet relief.
  - inset : effet 3D enfoncé.
  - outset : effet 3D surélevé.

Ainsi, pour avoir une bordure bleue en tirets de 3 pixels autour des mes titres, je vais écrire :

```
h1
{
    border: 3px blue dashed;
}
```

Voici les différents styles de bordures en images pour vous aider à faire votre choix :

### En haut, à droite, à gauche, en bas...

Si vous voulez mettre des bordures différentes en fonction du côté (haut, bas, gauche ou droite), vous pouvez le faire sans problème. Dans ce cas, vous devrez utiliser ces 4 propriétés :

- `border-top` : bordure en haut.
- `border-bottom` : bordure en bas.
- `border-left` : bordure à gauche.
- `border-right` : bordure à droite.

Il existe aussi des équivalents pour paramétrer chaque détail de la bordure si vous le désirez : `border-top-width` pour modifier l'épaisseur de la bordure du haut, `border-top-color` pour la couleur du haut, etc.

Ce sont aussi des super-propriétés, elles fonctionnent comme `border` mais ne s'appliquent donc qu'à un seul côté.

Pour ajouter une bordure uniquement à gauche et à droite des paragraphes, on écrira donc :

```
p
{
    border-left: 2px solid black;
    border-right: 2px solid black;
}
```

On peut modifier les bordures de n'importe quel type d'élément sur sa page. Nous l'avons fait ici sur les paragraphes, mais on peut aussi modifier la bordure de ses images, des textes importants comme `<strong>`, etc.

## VI.2 Bordures arrondies

Depuis que CSS3 est arrivé, c'est enfin possible d'en créer facilement ! La propriété `border-radius` va nous permettre d'arrondir les angles de n'importe quel élément facilement. Il suffit d'indiquer la taille ("l'importance") de l'arrondi en pixels :

```
p
{
    border-radius: 10px;
}
```

L'arrondi se voit notamment si l'élément a des bordures ou s'il a une couleur de fond :

On peut aussi préciser la forme de l'arrondi pour chaque coin. Dans ce cas, indiquez 4 valeurs

```
p
{
    border-radius: 10px 5px 10px 5px;
}
```

Les valeurs correspondent aux angles suivants dans cet ordre :

1. En haut à gauche
2. En haut à droite
3. En bas à droite
4. En bas à gauche

Enfin, il est possible d'affiner l'arrondi de nos angles en créant des courbes elliptiques. Il faut indiquer 2 valeurs séparées par un slash (/). Le mieux est certainement de tester pour voir l'effet :

### Code : CSS

```
1 p
2 {
3     border-radius: 20px / 10px;
4 }
```

Les bordures arrondies fonctionnent avec tous les navigateurs récents, y compris Internet Explorer à partir de la version 9 (IE9).

Pour les anciennes versions de Mozilla Firefox, Chrome et Safari, il était nécessaire d'utiliser ce qu'on appelle des "préfixes vendeurs". C'est-à-dire qu'il fallait écrire dans son code CSS une version `-moz-border-radius` pour Firefox, `-webkit-border-radius` pour Safari, etc. Ce n'est heureusement plus nécessaire aujourd'hui, sauf si vous voulez gérer les anciennes versions de ces navigateurs.

## VI.3 Les ombres

Les ombres font partie des nouveautés récentes offertes par CSS3. Aujourd'hui, ajouter des ombres à ses pages se fait en une seule ligne de CSS !

Nous allons ici découvrir deux types d'ombres :

- Les ombres des boîtes
- Les ombres du texte

### **box-shadow : les ombres des boîtes**

La propriété box-shadow s'applique à tout le bloc et prend 4 valeurs dans cet ordre :

1. Le décalage horizontal de l'ombre
2. Le décalage vertical de l'ombre
3. L'adoucissement du dégradé
4. La couleur de l'ombre

Par exemple, pour une ombre noire de 6 pixels, sans adoucissement, on écrira :

```
p
{
    box-shadow: 6px 6px 0px black;
}
```

Ajoutons un adoucissement avec le troisième paramètre. L'adoucissement peut être faible (inférieur au décalage), normal (égal au décalage) ou élevé (supérieur au décalage). Essayons un décalage normal :

```
p
{
    box-shadow: 6px 6px 6px black;
}
```

On peut aussi rajouter une quatrième valeur facultative : inset. Dans ce cas, l'ombre sera placée à l'intérieur du bloc, pour donner un effet enfoncé :

```
p
{
    box-shadow: 6px 6px 6px black inset;
}
```

La propriété box-shadow marche sur tous les navigateurs récents, IE9 inclus. Pour certains navigateurs, en particulier les navigateurs mobiles, il faut encore rajouter un préfixe.

### **text-shadow : l'ombre du texte**

Avec text-shadow, vous pouvez ajouter une ombre directement sur les lettres de votre texte ! Les valeurs fonctionnent exactement de la même façon que box-shadow : décalage, adoucissement et couleur.

```
p
{
    text-shadow: 2px 2px 4px black;
}
```

Cette propriété est reconnue par tous les navigateurs récents, sauf Internet Explorer, qui ne la reconnaît qu'à partir de la version IE10.

# CHAPITRE VII CREATION D'APPARENCES DYNAMIQUES

## VII.1 Au survol

Nous allons découvrir dans ce chapitre plusieurs pseudo-formats CSS. Le premier que je vais vous montrer s'appelle `:hover`. Comme tous les autres pseudo-formats que nous allons voir, c'est une information que l'on rajoute après le nom de la balise (ou de la classe) dans le CSS, comme ceci :

```
a:hover
{

}
```

`:hover` signifie "dessus". `a:hover` signifie donc : "Quand la souris est sur le lien" (quand on pointe dessus).

À partir de là, c'est à vous de définir l'apparence que doivent avoir les liens lorsqu'on pointe dessus. Laissez libre cours à votre imagination, il n'y a pas de limite.

Voici un exemple de présentation des liens, mais n'hésitez pas à inventer le vôtre :

```
a /* Liens par défaut (non survolés) */
{
    text-decoration: none;
    color: red;
    font-style: italic;
}

a:hover /* Apparence au survol des liens */
{
    text-decoration: underline;
    color: green;
}
```

On a défini ici 2 versions des styles pour les liens :

- Pour les liens par défaut (non survolés)
- Pour les liens au survol

Même si on l'utilise souvent sur les liens, vous pouvez modifier l'apparence de n'importe quel élément. Par exemple, vous pouvez modifier l'apparence des paragraphes lorsqu'on pointe dessus :

```
p:hover /* Quand on pointe sur un paragraphe */
{

}
```

## VII.2 Au clic et lors de la sélection

Vous pouvez interagir encore plus finement en CSS. Nous allons voir ici que nous pouvons changer l'apparence des éléments lorsqu'on clique dessus et lorsqu'ils sont sélectionnés !

### **:active : au moment du clic**

Le pseudo-format `:active` permet d'appliquer un style particulier au moment du clic. En pratique, il n'est utilisé que sur les liens. Le lien gardera cette apparence très peu de temps : en fait, le changement apparaît lorsque le bouton de la souris est enfoncé. En clair, ce n'est pas forcément toujours bien visible.

On peut par exemple changer la couleur de fond du lien lorsqu'on clique dessus :

```
a:active /* Quand le visiteur clique sur le lien */
{
    background-color: #FFCC66;
}
```

### **:focus : lorsque l'élément est sélectionné**

Là, c'est un peu différent. Le pseudo-format :focus applique un style lorsque l'élément est sélectionné.

Une fois que vous avez cliqué, le lien reste "sélectionné" (il y a une petite bordure en pointillés autour). C'est ça la sélection.

Ce pseudo-format pourra être appliqué à d'autres balises HTML que nous n'avons pas encore vues, comme les éléments de formulaires.

Essayons pour l'instant sur les liens :

```
a:focus /* Quand le visiteur sélectionne le lien */
{
    background-color: #FFCC66;
}
```

### **VII.3 Lorsque le lien a déjà été visité**

Il est possible d'appliquer un style à un lien vers une page qui a déjà été vue. Par défaut, le navigateur colore le lien en un violet.

Vous pouvez changer cette apparence avec :visited (qui signifie "visité"). En pratique, on ne peut pas changer beaucoup de choses à part la couleur sur les liens visités.

```
a:visited /* Quand le visiteur a déjà vu la page concernée */
{
    color: #AAA; /* Appliquer une couleur grise */
}
```

Si vous ne souhaitez pas que les liens déjà visités soient colorés d'une façon différente, il vous faudra leur appliquer la même couleur qu'aux liens normaux. De nombreux sites web font cela. Une exception notable : Google... ce qui est plutôt pratique, puisqu'on peut voir dans les résultats de ses recherches si on a déjà visité ou non les sites que Google nous présente.

## CHAPITRE VIII STRUCTURER SA PAGE

Nous approchons de plus en plus du but. Si nos pages web ne ressemblent pas encore tout à fait aux sites web que nous connaissons, c'est qu'il nous manque les connaissances nécessaires pour faire la mise en page.

En général, une page web est constituée d'un en-tête (tout en haut), de menus de navigation (en haut ou sur les côtés), de différentes sections au centre... et d'un pied de page (tout en bas). Nous allons nous intéresser dans ce chapitre aux nouvelles balises HTML dédiées à la structuration du site. Ces balises ont été introduites par HTML5 (elles n'existaient pas avant) et vont nous permettre de dire : "Ceci est mon en-tête", "Ceci est mon menu de navigation", etc.

Nous n'allons pas faire de mise en page pour le moment. Nous allons en fait préparer notre document HTML pour pouvoir découvrir la mise en page dans les prochains chapitres.

### VIII.1 Les balises structurantes de HTML5

Je vais vous présenter ici les nouvelles balises introduites par HTML5 pour structurer nos pages. Vous allez voir, cela ne va pas beaucoup changer l'apparence de notre site pour le moment, mais il sera bien construit et prêt à être mis en forme ensuite !

#### **<header> : l'en-tête**

La plupart des sites web possèdent en général un en-tête, appelé header en anglais. On y trouve le plus souvent un logo, une bannière, le slogan de votre site...

Vous devrez placer ces informations à l'intérieur de la balise <header> :

```
<header>
    <!-- Placez ici le contenu de l'en-tête de votre page -->
</header>
```

L'en-tête peut contenir tout ce que vous voulez : images, liens, textes...

Il peut y avoir plusieurs en-têtes dans votre page. Si celle-ci est découpée en plusieurs sections, chaque section peut en effet avoir son propre <header>.

#### **<footer> : le pied de page**

A l'inverse de l'en-tête, le pied de page se trouve en général tout en bas du document. On y trouve des informations comme des liens de contact, le nom de l'auteur, les mentions légales, etc.

```
<footer>
    <!-- Placez ici le contenu du pied de page -->
</footer>
```

#### **<nav> : principaux liens de navigation**

La balise <nav> doit regrouper tous les principaux liens de navigation du site. Vous y placerez par exemple le menu principal de votre site.

Généralement, le menu est réalisé sous forme de liste à puces à l'intérieur de la balise <nav> :

```
<nav>
    <ul>
        <li><a href="index.html">Accueil</a></li>
        <li><a href="forum.html">Forum</a></li>
        <li><a href="contact.html">Contact</a></li>
    </ul>
</nav>
```

Voici le menu principal de navigation du site d'Amazon qui pourrait utiliser la balise <nav> :

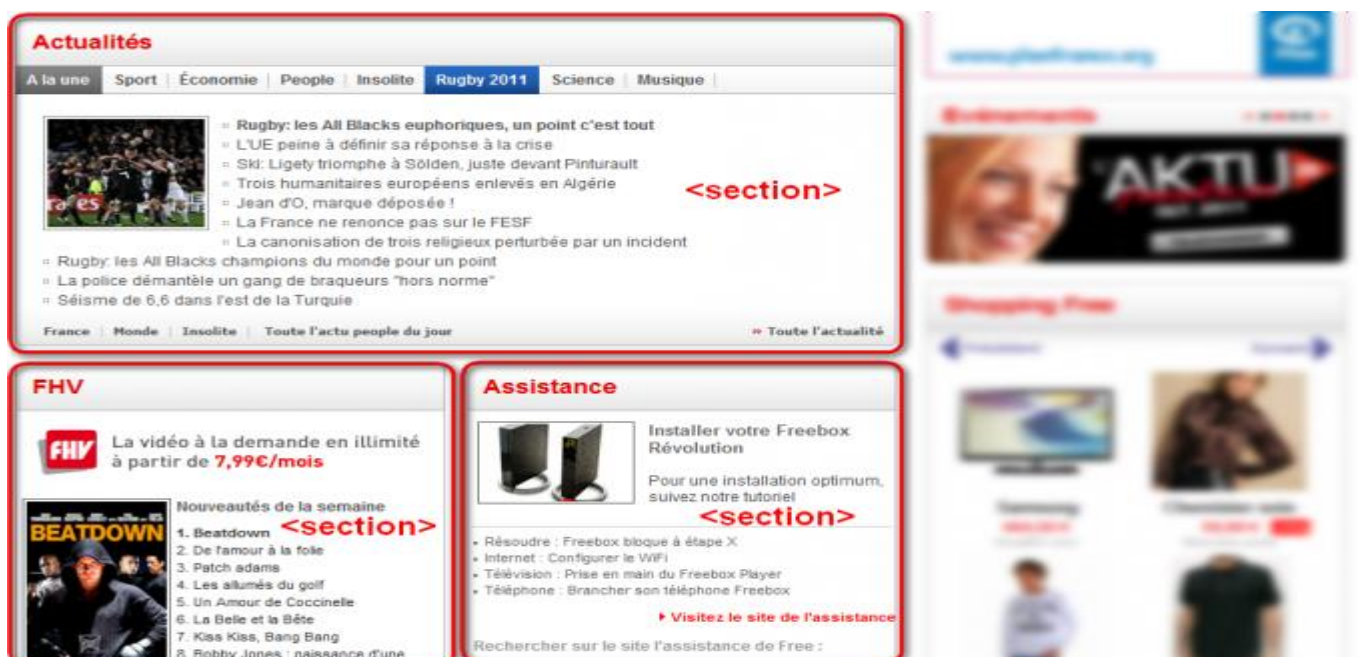


## <section> : une section de page

La balise <section> sert à regrouper des contenus en fonction de leur thématique. Elle englobe généralement une portion du contenu au centre de la page.

```
<section>
  <h1>Ma section de page</h1>
  <p>Bla bla bla bla</p>
</section>
```

Sur la page d'accueil du portail Free.fr, on trouve plusieurs blocs qui pourraient être considérés comme des sections de page :





Chaque section peut avoir son titre de niveau 1 (<h1>), de même que l'en-tête peut contenir un titre <h1> lui aussi. Chacun de ces blocs étant indépendant des autres, il n'est pas illogique de retrouver plusieurs titres <h1> dans le code de sa page web. On a ainsi "Le titre <h1> du <header>", "Le titre <h1> de cette <section>", etc.

## <aside> : informations complémentaires

La balise <aside> est conçue pour contenir des informations complémentaires au document que l'on visualise. Ces informations sont généralement placées sur le côté (bien que ça ne soit pas une obligation).

```
<aside>
  <!-- Placez ici des informations complémentaires -->
</aside>
```

Il peut y avoir plusieurs blocs <aside> dans la page.

## <article> : un article indépendant

La balise <article> sert à englober une portion généralement autonome de la page. C'est une portion de la page qui pourrait (par exemple) être reprise ailleurs sur un autre site. C'est le cas par exemple des actualités (articles de journaux ou de blogs).

```
<article>
  <h1>Mon article</h1>
  <p>Bla bla bla bla</p>
</article>
```

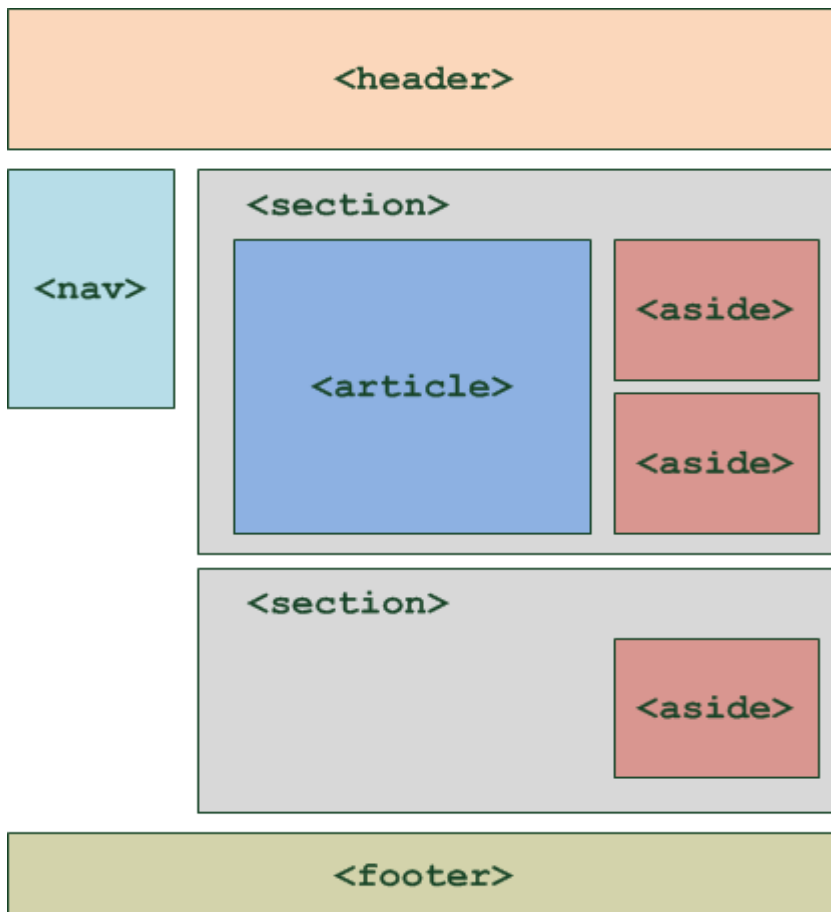
Par exemple, voici un article sur le journal Le Monde :



The image shows a screenshot of a news article on the Le Monde website. The article is titled "Les satellites de Galileo ont été placés sur orbite par la fusée Soyouz". It is dated 21.10.11 and includes a sub-header "LEMONDE.FR avec AFP | 21.10.11 | 08h53 • Mis à jour le 21.10.11 | 17h09". The article features a large photo of a Soyuz rocket being launched. Below the photo, there is a section for recommendations and a brief summary of the article. To the right of the main article, there is a sidebar with a list of other articles. One article in the sidebar is highlighted with a red arrow and the label "<article>", indicating that the main article is an example of an independent article.

## Résumé

Voici un petit schéma pour vous aider à retenir ces nouvelles balises ainsi que leur rôle !



Ne vous y laissez pas tromper : ce schéma propose un exemple d'organisation de la page. Rien ne vous empêche de décider que votre menu de navigation doit être à droite, ou tout en haut, que vos balises `<aside>` sont au-dessus, etc.

On peut même imaginer une seconde balise `<header>`, placée cette fois à l'intérieur d'une `<section>`. Dans ce cas-là, elle sera considérée comme étant l'en-tête de la section. Enfin, une section ne doit pas forcément contenir un `<article>` et des `<aside>`. Utilisez ces balises uniquement si vous en avez besoin. Rien ne vous interdit de créer des sections contenant seulement des paragraphes par exemple.

## VIII.2 Exemple concret d'utilisation des balises

Essayons d'utiliser les balises que nous venons de découvrir pour structurer notre page web. Le code ci-dessous reprend toutes les balises que nous venons de voir au sein d'une page web complète :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Zozor - Le Site Web</title>
  </head>

  <body>
    <header>
      <h1>Zozor</h1>
      <h2>Carnets de voyage</h2>
    </header>

    <nav>
```

```

        <ul>
            <li><a href="#" # ">Accueil</a></li>
            <li><a href="#" # ">Blog</a></li>
            <li><a href="#">CV</a></li>
        </ul>
    </nav>

    <section>
        <aside>
            <h1>A propos de l'auteur</h1>
            <p>C'est moi, Zozor ! Je suis né un 23 novembre
2005.</p>
        </aside>
        <article>
            <h1>Je suis un grand voyageur</h1>
            <p>Bla bla bla bla (texte de l'article)</p>
        </article>
    </section>

    <footer>
        <p>Copyright Zozor - Tous droits réservés<br />
        <a href="#">Me contacter !</a></p>
    </footer>

</body>
</html>

```

Ce code peut vous aider à comprendre comment sont supposées être agencées les balises. Vous y reconnaissez un en-tête, un menu de navigation, un pied de page... et une section au centre, avec un article et un bloc `<aside>` qui donne des informations sur l'auteur de l'article.

Si vous testez le résultat, vous verrez juste du texte noir sur fond blanc. C'est normal, il n'y a pas de CSS ! Par contre, la page est bien structurée ce qui va nous être utile pour la suite :

Les liens sont volontairement factices (d'où la présence d'un simple #), ils n'amènent donc nulle part. On peut très bien obtenir le même résultat sans les utiliser !

C'est vrai. En fait, ces balises sont seulement là pour expliquer à l'ordinateur "Ceci est l'en-tête", "Ceci est mon pied de page"... Elles n'indiquent pas, contrairement à ce qu'on pourrait penser, où doit être placé le contenu. C'est le rôle du CSS, comme nous le verrons dans peu de temps maintenant.

A l'heure actuelle, ces balises ont encore assez peu d'utilité pour tout vous dire. On pourrait très bien utiliser des balises génériques `<div>` à la place pour englober les différentes portions de notre contenu. D'ailleurs, c'est comme cela qu'on faisait avant l'arrivée de ces nouvelles balises HTML5.

Néanmoins, il est assez probable que dans un futur proche les ordinateurs commencent à tirer parti intelligemment de ces nouvelles balises. On peut imaginer par exemple un navigateur qui choisit d'afficher les liens de navigation `<nav>` de manière toujours visible par exemple ! Quand l'ordinateur "comprend" la structure de la page, tout devient possible.

### VIII.3 Assurer la compatibilité avec IE

Les nouvelles balises que nous venons de voir ne sont reconnues par Internet Explorer que depuis la version 9 (IE9). Cela va poser un problème car, quand les anciennes versions d'IE ne connaissent pas une balise... elles ne traitent pas correctement la page (impossible de modifier leur css par exemple) !

Cela peut heureusement se régler assez facilement à l'aide d'un script Javascript. Les scripts sont des petits morceaux de code qui permettent de manipuler la page web et d'effectuer certaines actions. Nous n'allons pas nous intéresser ici au Javascript (ce n'est pas le sujet de ce cours), mais il faut savoir qu'ils sont appelés depuis les pages HTML, un peu de la même façon que les fichiers CSS.

Les fichiers Javascript sont généralement des fichiers ayant l'extension .js. Dans notre code HTML, on les place en général dans la balise <head> avec cette balise :

```
<script src="monscript.js"></script>
```

Si je vous présente (brièvement) Javascript ici, c'est parce qu'un petit script répondant au doux nom de HTML5shiv permet de faire en sorte que les balises que nous venons de voir (<header>, <footer>, <section>...) s'affichent correctement sur les anciennes versions d'Internet Explorer (IE6, IE7, IE8). Concrètement, il vous suffit d'ajouter la ligne suivante dans votre code :

```
<!--[if lt IE 9]>
<script
src="http://html5shiv.googlecode.com/svn/trunk/html5.js"></script>
<![endif]-->
```

Placez ce code dans la balise <head> comme ceci :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <!--[if lt IE 9]>
      <script
src="http://html5shiv.googlecode.com/svn/trunk/html5.js"></script>
    <![endif]-->
    <title>Inclusion de HTML5shiv</title>
  </head>

  <body>

  </body>
</html>
```

Et voilà, votre site s'affichera désormais sans problème sous Internet Explorer !

Sans rentrer dans le détail, sachez que <!--[if lt IE 9]> est un commentaire conditionnel. C'est un commentaire spécial qui n'est lu que par Internet Explorer. Il permet de faire en sorte que le script ne s'exécute que sur les versions d'Internet Explorer inférieures à IE9 (les autres navigateurs n'en ont pas besoin et ignoreront le commentaire).

## CHAPITRE IX : LE MODELE DE BOITE

Une page web peut être vue comme une succession et un empilement de boîtes, qu'on appelle "blocs". La plupart des éléments vus au chapitre précédent sont des blocs : `<header>`, `<article>`, `<nav>`... Mais nous connaissions déjà d'autres blocs : les paragraphes `<p>`, les titres `<h1>`...

Dans ce chapitre, nous allons apprendre à manipuler ces blocs comme de véritables boîtes. Nous allons leur donner des dimensions, les agencer en jouant sur leurs marges, mais aussi apprendre à gérer leur contenu... pour éviter que le texte ne dépasse de ces blocs !

Ce sont des notions fondamentales dont nous allons avoir besoin pour mettre en page notre site web... Soyez attentifs !

### IX.1 Les balises de type block et inline

En HTML, la plupart des balises peuvent se ranger dans deux catégories :

- Les balises inline : c'est le cas par exemple des liens `<a></a>`.
- Les balises block : c'est le cas par exemple des paragraphes `<p></p>`.

Il existe en fait plusieurs autres catégories très spécifiques, par exemples pour les cellules de tableau (type `table-cell`) ou les puces (type `list-item`). Nous n'allons pas nous y intéresser pour le moment, car ces balises sont minoritaires.

Comment reconnaître une balise inline d'une balise block ?

C'est en fait assez facile :

- **block** : une balise de type "block" sur votre page web crée automatiquement un retour à la ligne avant et après. Il suffit d'imaginer tout simplement un bloc. Votre page web sera en fait constituée d'une série de blocs à la suite les uns des autres. Mais vous verrez qu'en plus, il est possible de mettre un bloc à l'intérieur d'un autre, ce qui va augmenter considérablement nos possibilités pour créer le design de notre site !
- **inline** : une balise de type "inline" se trouve obligatoirement à l'intérieur d'une balise "block". Une balise inline ne crée pas de retour à la ligne, le texte qui se trouve à l'intérieur s'écrit donc à la suite du texte précédent, sur la même ligne (c'est pour cela que l'on parle de balise "en ligne").

### Quelques exemples

Afin de mieux vous aider à assimiler quelles balises sont inline et quelles balises sont block, voici un petit tableau listant quelques balises courantes.

| Balises blocks               | Balises inline              |
|------------------------------|-----------------------------|
| <code>&lt;p&gt;</code>       | <code>&lt;em&gt;</code>     |
| <code>&lt;footer&gt;</code>  | <code>&lt;strong&gt;</code> |
| <code>&lt;h1&gt;</code>      | <code>&lt;mark&gt;</code>   |
| <code>&lt;h2&gt;</code>      | <code>&lt;a&gt;</code>      |
| <code>&lt;article&gt;</code> | <code>&lt;img /&gt;</code>  |
| ...                          | ...                         |

Ce tableau n'est pas complet, loin de là.

## Les balises universelles

Vous les connaissez déjà car on les a vues il y a quelques chapitres. Ce sont des balises qui n'ont aucun sens particulier (contrairement à `<p>` qui veut dire "paragraphe", `<strong>` "important", etc.). Le principal intérêt de ces balises c'est de pouvoir leur appliquer une class (ou un id) pour le CSS quand aucune autre balise ne convient.

Il existe 2 balises génériques, et comme par hasard la seule différence entre les deux c'est que l'une d'elle est inline, l'autre est block :

- `<span></span>` (inline)
- `<div></div>` (block)

## Respectez la sémantique !

Les balises universelles sont "pratiques" dans certains cas, certes, mais attention à ne pas en abuser. Je tiens à vous avertir de suite : beaucoup de webmasters mettent des `<div>` et des `<span>` trop souvent, et oublient que d'autres balises plus adaptées existent.

Voici 2 exemples :

- **Exemple d'un span inutile** : `<span class="important">`. Je ne devrais jamais voir ceci dans un de vos codes alors qu'il existe la balise `<strong>` qui sert à indiquer l'importance !
- **Exemple d'un div inutile** : `<div class="titre">`. Ceci est complètement absurde puisqu'il existe des balises faites spécialement pour les titres (`<h1>`, `<h2>`...)

Les balises génériques n'apportent aucun sens à la page et ne peuvent pas être comprises par l'ordinateur. Utilisez toujours d'autres balises plus adaptées quand c'est possible. Google lui-même le conseille pour vous aider à améliorer la position de vos pages au sein de ses résultats de recherche !

## IX.2 Les dimensions

Nous allons ici travailler uniquement sur des balises de type "block".

Pour commencer, intéressons-nous à la taille des blocs. Contrairement à un inline, un bloc a des dimensions précises. Il a une largeur et une hauteur.

Ce qui fait, qu'on dispose de 2 propriétés CSS :

- `width` : c'est la largeur du bloc. A exprimer en pixels (px) ou en pourcentage (%).
- `height` : c'est la hauteur du block. Là encore, on l'exprime soit en pixels (px), soit en pourcentage (%).

Pour être exact, `width` et `height` représentent la largeur et la hauteur du contenu des blocs. Si le bloc a des marges (on va découvrir ce principe un peu plus loin), celles-ci s'ajouteront à la largeur et la hauteur.

Par défaut, un bloc prend 100% de la largeur disponible. On peut le vérifier en ajoutant des bordures ou une couleur de fond sur nos blocs.

Maintenant, ajoutons un peu du CSS pour modifier la largeur des paragraphes. Le CSS suivant dit : "Je veux que tous mes paragraphes aient une largeur de 50%".

```
p
{
    width: 50%;
}
```

Les pourcentages seront utiles pour créer un design qui s'adapte automatiquement à la résolution du visiteur.

Toutefois, il se peut que vous ayez besoin de créer des blocs ayant une dimension précise en pixels :

```
p
{
    width: 250px;
}
```

## Minimum et maximum

On peut demander à ce qu'un bloc ait des dimensions minimales et maximales. C'est très pratique, car cela nous permet de définir des dimensions "limites" pour que notre site s'adapte aux différentes résolutions d'écran de nos visiteurs.

- min-width : largeur minimale
- min-height : hauteur minimale
- max-width : largeur maximale
- max-height : hauteur maximale

Par exemple, on peut demander à ce que les paragraphes occupent 50% de la largeur et exiger qu'il fassent au moins 400 pixels de large dans tous les cas :

```
p
{
    width: 50%;
    min-width: 400px;
}
```

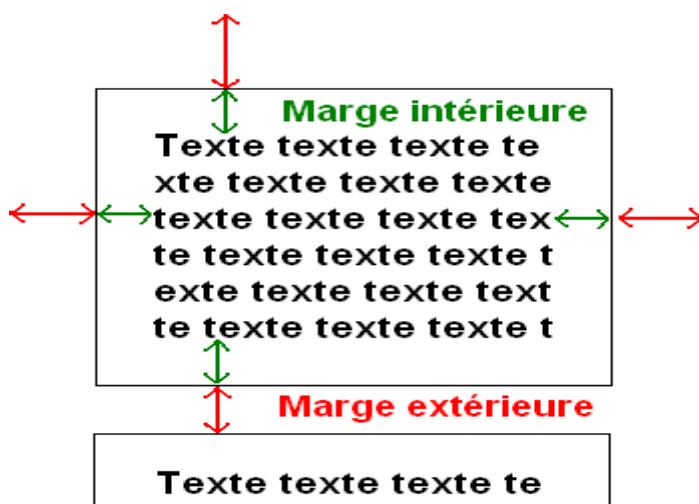
Essayez le résultat en modifiant la largeur de la fenêtre de votre navigateur. Vous allez voir que, si celle-ci est trop petite, le paragraphe se force à occuper au moins 400 pixels de largeur

## IX.3 Les marges

Il faut savoir que tous les blocs possèdent des marges. Il existe **2 types de marges** :

- Les marges intérieures
- Les marges extérieures

Regardez bien ce schéma :



Sur ce bloc, j'ai mis une bordure pour qu'on repère mieux ses bords.

- L'espace entre le texte et la bordure est la marge intérieure.
- L'espace entre la bordure et le prochain bloc est la marge extérieure.

En CSS, on peut modifier la taille des marges avec les 2 propriétés suivantes :

- `padding` : indique la taille de la marge intérieure. A exprimer en général en pixels (px).
- `margin` : indique la taille de la marge extérieure. Là encore, on utilise le plus souvent des pixels.

Les balises de type inline possèdent aussi des marges. Vous pouvez donc aussi essayer ces manipulations sur ce type de balise.

Pour bien voir les marges, prenons 2 paragraphes auxquels je mets simplement une petite bordure :

```
p
{
  width: 350px;
  border: 1px solid black;
  text-align: justify;
}
```



# CHAPITRE X : Le positionnement en CSS

## X.1 Le positionnement flottant

La technique présentée ici l'est à titre d'information. Elle est utilisée par la majorité des sites à l'heure actuelle mais comporte un certain nombre de défauts. Une meilleure technique sera présentée un peu plus loin, le positionnement inline-block, que je vous encourage à utiliser autant que possible.

Nous avons utilisé la propriété float pour faire flotter une image autour du texte :

Il se trouve que cette propriété est aujourd'hui utilisée par la majorité des sites web pour... faire la mise en page ! En effet, si on veut placer son menu à gauche et le contenu de sa page à droite, c'est a priori un bon moyen.

Reprenons le code HTML structuré que nous avons réalisé il y a quelques chapitres :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Zozor - Le Site Web</title>
  </head>
  <body>
    <header>
      <h1>Zozor</h1>
      <h2>Carnets de voyage</h2>
    </header>

    <nav>
      <ul>
        <li><a href="#">Accueil</a></li>
        <li><a href="#">Blog</a></li>
        <li><a href="#">CV</a></li>
      </ul>
    </nav>

    <section>
      <aside>
        <h1>A propos de l'auteur</h1>
        <p>C'est moi, Zozor ! Je suis né un 23 novembre
2005.</p>
      </aside>
      <article>
        <h1>Je suis un grand voyageur</h1>
        <p>Bla bla bla bla (texte de l'article)</p>
      </article>
    </section>

    <footer>
      <p>Copyright Zozor - Tous droits réservés

      <a href="#">Me contacter !</a></p>
    </footer>

  </body>
</html>
```

Nous allons essayer de placer le menu à gauche, et le reste du texte à droite. Pour cela, nous allons faire flotter le menu à gauche, et laisser le reste du texte se placer à sa droite. Nous voulons que le menu occupe 150 pixels de large. Nous allons aussi rajouter une bordure noire au menu et une bordure bleue au corps (à la <section>) pour bien les distinguer :

```
nav
{
    float: left;
    width: 150px;
    border: 1px solid black;
}

section
{
    border: 1px solid blue;
}
```

Il y a deux défauts :

- Le texte du corps de la page touche la bordure du menu. Il manque une petite marge...
- Plus embêtant encore : la suite du texte passe... sous le menu !

On veut bien que le pied de page ("Copyright Zozor") soit placé en bas sous le menu, mais par contre on aimerait que tout le corps de page soit constitué comme un seul bloc à droite.

Pour résoudre ces deux problèmes d'un seul coup, il faut ajouter une marge extérieure à gauche de notre <section> supérieure à la largeur du menu. Si notre menu fait 150px, nous allons par exemple donner une marge extérieure gauche de 170px à notre section de page.

```
nav
{
    float: left;
    width: 150px;
    border: 1px solid black;
}

section
{
    margin-left: 170px;
    border: 1px solid blue;
}
```

A l'inverse, il se peut que vous souhaitiez qu'un élément se place obligatoirement sous le menu. Dans ce cas, il faudra utiliser... `clear: both;`, que nous avons déjà découvert, qui oblige la suite du texte à se positionner sous l'élément flottant.

Comme vous pouvez le constater, il n'y a par défaut pas de marge intérieure (padding). En revanche, il y a une marge extérieure (margin). C'est cette marge qui fait que 2 paragraphes ne soient pas collés et qu'on a l'impression de "sauter une ligne".

Les marges par défaut ne sont pas les mêmes pour toutes les balises de type block. Essayez d'appliquer ce CSS à des balises <div> qui contiennent du texte par exemple, vous verrez que là il n'y a par défaut ni marge intérieure, ni marge extérieure !

Supposons que je veuille rajouter une marge intérieure de 12px aux paragraphes :

```
p
{
    width: 350px;
    border: 1px solid black;
    text-align: justify;
    padding: 12px; /* Marge intérieure de 12px */
}
```

Maintenant, je veux que mes paragraphes soient plus espacés entre eux. Je rajoute la propriété margin pour demander à ce qu'il y ait 50px de marge entre 2 paragraphes :

```
p
{
    width: 350px;
    border: 1px solid black;
    text-align: justify;
    padding: 12px;
    margin: 50px; /* Marge extérieure de 50px */
}
```

Une marge s'est rajoutée à gauche aussi !  
margin (comme padding d'ailleurs) s'applique aux 4 côtés du bloc.

Si vous voulez indiquer une marge en haut, en bas, à gauche et à droite, il va falloir utiliser des propriétés plus précises... Le principe est le même que pour la propriété border, vous allez voir !

### **En haut, à droite, à gauche, en bas... Et on recommence !**

L'idéal serait que vous reteniez les traductions suivantes en anglais :

- top : haut
- bottom : bas
- left : gauche
- right : droite

Voici la liste pour margin :

- margin-top : marge extérieure en haut.
- margin-bottom : marge extérieure en bas.
- margin-left : marge extérieure à gauche.
- margin-right : marge extérieure à droite

Et la liste pour padding :

- padding-top : marge intérieure en haut.
- padding-bottom : marge intérieure en bas.
- padding-left : marge intérieure à gauche.
- padding-right : marge intérieure à droite.

Il y a d'autres façons de spécifier les marges avec les propriétés margin et padding. Par exemple :  
margin: 2px 0 3px 1px;  
... signifie "2px de marge en haut, 0px à droite (le px est facultatif), 3px en bas, 1px à gauche.  
Autre notation raccourcie :

margin: 2px 1px;

... signifie "2px de marge en haut et en bas, 1px de marge à gauche et à droite".

## Centrer des blocs

Il est tout à fait possible de centrer des blocs. C'est même très pratique pour réaliser un design centré quand on ne connaît pas la résolution du visiteur.

Pour centrer, il faut respecter les règles suivantes :

- Donnez une largeur au bloc (avec la propriété width)
- Indiquez que vous voulez des marges extérieures automatiques, comme ceci : margin: auto;

Essayons cette technique sur nos petits paragraphes :

```
p
{
    width: 350px; /* On a indiqué une largeur (obligatoire) */
    margin: auto; /* On peut donc demander à ce que le bloc soit
centré avec "auto" */
    border: 1px solid black;
    text-align: justify;
    padding: 12px;
    margin-bottom: 20px;
}
```

Ainsi, le navigateur centre automatiquement nos paragraphes !

Il n'est cependant pas possible de centrer verticalement un bloc avec cette technique. Seul le centrage horizontal est permis.

## Quand ça dépasse...

Lorsqu'on commence à définir des dimensions précises à nos blocs, comme on vient de le faire, il arrive qu'ils deviennent trop petits pour le texte qu'ils contiennent.

Les propriétés CSS que nous allons voir ici ont justement été créées pour contrôler les dépassements... et décider quoi faire si jamais cela devait arriver.

## overflow : couper un bloc

Supposons que vous ayez un long paragraphe et que vous vouliez (pour une raison qui ne regarde que vous) qu'il fasse 250px de large et 110px de haut. Ajoutons-lui une bordure et remplissons-le de texte :

```
p
{
    width: 250px;
    height: 110px;
    text-align: justify;
    border: 1px solid black;
}
```

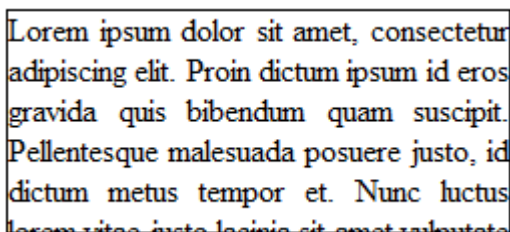
Horreur ! Le texte dépasse des limites du paragraphe ! 🤔

Eh oui ! Vous avez demandé des dimensions précises, vous les avez eues ! Mais... le texte ne tient pas à l'intérieur d'un si petit bloc.

Si vous voulez que le texte ne dépasse pas des limites du paragraphe, il va falloir utiliser la propriété `overflow`. Voici les valeurs qu'elle peut accepter :

- `visible` (par défaut) : si le texte dépasse les limites de taille, il reste visible et sort volontairement du bloc.
- `hidden` : si le texte dépasse les limites, il sera tout simplement coupé. On ne pourra pas voir tout le texte.
- `scroll` : là encore, le texte sera coupé s'il dépasse les limites. Sauf que cette fois, le navigateur mettra en place des barres de défilement pour qu'on puisse voir tout le texte. C'est un peu comme un cadre à l'intérieur de la page.
- `auto` : c'est le mode "pilote automatique". En gros, c'est le navigateur qui décide ou pas de mettre des barres de défilement (il n'en mettra que si c'est nécessaire). C'est la valeur que je conseille d'utiliser le plus souvent.

Avec `overflow: hidden`; le texte est donc coupé (on ne peut pas voir la suite) :



Essayons maintenant `overflow: auto`; avec le code CSS suivant :

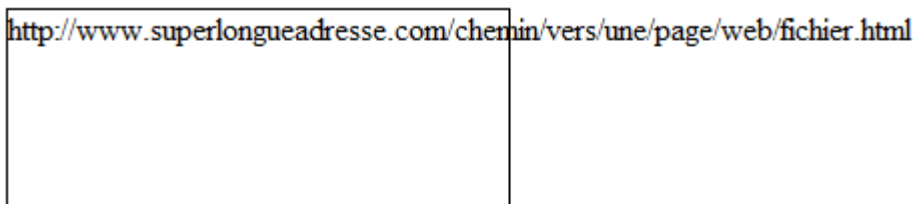
```
p
{
    width: 250px;
    height: 110px;
    text-align: justify;
    border: 1px solid black;
    overflow: auto;
}
```

Il existe une balise HTML, `<iframe>`, qui donne à peu près le même résultat. Elle permet de charger tout le contenu d'une autre page HTML au sein de votre page. Je vous conseille d'éviter d'en abuser, car il peut être lourd de charger une page qui charge elle-même d'autres pages web.

### **word-wrap : couper les textes trop larges**

Cette propriété permet de forcer la césure des très longs mots (généralement des adresses un peu longues).

Voici par exemple un problème que l'on peut avoir quand on écrit une URL un peu longue dans un bloc :



L'ordinateur ne sait pas "couper" l'adresse car il n'y a ni espace, ni tiret. Il ne sait pas faire la césure. Avec le code suivant, la césure sera forcée dès que le texte risque de dépasser :

```
p
{
    word-wrap: break-word;
}
```

C'est conseillé d'utiliser cette fonctionnalité dès qu'un bloc de texte est susceptible de contenir du texte saisi par des utilisateurs (par exemple sur les forums). Sans cette astuce, on peut "casser" facilement le design d'un site.

## X.2 Transformez vos éléments avec display

Il existe en CSS une propriété très puissante : **display**. Elle est capable de **transformer** n'importe quel élément de votre page d'un type vers un autre. Avec cette propriété magique, je peux par exemple transformer mes liens (originellement de type inline) sous forme de blocs :

```
a
{
    display: block;
}
```

A ce moment-là, les liens vont se positionner les uns en-dessous des autres (comme des blocs normaux) et il devient possible de modifier leurs dimensions !

Voici quelques-unes des principales valeurs que peut prendre la propriété display en CSS (il y en a encore d'autres) :

| Valeur       | Exemples                    | Description                                                                                                               |
|--------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------|
| inline       | <a>, <em>,<br><span>...     | Eléments d'une ligne. Se placent les uns à côté des autres.                                                               |
| block        | <p>, <div>,<br><section>... | Eléments en forme de blocs. Se placent les uns en-dessous des autres et peuvent être redimensionnés.                      |
| inline-block | <select>, <input>           | Eléments positionnés les uns à côté des autres (comme les inline) mais qui peuvent être redimensionnés (comme les blocs). |
| none         | <head>                      | Eléments non affichés.                                                                                                    |

On peut donc décider de masquer complètement un élément de la page avec cette propriété. Par exemple, si je veux masquer les éléments qui ont la classe "secret", je vais écrire :

```
.secret
{
    display: none;
}
```

## X.3 Le positionnement inline-block

Les manipulations que demande le positionnement flottant se révèlent parfois un peu délicates sur des sites complexes. Dès qu'il y a un peu plus qu'un simple menu à mettre en page, on risque d'avoir à recourir à des `clear: both;` qui complexifient rapidement le code de la page.

Si le positionnement flottant reste, de loin, le mode de positionnement le plus utilisé sur le Web à l'heure actuelle, d'autres techniques existent et bien peu de webmasters le savent. L'une d'elles, étonnamment puissante, est passée sous les yeux des concepteurs de sites web alors qu'elle existe depuis CSS 2.1, c'est-à-dire depuis plus de 10 ans ! Elle consiste à transformer vos éléments en inline-block avec la propriété display.

Quelques petits rappels sur les éléments de type inline-block :

- Ils se positionnent les uns à côté des autres (exactement ce qu'on veut pour placer notre menu et le corps de notre page !).
- On peut leur donner des dimensions précises (là encore, exactement ce qu'on veut !).

Nous allons transformer en inline-block les deux éléments que nous voulons placer côte à côte : le menu de navigation et la section du centre de la page.

```
nav
{
    display: inline-block;
    width: 150px;
    border: 1px solid black;
}
```

```
section
{
    display: inline-block;
    border: 1px solid blue;
}
```

Ca n'est pas tout à fait ce qu'on voulait là encore. Et en fait, c'est normal : les éléments inline-block se positionnent sur une même ligne de base (appelée baseline), en bas.

Heureusement, le fait d'avoir transformé les éléments en inline-block nous permet d'utiliser une nouvelle propriété, normalement réservée aux tableaux : vertical-align. Cette propriété permet de modifier l'alignement vertical des éléments. Voici quelques-unes des valeurs possibles pour cette propriété :

- baseline : aligne la base de l'élément avec celle de l'élément parent (par défaut)
- top : aligne en haut
- middle : aligne au milieu
- bottom : aligne en bas
- (valeur en px ou %) : aligne à une certaine distance de la ligne de base (baseline)

Il ne nous reste plus qu'à aligner nos éléments en haut, et le tour est joué !

```
nav
{
    display: inline-block;
    width: 150px;
    border: 1px solid black;
    vertical-align: top;
}
section
{
    display: inline-block;
    border: 1px solid blue;
    vertical-align: top;
}
```

Vous noterez que le corps (la <section>) ne prend pas toute la largeur. En effet, ce n'est plus un bloc ! La section occupe seulement la place dont elle a besoin.

Si cela ne vous convient pas pour votre design, modifiez la taille de la section avec width.

## X.4 Les positionnements absolu, fixe et relatif

Il existe d'autres techniques un peu particulières permettant de positionner avec précision des éléments sur la page :

- **Le positionnement absolu** : il nous permet de placer un élément n'importe où sur la page (en haut à gauche, en bas à droite, tout au centre, etc..)
- **Le positionnement fixe** : identique au positionnement absolu, mais cette fois l'élément reste toujours visible, même si on descend plus bas dans la page. C'est un peu comme le même principe que `background-attachment: fixed`
- **Le positionnement relatif** : permet de décaler l'élément par rapport à sa position normale.

Comme pour les flottants, le positionnement absolu, fixé et relatif fonctionne aussi sur des balises de type inline.

Toutefois, vous verrez qu'on l'utilise bien plus souvent sur des balises block que sur des balises inline.

Il faut d'abord faire son choix entre les 3 modes de positionnement disponibles. Pour cela, on utilise la propriété CSS `position` à laquelle on donne une de ces valeurs :

- `absolute` : positionnement absolu.
- `fixed` : positionnement fixe.
- `relative` : positionnement relatif.

Nous allons étudier chacun de ces positionnements un à un.

### Le positionnement absolu

Le positionnement absolu permet de placer un élément (réellement) n'importe où sur la page. Pour effectuer un positionnement absolu, on doit écrire :

```
element
{
    position: absolute;
}
```

Mais ça ne suffit pas ! On a dit qu'on voulait un positionnement absolu, mais encore faut-il dire où on veut que le bloc soit positionné sur la page.

Pour ce faire, on va utiliser 4 propriétés CSS :

- `left` : position par rapport à la gauche de la page.
- `right` : position par rapport à la droite de la page.
- `top` : position par rapport au haut de la page.
- `bottom` : position par rapport au bas de la page.

On peut leur donner une valeur en pixels, comme `"14px"`, ou bien une valeur en pourcentage, comme `"50%"`.

Il faut donc utiliser la propriété `position` et au moins une des 4 propriétés ci-dessus (`top`, `left`, `right` ou `bottom`). Si on écrit par exemple :

```
element
{
    position: absolute;
    right: 0px;
    bottom: 0px;
}
```



... cela signifie que le bloc doit être positionné tout en bas à droite (0 pixels par rapport à la droite de la page, 0 par rapport au bas de la page).

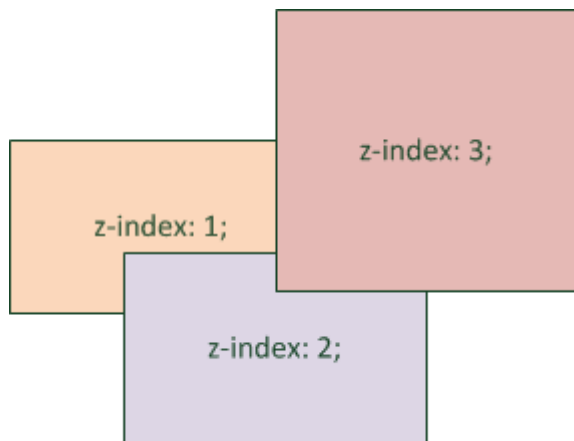
Si on essaye de placer notre bloc <nav> en bas à droite de la page, on obtient ce résultat :

On peut bien entendu ajouter une marge intérieure (padding) au menu pour qu'il soit moins collé à sa bordure.

Les éléments positionnés en absolu sont placés par-dessus le reste des éléments de la page ! Par ailleurs, si vous placez deux éléments en absolu vers le même endroit, ils risquent de se chevaucher. Dans ce cas, utilisez la propriété z-index pour indiquer quel élément doit apparaître par-dessus les autres :

```
element
{
    position: absolute;
    right: 0px;
    bottom: 0px;
    z-index: 1;
}
element2
{
    position: absolute;
    right: 30px;
    bottom: 30px;
    z-index: 2;
}
```

L'élément ayant la valeur de z-index la plus élevée sera placé par-dessus les autres, comme le montre ce schéma :



Une petite précision technique qui a son importance : le positionnement absolu ne se fait pas toujours forcément par rapport au coin en haut à gauche de la fenêtre ! Si vous positionnez en absolu un bloc A qui se trouve dans un autre bloc B lui-même positionné en absolu (ou fixe ou relatif), alors votre bloc A se positionnera par rapport au coin en haut à gauche du bloc B.

## Le positionnement fixe

Le principe est **exactement le même** que pour le positionnement absolu, sauf que cette fois le bloc reste fixe à sa position, même si on descend plus bas dans la page.

```

element
{
    position: fixed;
    right: 0px;
    bottom: 0px;
}

```

Essayez le résultat, vous verrez que le menu reste dans le cas présent affiché en bas à droite même si on descend plus bas dans la page.

## Le positionnement relatif

Le positionnement relatif est un petit peu plus délicat à utiliser. Ce positionnement permet d'effectuer des "ajustements" : l'élément est décalé par rapport à sa position initiale.

Prenons par exemple un texte important, situé entre 2 balises <strong>. Pour commencer, je le mets sur fond rouge pour qu'on puisse mieux le repérer :

```

strong
{
    background-color: red; /* Fond rouge */
    color: yellow; /* Texte de couleur jaune */
}

```

Cette fois, le schéma vu tout à l'heure pour les positions absolue et fixe ne marche plus. Pourquoi ? Parce que l'origine a changé : le point de coordonnées (0, 0) ne se trouve plus en haut à gauche de votre fenêtre comme c'était le cas tout à l'heure. Non, cette fois l'origine se trouve en haut à gauche... de la position actuelle de votre élément.

Ce schéma devrait vous aider à comprendre où se trouve l'origine des points :

Pas de doute,  point d'origine (0, 0) **ce texte est important** si on veut comprendre corr

Donc, si vous faites une position: relative; et que vous appliquez une des propriétés top, left, right ou bottom, le texte sur fond rouge va se déplacer par rapport à la position où il se trouve. Prenons un exemple : je veux que mon texte se décale de 55 pixels vers la droite et de 10 pixels vers le bas. Je vais donc demander à ce qu'il soit décalé de 55 pixels par rapport au "bord gauche", et de 10 pixels par rapport au "bord haut" :

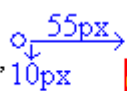
```

Strong
{
    background-color: red;
    color: yellow;

    position: relative;
    left: 55px;
    top: 10px;
}

```

Le texte s'est alors décalé par rapport à sa position initiale, comme ceci :

Pas de doute,  55px 10px **ce texte est important** si on veut com

# CHAPITRE XI : LES TABLEAUX ET LES FORMULAIRES

## XI.1 Un tableau simple

La première balise à connaître est `<table>` `</table>`. C'est cette balise qui permet d'indiquer le début et la fin d'un tableau.

Cette balise est de type block, il faut donc la placer en dehors d'un paragraphe. Exemple :

```
<p>Ceci est un paragraphe avant le tableau.</p>
```

```
<table>
  <!-- Ici, on écrira le contenu du tableau -->
</table>
```

```
<p>Ceci est un paragraphe après le tableau.</p>
```

Qu'écrit-on à l'intérieur du tableau ?

Pour commencer en douceur, voici 2 nouvelles balises très importantes :

- `<tr>` `</tr>` : indique le début et la fin d'une ligne du tableau.
- `<td>` `</td>` : indique le début et la fin du contenu d'une cellule.

En HTML, votre tableau se construit ligne par ligne. Dans chaque ligne (`<tr>`), on indique le contenu des différentes cellules (`<td>`).

Schématiquement, notre tableau de tout à l'heure se construit comme ça :

| Nom                 | Age    | Pays       |
|---------------------|--------|------------|
| Anne                | 27 ans | France     |
| Carmen              | 33 ans | Espagne    |
| Michelle            | 26 ans | Etats-Unis |
| Ogasaku Nyagatosoka | 18 ans | Japon      |

On a une balise de ligne (`<tr>`) qui englobe chacune des cellules (`<td>`)

Par exemple, si je veux faire un tableau à deux lignes, avec 3 cellules par ligne (donc 3 colonnes), je devrai taper ceci :

```
<table>
  <tr>
    <td>Carmen</td>
    <td>33 ans</td>
    <td>Espagne</td>
  </tr>
  <tr>
    <td>Michelle</td>
    <td>26 ans</td>
    <td>Etats-Unis</td>
  </tr>
</table>
```

Le texte s'est écrit à la suite, et il n'y a même pas de bordures !

Oui, un tableau sans CSS paraît bien vide. Alors justement, rajouter des bordures est très simple, vous connaissez déjà le code CSS correspondant !

```
td /* Toutes les cellules des tableaux... */
{
    border: 1px solid black; /* ... auront une bordure de 1px */
}
```

|          |        |            |
|----------|--------|------------|
| Carmen   | 33 ans | Espagne    |
| Michelle | 26 ans | Etats-Unis |

En effet, on aimerait qu'il n'y ait qu'une seule bordure entre 2 cellules, or ce n'est pas le cas ici. Heureusement, il existe une propriété CSS spécifique aux tableaux : `border-collapse`, qui signifie "coller les bordures entre elles".

Cette propriété peut prendre 2 valeurs :

- `collapse` : les bordures seront collées entre elles, c'est l'effet qu'on recherche.
- `separate` : les bordures seront dissociées (valeur par défaut)

```
table
{
    border-collapse: collapse; /* Les bordures du tableau seront
collées (plus joli) */
}
td
{
    border: 1px solid black;
}
```

|          |        |            |
|----------|--------|------------|
| Carmen   | 33 ans | Espagne    |
| Michelle | 26 ans | Etats-Unis |

La ligne d'en-tête

Maintenant que l'on a ce qu'on voulait, on va rajouter la ligne d'en-tête du tableau. Dans notre exemple, les en-têtes sont "Nom", "Age" et "Pays".

La ligne d'en-tête se crée avec un `<tr>` comme on a fait jusqu'ici, mais les cellules à l'intérieur sont cette fois des `<th>` et non pas des `<td>` !

```
<table>
  <tr>
    <th>Nom</th>
    <th>Age</th>
    <th>Pays</th>
  </tr>
  <tr>
    <td>Carmen</td>
    <td>33 ans</td>
    <td>Espagne</td>
  </tr>
  <tr>
    <td>Michelle</td>
    <td>26 ans</td>
    <td>Etats-Unis</td>
  </tr>
</table>
```

La ligne d'en-tête est très facile à reconnaître pour 2 raisons :

- Les cellules sont des <th> au lieu des <td> habituels.
- C'est la première ligne du tableau

Comme le nom des cellules est un peu différent pour l'en-tête, il faut penser à mettre à jour le CSS pour lui dire d'appliquer une bordure sur les cellules normales ET sur l'en-tête :

```
table
{
    border-collapse: collapse;
}
td, th /* Mettre une bordure sur les td ET les th */
{
    border: 1px solid black;
}
```

| Nom      | Age    | Pays       |
|----------|--------|------------|
| Carmen   | 33 ans | Espagne    |
| Michelle | 26 ans | Etats-Unis |

Comme vous pouvez le constater, votre navigateur a mis le texte des cellules d'en-tête en gras. C'est ce que font en général les navigateurs, mais si vous le désirez vous pouvez changer ça à coup de CSS : modifier la couleur de fond des cellules d'en-tête, leur police, leur bordure, etc.

## Titre du tableau

Normalement, tout tableau doit avoir un titre. Le titre permet de renseigner rapidement le visiteur sur le contenu du tableau.

Dans notre exemple, on a une liste de personnes? Qu'est-ce que ça représente ?

Heureusement, il y a <caption> !

Cette balise se place tout au début du tableau, juste avant l'en-tête. C'est elle qui indique le titre du tableau :

```
<table>
  <caption>Passagers du vol 377</caption>

  <tr>
    <th>Nom</th>
    <th>Age</th>
    <th>Pays</th>
  </tr>
  <tr>
    <td>Carmen</td>
    <td>33 ans</td>
    <td>Espagne</td>
  </tr>
  <tr>
    <td>Michelle</td>
    <td>26 ans</td>
    <td>Etats-Unis</td>
  </tr>
</table>
```

Passagers du vol 377

| Nom      | Age    | Pays       |
|----------|--------|------------|
| Carmen   | 33 ans | Espagne    |
| Michelle | 26 ans | Etats-Unis |

Sachez que vous pouvez changer la position du titre avec la propriété CSS `caption-side`, qui peut prendre quatre valeurs :

- `top` : le titre sera placé en haut du tableau (par défaut).
- `bottom` : le titre sera placé en bas du tableau.
- `left` : le titre sera placé à gauche du tableau.
- `right` : le titre sera placé à droite du tableau.

## XI.2 Un tableau structuré

Nous avons appris à construire des petits tableaux simples. Ces petits tableaux suffisent dans la plupart des cas, mais il arrivera que vous ayez besoin de réaliser des tableaux plus... complexes. Nous allons découvrir 2 techniques particulières :

- Pour les gros tableaux, il est possible de les **diviser** en 3 parties :
  - En-tête
  - Corps du tableau
  - Pied de tableau
- Pour certains tableaux, il se peut que vous ayez besoin de **fusionner** des cellules entre elles.

### Diviser un gros tableau

Si votre tableau est assez gros, vous aurez tout intérêt à le découper en plusieurs parties. Pour cela, il existe des balises HTML qui permettent de définir les 3 "zones" du tableau :

- **L'en-tête (en haut)** : il se définit avec les balises `<thead></thead>`
- **Le corps (au centre)** : il se définit avec les balises `<tbody></tbody>`
- **Le pied du tableau (en bas)** : il se définit avec les balises `<tfoot></tfoot>`

Que mettre dans le pied de tableau ? Généralement, si c'est un long tableau, vous y recopiez les cellules d'en-tête. Ça permet de voir même en bas du tableau à quoi se rapporte chacune des colonnes.

Schématiquement, un tableau en 3 parties se découpe donc comme cela :

|                           |                      |            |             |                      |
|---------------------------|----------------------|------------|-------------|----------------------|
|                           | Passagers du vol 377 |            |             |                      |
| <b>En-tête du tableau</b> | <b>Nom</b>           | <b>Age</b> | <b>Pays</b> | <b>&lt;thead&gt;</b> |
|                           | Carmen               | 33 ans     | Espagne     |                      |
|                           | Michelle             | 26 ans     | Etats-Unis  |                      |
| <b>Corps du tableau</b>   | François             | 43 ans     | France      | <b>&lt;tbody&gt;</b> |
|                           | Martine              | 34 ans     | France      |                      |
|                           | Jonathan             | 13 ans     | Australie   |                      |
|                           | Xu                   | 19 ans     | Chine       |                      |
| <b>Pied du tableau</b>    | <b>Nom</b>           | <b>Age</b> | <b>Pays</b> | <b>&lt;tfoot&gt;</b> |

C'est un peu déroutant, mais il est conseillé d'écrire les balises dans l'ordre suivant :

1. `<thead>`
2. `<tfoot>`

### 3. <tbody>

On met donc dans le code d'abord la partie du haut, ensuite la partie du bas, et enfin la partie principale (<tbody>). Le navigateur se chargera d'afficher les éléments au bon endroit, ne vous inquiétez pas. 😊

Voici donc le code à écrire pour construire le tableau en 3 parties :

```
<table>
  <caption>Passagers du vol 377</caption>
  <thead> <!-- En-tête du tableau -->
    <tr>
      <th>Nom</th>
      <th>Age</th>
      <th>Pays</th>
    </tr>
  </thead>

  <tfoot> <!-- Pied de tableau -->
    <tr>
      <th>Nom</th>
      <th>Age</th>
      <th>Pays</th>
    </tr>
  </tfoot>

  <tbody> <!-- Corps du tableau -->
    <tr>
      <td>Carmen</td>
      <td>33 ans</td>
      <td>Espagne</td>
    </tr>
    <tr>
      <td>Michelle</td>
      <td>26 ans</td>
      <td>Etats-Unis</td>
    </tr>
    <tr>
      <td>François</td>
      <td>43 ans</td>
      <td>France</td>
    </tr>
    <tr>
      <td>Martine</td>
      <td>34 ans</td>
      <td>France</td>
    </tr>
    <tr>
      <td>Jonathan</td>
      <td>13 ans</td>
      <td>Australie</td>
    </tr>
    <tr>
      <td>Xu</td>
      <td>19 ans</td>
      <td></td>
    </tr>
```

```

        <td>Chine</td>
    </tr>
</tbody>
</table>

```

Il n'est pas obligatoire d'utiliser ces 3 balises (<thead>, <tbody>, <tfoot>) sur son tableau. En fait, vous vous en servirez surtout si votre tableau est assez gros et que vous avez besoin de l'organiser plus clairement.

Pour les "petits" tableaux vous pouvez garder sans problème l'organisation plus simple qu'on a vue au début.

Sur certains tableaux complexes, vous aurez besoin de "fusionner" des cellules entre elles. Un exemple de fusion ? Regardez, ce tableau qui liste des films et qui indique à qui ils s'adressent :

| Titre du film                | Pour enfants ?          | Pour adolescents ?   |
|------------------------------|-------------------------|----------------------|
| Massacre à la tronçonneuse   | Non, trop violent       | Oui                  |
| Les bisounours font du ski   | Oui, adapté             | Pas assez violent... |
| Lucky Luke, seul contre tous | Pour toute la famille ! |                      |

Pour le dernier film, vous voyez que les cellules ont été fusionnées : elles ne font plus qu'un. C'est exactement l'effet qu'on cherche à obtenir.

Pour effectuer une fusion, il faut rajouter un attribut à la balise <td>. Il faut savoir qu'il existe 2 types de fusion :

- **La fusion de colonnes** : c'est ce que je viens de faire sur cet exemple. La fusion s'effectue horizontalement.  
On utilisera l'attribut colspan.
- **La fusion de lignes** : là, deux lignes seront groupées entre elles. La fusion s'effectuera verticalement.  
On utilisera l'attribut rowspan.

Comme vous le savez, vous devez donner une valeur à l'attribut (que ce soit colspan ou rowspan). Il faut indiquer le nombre de cellules à fusionner entre elles. Sur notre exemple, on a fusionné deux cellules : celle de la colonne "Pour enfants ?", et celle de "Pour adolescents ?". On devra donc écrire :

```
<td colspan="2">
```

... qui signifie : "Cette cellule est la fusion de 2 cellules". Il est possible de fusionner plus de cellules à la fois (3, 4, 5... tant que vous voulez).

Voilà le code HTML qui me permet de réaliser la fusion correspondant au tableau précédent :

```

<table>
  <tr>
    <th>Titre du film</th>
    <th>Pour enfants ?</th>
    <th>Pour adolescents ?</th>
  </tr>
  <tr>
    <td>Massacre à la tronçonneuse</td>
    <td>Non, trop violent</td>
    <td>Oui</td>
  </tr>
  <tr>
    <td>Les bisounours font du ski</td>
    <td>Oui, adapté</td>
    <td>Pas assez violent...</td>
  </tr>
  <tr>
    <td>Lucky Luke, seul contre tous</td>
    <td colspan="2">Pour toute la famille !</td>
  </tr>
</table>

```



```

</tr>
<tr>
    <td>Lucky Luke, seul contre tous</td>
    <td colspan="2">Pour toute la famille !</td>
</tr>
</table>

```

Une remarque importante : vous voyez que la dernière ligne ne contient que 2 cellules au lieu de 3 (il n'y a que 2 balises <td>). C'est tout à fait normal, car j'ai fusionné les deux dernières cellules entre elles. Le <td colspan="2"> indique que cette cellule prend la place de 2 cellules à la fois.

Et pour la fusion verticale avec rowspan, on fait comment ?

Ça se complique un petit peu. Pour notre exemple, on va "inverser" l'ordre de notre tableau : au lieu de mettre les titres de films à gauche, on va les placer en haut.

C'est une autre façon de voir le tableau : au lieu de le construire en hauteur, on peut le construire en longueur.

Dans ce cas, le colspan n'est plus adapté, c'est un rowspan qu'il faut utiliser :

```

<table>
  <tr>
    <th>Titre du film</th>
    <td>Massacre à la tronçonneuse</td>
    <td>Les bisounours font du ski</td>
    <td>Lucky Luke, seul contre tous</td>
  </tr>
  <tr>
    <th>Pour enfants ?</th>
    <td>Non, trop violent</td>
    <td>Oui, adapté</td>
    <td rowspan="2">Pour toute la famille !</td>
  </tr>
  <tr>
    <th>Pour adolescents ?</th>
    <td>Oui</td>
    <td>Pas assez violent...</td>
  </tr>
</table>

```

Résultat : les cellules sont fusionnées verticalement !

| Titre du film      | Massacre à la tronçonneuse | Les bisounours font du ski | Lucky Luke, seul contre tous |
|--------------------|----------------------------|----------------------------|------------------------------|
| Pour enfants ?     | Non, trop violent          | Oui, adapté                | Pour toute la famille !      |
| Pour adolescents ? | Oui                        | Pas assez violent...       |                              |

Notez qu'on peut modifier l'alignement vertical du texte des cellules de tableaux, avec la propriété vertical-align que nous avons découverte dans le chapitre sur la mise en page.

### XI.3 Créer un formulaire

Lorsqu'il vous prend subitement l'envie d'insérer un formulaire dans votre page HTML, vous devez pour commencer écrire une balise <form></form>. C'est la balise principale du formulaire, elle permet d'en indiquer le début et la fin.

```
<p>Texte avant le formulaire</p>
```

```
<form>
  <p>Texte à l'intérieur du formulaire</p>
</form>
```

```
<p>Texte après le formulaire</p>
```

Notez qu'il faut obligatoirement mettre des balises de type block (comme `<p></p>`) à l'intérieur de votre formulaire si vous souhaitez écrire du texte à l'intérieur.

Voilà pour la structure de base..

On va prendre un exemple pour que les choses soient claires. Supposons que votre visiteur vienne de taper un commentaire dans votre formulaire, comme par exemple un message qu'il aimerait poster sur vos forums. Ce message doit être envoyé pour que vous puissiez le recevoir et l'afficher à vos autres visiteurs.

Eh bien c'est là le problème, ou plutôt les problèmes que l'on va se poser :

- **Problème n°1** : comment envoyer le texte rentré par le visiteur ? Par quel moyen ?
- **Problème n°2** : une fois que les données ont été envoyées, comment les traiter ? Souhaitez-vous recevoir le message automatiquement par mail, ou préférez-vous qu'un programme se charge de l'enregistrer quelque part, puis de l'afficher sur une page visible par tout le monde ?

Vous devez ajouter 2 attributs à la balise `<form>` afin de donner les réponses à ces 2 problèmes :

- **method** : cet attribut indique par quel moyen les données vont être envoyées (réponse au **problème n°1**). Il existe 2 moyens pour envoyer des données sur le web :
  - **method="get"** : c'est une méthode en général assez peu adaptée, car elle est limitée à 255 caractères. La particularité vient du fait que les informations seront envoyées dans l'adresse de la page (`http://...`), mais ce détail ne nous intéresse pas vraiment pour le moment. La plupart du temps, je vous recommande d'utiliser l'autre méthode : "post".
  - **method="post"** : c'est la méthode la plus utilisée pour les formulaires car on peut envoyer un grand nombre d'informations grâce à elle. Les données saisies dans le formulaire ne transitent pas par la barre d'adresse.
- **action** : c'est l'adresse de la page ou du programme qui va traiter les informations (réponse au **problème n°2**). Cette page se chargera de vous envoyer un e-mail avec le message si c'est ce que vous voulez, ou bien d'enregistrer le message avec tous les autres dans une base de données.

Cela ne peut pas se faire en HTML et CSS, on utilisera en général un autre langage dont vous avez peut-être entendu parler : PHP.

On va donc maintenant compléter la balise `<form>` avec les 2 attributs qu'on vient de voir.

Pour **method**, vous l'aurez deviné je vais mettre la valeur "post".

Pour **action**, je vais taper le nom d'une page fictive en PHP (`traitement.php`). C'est cette page qui sera appelée lorsque le visiteur cliquera sur le bouton "Envoyer le formulaire".

```
<p>Texte avant le formulaire</p>

<form method="post" action="traitement.php">
  <p>Texte à l'intérieur du formulaire</p>
</form>
```

```
<p>Texte après le formulaire</p>
```

Pour le moment, on ne sait pas ce qu'il se passe à l'intérieur de la page traitement. Notre priorité pour le moment est de découvrir en HTML / CSS comment faire pour insérer des zones de texte, des boutons et des cases à cocher dans votre page web. C'est ce que nous allons voir maintenant.

## ***Les zones de saisie basiques***

Bien, retour au concret.

Nous allons passer en revue les différentes balises HTML permettant de rentrer du texte dans un formulaire.

Il faut savoir qu'il y a 2 zones de texte différentes :

- **La zone de texte monoligne** : comme son nom l'indique, on ne peut écrire qu'une seule ligne à l'intérieur. Elle sert à rentrer des textes courts, comme par exemple : "Entrez votre pseudo".
- **La zone de texte multiligne** : cette zone de texte permet d'écrire une quantité importante de texte sur plusieurs lignes, comme par exemple : "Rédigez une dissertation sur l'utilité du HTML dans le développement des pays"

### **Zone de texte monoligne**

Voici à quoi ressemble une zone de texte monoligne :

Votre pseudo :

Pour insérer une zone de texte à une ligne, on va utiliser la balise `<input />`.

On retrouvera cette balise plusieurs fois par la suite dans ce chapitre. A chaque fois, c'est la valeur de son attribut `type` qui va changer.

Pour créer une zone de texte à une ligne, on doit écrire :

```
<input type="text" />
```

Ce n'est pas encore suffisant : il faut donner un nom à votre zone de texte. Ce nom n'apparaît pas sur la page, mais il vous sera indispensable par la suite. En effet, cela vous permettra (en PHP par exemple) de reconnaître d'où viennent les informations : vous saurez que tel texte est le pseudo du visiteur, tel texte est le mot de passe qu'il a choisi, etc.

Pour donner un nom à un élément de formulaire, on utilise l'attribut `name`. Ici, on va supposer qu'on demande au visiteur de rentrer son pseudo :

```
<input type="text" name="pseudo" />
```

Essayons donc de créer un formulaire très basique avec notre champ de texte :

```
<form method="post" action="traitement.php">
  <p><input type="text" name="pseudo" /></p>
</form>
```

### **Les libellés**

Cette zone de texte est bien jolie, mais si votre visiteur tombe dessus il ne sait pas ce qu'il doit écrire. C'est justement le rôle de la balise `<label>` :

```
<form method="post" action="traitement.php">
  <p>
    <label>Votre      pseudo</label>      :      <input      type="text"
```

```
name="pseudo" />
</p>
</form>
```

Mais ça ne suffit pas. Il faut lier le label avec la zone de texte.

Pour ce faire, il faut donner un nom à la zone de texte, non pas avec l'attribut name mais avec l'attribut id (que l'on peut utiliser sur toutes les balises).

Pour lier le label au champ, il faut lui donner un attribut for qui a la même valeur que l'id du champ... Le mieux est de le voir sur un exemple :

```
<form method="post" action="traitement.php">
  <p>
    <label for="pseudo">Votre pseudo</label> : <input type="text"
name="pseudo" id="pseudo" />
  </p>
</form>
```

Essayez de cliquer sur le texte "Votre pseudo" : vous allez voir que le curseur se place automatiquement dans la zone de texte correspondante.

## Quelques attributs supplémentaires

On peut placer un certain nombre d'autres attributs sur la balise <input /> pour personnaliser son fonctionnement : On peut agrandir le champ avec size.

- On peut limiter le nombre de caractères que l'on peut saisir avec maxlength.
- On peut pré-remplir le champ avec une valeur par défaut avec value.
- On peut donner une indication sur le contenu du champ avec placeholder. Cette indication disparaîtra dès que le visiteur aura cliqué à l'intérieur du champ.

Dans l'exemple suivant, la zone de texte contient une indication permettant de comprendre ce qu'il faut saisir, elle fait 30 caractères de long mais on ne peut écrire que 10 caractères maximum à l'intérieur :

```
<form method="post" action="traitement.php">
  <p>
    <label for="pseudo">Votre pseudo :</label>
    <input type="text" name="pseudo" id="pseudo" placeholder="Ex
: Zozor" size="30" maxlength="10" />
  </p>
</form>
```

Testez vous-mêmes le résultat pour voir le comportement du champ :

Votre pseudo :

## Zone de mot de passe

Vous pouvez facilement faire en sorte que la zone de texte se comporte comme une zone "mot de passe", c'est-à-dire une zone où on ne voit pas à l'écran les caractères saisis. Pour créer ce type de zone de saisie, utilisez l'attribut type="password".

```
<form method="post" action="traitement.php">
  <p>
    <label for="pseudo">Votre pseudo :</label>
    <input type="password" name="pseudo" id="pseudo" />
  </p>
</form>
```

```

        <br />
        <label for="pass">Votre mot de passe :</label>
        <input type="password" name="pass" id="pass" />

    </p>
</form>

```

## Zone de texte multiligne

Pour créer une zone de texte multiligne, on change de balise : nous allons utiliser `<textarea></textarea>`.

Comme pour tout autre élément du formulaire, il faut lui donner un nom avec `name`, et utiliser un label qui explique de quoi il s'agit.

```

1 <form method="post" action="traitement.php">
2   <p>
3     <label for="ameliorer">Comment pensez-vous que je pourrais
4   améliorer mon site ?</label><br />
5     <textarea name="ameliorer" id="ameliorer"></textarea>
6   </p>
</form>

```

Comment pensez-vous que je pourrais améliorer mon site ?

C'est un peu petit comme vous pouvez le constater ! Heureusement, on peut modifier la taille du `<textarea>` de 2 façons différentes :

- **En CSS** : il suffit d'appliquer les propriétés CSS `width` et `height` au `<textarea>`.
- **Avec des attributs** : on peut ajouter les attributs `rows` et `cols` à la balise `<textarea>`. Le premier indique le nombre de lignes de texte qui peuvent être affichées simultanément, et le second le nombre de colonnes.

Pourquoi ouvre-t-on la balise `<textarea>` pour la fermer juste après ?

Vous pouvez pré-remplir le `<textarea>` avec une valeur par défaut. Dans ce cas, on n'utilise pas l'attribut `value` : on écrit tout simplement le texte par défaut entre la balise ouvrante et la balise fermante !

```

<form method="post" action="traitement.php">
  <p>
    <label for="ameliorer">Comment pensez-vous que je puisse
    améliorer mon site ?</label><br />
    <textarea name="ameliorer" id="ameliorer" rows="10"
    cols="50">Améliorer ton site ?!
    Mais enfin ! Il est tellement beau qu'il n'est pas nécessaire de
    l'améliorer !</textarea>
  </p>
</form>

```

## Les zones de saisie enrichies

HTML5 apporte de nombreuses nouvelles fonctionnalités aux formulaires. De nouveaux types de champs sont en effet apparus avec cette version. Il suffit de donner à l'attribut type de la balise `<input />` l'une des nouvelles valeurs disponibles. Faisons un petit tour d'horizon ! Tous les navigateurs ne connaissent pas encore ces zones de saisie enrichies. Les anciennes versions des navigateurs afficheront une simple zone de saisie monoligne à la place (comme si on avait écrit `type="text"`). Entre nous, c'est parfait : les nouveaux navigateurs peuvent profiter des nouvelles fonctionnalités, tandis que les anciens affichent une zone de texte de remplacement qui convient tout aussi bien.

### E-mail

Vous pouvez demander à saisir une adresse e-mail :

```
<input type="email" />
```

Le champ vous semblera a priori identique, mais votre navigateur sait désormais que l'utilisateur doit saisir une adresse e-mail. Il peut afficher une indication si l'adresse n'est pas un e-mail, c'est ce que fait Firefox par exemple :

### Une URL

Avec le type url, on peut demander à saisir une adresse absolue (commençant généralement par `http://`) :

```
<input type="url" />
```

Même principe : si le champ ne vous semble pas différent sur votre ordinateur, sachez que votre ordinateur comprend bel et bien que le visiteur est censé entrer une adresse. Les navigateurs mobiles affichent par exemple un clavier adapté à la saisie d'adresses.



### Numéro de téléphone

Ce champ est dédié à la saisie de numéros de téléphone :

```
<input type="tel" />
```

Sur iPhone par exemple, un clavier adapté s'affiche lorsqu'on doit remplir le champ :



## Nombre

Ce champ permet de saisir un nombre entier :

```
<input type="number" />
```

Le champ s'affichera en général avec des petites flèches pour changer la valeur :

Vous pouvez personnaliser le fonctionnement du champ avec les attributs suivants :

- min : valeur minimale autorisée.
- max : valeur maximale autorisée.
- step : c'est le "pas" de déplacement. Si vous indiquez un pas de 2, le champ n'acceptera que des valeurs de 2 en 2 (par exemple 0, 2, 4, 6...).

## Un curseur

Le type range permet de sélectionner un nombre avec un curseur (aussi appelé slider) :

```
1 <input type="range" />
```

Vous pouvez utiliser là aussi les attributs min, max et step pour définir les limites qui peuvent être sélectionnées.

## Couleur

Ce champ permet de saisir une couleur :

```
<input type="color" />
```

En pratique, il reste assez peu implémenté par les navigateurs à l'heure actuelle. Ne vous étonnez pas si vous voyez seulement un champ de texte classique.

## Date

Différents types de champs de sélection de date existent :

- date : pour la date (05/08/1985 par exemple)
- time : pour l'heure (13:37 par exemple)
- week : pour la semaine
- month : pour le mois
- datetime : pour la date et l'heure (avec gestion du décalage horaire)
- datetime-local pour la date et l'heure (sans gestion du décalage horaire)

### Exemple :

```
<input type="date" />
```

Comme vous le voyez, il y a le choix !

A l'heure actuelle, peu de navigateurs gèrent ce type de champ à part Opera.

## Recherche

On peut créer un champ de recherche comme ceci :

```
<input type="search" />
```

Le navigateur décide ensuite comment afficher le champ de recherche. Il peut ajouter une petite loupe au champ pour signifier que c'est un champ de recherche par exemple, et éventuellement mémoriser les dernières recherches effectuées par le visiteur.

## Les éléments d'options

HTML vous offre une ribambelle d'éléments d'options à utiliser dans votre formulaire. Ce sont des éléments qui demandent au visiteur de faire un choix parmi une liste de possibilités. Nous allons passer en revue :

- Les cases à cocher
- Les zones d'options
- Les listes déroulantes

## Les cases à cocher

Créer une case à cocher ? Rien de plus simple ! Nous allons réutiliser la balise `<input />`, en spécifiant cette fois le type `checkbox` :

```
<input type="checkbox" name="choix" />
```

Rajoutez un `<label>` bien placé, et le tour est joué !

```
<form method="post" action="traitement.php">
  <p>
    Cochez les aliments que vous aimez manger :<br />
    <input type="checkbox" name="frites" id="frites" /> <label
for="frites">Frites</label><br />
    <input type="checkbox" name="steak" id="steak" /> <label
for="steak">Steak haché</label><br />
    <input type="checkbox" name="epinards" id="epinards" />
<label for="epinards">Epinards</label><br />
    <input type="checkbox" name="huitres" id="huitres" /> <label
for="huitres">Huitres</label>
  </p>
</form>
```



N'oubliez pas de donner un nom différent à chaque case à cocher, cela vous permettra d'identifier plus tard quelles cases le visiteur a coché.

Enfin, sachez que vous pouvez faire en sorte qu'une case soit cochée par défaut avec l'attribut checked :

```
<input type="checkbox" name="choix" checked />
```

Normalement, tout attribut possède une valeur. Dans le cas présent en revanche, ajouter une valeur n'est pas obligatoire : la présence de l'attribut suffit à faire comprendre à l'ordinateur que la case doit être cochée.

Si cela vous perturbe, sachez que vous pouvez donner n'importe quelle valeur à l'attribut (certains webmasters écrivent parfois checked="checked", mais c'est un peu redondant !). Dans tous les cas, la case sera cochée.

## Les zones d'options

Les zones d'options vous permettent de faire un choix (et un seul) parmi une liste de possibilités. Elles ressemblent un peu aux cases à cocher, mais il y a une petite difficulté supplémentaire : elles doivent être organisées en groupes. Un même groupe d'options a le même nom (name), mais chaque option doit avoir une valeur (value) différente.

La balise à utiliser est toujours un <input />, avec cette fois la valeur radio pour l'attribut type. Les choses seront plus claires sur l'exemple ci-dessous :

```
<form method="post" action="traitement.php">
  <p>
    Veuillez indiquer la tranche d'âge dans laquelle vous vous
    situez :<br />
    <input type="radio" name="age" value="moins15" id="moins15"
  /> <label for="moins15">Moins de 15 ans</label><br />
    <input type="radio" name="age" value="medium15-25"
  id="medium15-25" /> <label for="medium15-25">15-25 ans</label><br />
    <input type="radio" name="age" value="medium25-40"
  id="medium25-40" /> <label for="medium25-40">25-40 ans</label><br />
    <input type="radio" name="age" value="plus40" id="plus40" />
    <label for="plus40">Encore plus vieux que ça ?!</label>
  </p>
</form>
```

Veuillez indiquer la tranche d'âge dans laquelle vous vous situez :

- ☐ Moins de 15 ans
- ☐ 15-25 ans
- ☒ 25-40 ans
- ☐ Encore plus vieux que ça ?!

Pourquoi avoir mis le même nom pour chaque option ? Tout simplement pour que le navigateur sache de quel "groupe" le bouton fait partie.

Essayez d'enlever les attributs name, vous verrez qu'il devient possible de sélectionner tous les éléments d'option. Or, ce n'est pas ce que l'on veut, c'est pour ça qu'on les "lie" entre eux en leur donnant un nom identique.

Vous noterez que cette fois on a choisi un id différent de name. En effet, les name étant identiques, on n'aurait pas pu les différencier (et vous savez bien qu'un id doit être unique !). Voilà donc pourquoi on a choisi de mettre à l'id la même valeur que value.

Si vous avez 2 zones d'options différentes, il faut donner un nom unique à chaque groupe comme ceci :

```

<form method="post" action="traitement.php">
  <p>
    Veuillez indiquer la tranche d'âge dans laquelle vous vous
    situez :<br />
    <input type="radio" name="age" value="moins15" id="moins15"
  /> <label for="moins15">Moins de 15 ans</label><br />
    <input type="radio" name="age" value="medium15-25"
  id="medium15-25" /> <label for="medium15-25">15-25 ans</label><br />
    <input type="radio" name="age" value="medium25-40"
  id="medium25-40" /> <label for="medium25-40">25-40 ans</label><br />
    <input type="radio" name="age" value="plus40" id="plus40" />
    <label for="plus40">Encore plus vieux que ça ?!</label>
  </p>
  <p>
    Sur quel continent habitez-vous ?<br />
    <input type="radio" name="continent" value="europe"
  id="europe" /> <label for="europe">Europe</label><br />
    <input type="radio" name="continent" value="afrique"
  id="afrique" /> <label for="afrique">Afrique</label><br />
    <input type="radio" name="continent" value="asie" id="asie"
  /> <label for="asie">Asie</label><br />
    <input type="radio" name="continent" value="amerique"
  id="amerique" /> <label for="amerique">Amérique</label><br />
    <input type="radio" name="continent" value="australie"
  id="australie" /> <label for="australie">Australie</label>
  </p>
</form>

```

L'attribut checked est là aussi disponible pour sélectionner une valeur par défaut.

## Les listes déroulantes

Les listes déroulantes sont un autre moyen élégant de faire un choix parmi plusieurs possibilités. Le fonctionnement est un peu différent. On va utiliser la balise <select></select> qui indique le début et la fin de la liste déroulante. On ajoute l'attribut name à la balise pour donner un nom à la liste.

Puis, à l'intérieur du <select></select>, nous allons placer plusieurs balises <option></option> (une par choix possible). On ajoute à chacune d'elles un attribut value pour pouvoir identifier ce que le visiteur a choisi.

Voici un exemple d'utilisation :

```

<form method="post" action="traitement.php">
  <p>
    <label for="pays">Dans quel pays habitez-vous ?</label><br />
    <select name="pays" id="pays">
      <option value="france">France</option>
      <option value="espagne">Espagne</option>
      <option value="italie">Italie</option>
      <option value="royaume-uni">Royaume-Uni</option>
      <option value="canada">Canada</option>
      <option value="etats-unis">Etats-Unis</option>
      <option value="chine">Chine</option>
      <option value="japon">Japon</option>
    </select>
  </p>

```

```
</form>
```

Si vous voulez qu'une option soit sélectionnée par défaut, utilisez cette fois l'attribut `selected` :

```
<option value="canada" selected>Canada</option>
```

Vous pouvez aussi grouper vos options avec la balise `<optgroup></optgroup>`. Dans notre exemple, pourquoi ne pas séparer les pays en fonction de leur continent ?

```
<form method="post" action="traitement.php">
  <p>
    <label for="pays">Dans quel pays habitez-vous ?</label><br />
    <select name="pays" id="pays">
      <optgroup label="Europe">
        <option value="france">France</option>
        <option value="espagne">Espagne </option>
        <option value="italie">Italie</option>
        <option value="royaume-uni">Royaume-Uni</option>
      </optgroup>
      <optgroup label="Amérique">
        <option value="canada">Canada</option>
        <option value="etats-unis">Etats-Unis</option>
      </optgroup>
      <optgroup label="Asie">
        <option value="chine">Chine</option>
        <option value="japon">Japon</option>
      </optgroup>
    </select>
  </p>
</form>
```

Les groupes ne peuvent pas être sélectionnés. Dans notre exemple, on ne peut pas choisir "Europe" par exemple, seuls les noms de pays sont sélectionnables.

### ***Finaliser et envoyer le formulaire***

Nous y sommes presque. Il ne nous reste plus qu'à agrémenter notre formulaire de quelques dernières fonctionnalités (comme la validation), puis nous pourrions ajouter le bouton d'envoi du formulaire.

### **Regrouper les champs**

Si votre formulaire grossit et comporte beaucoup de champs, il peut être utile de les regrouper au sein de plusieurs balises `<fieldset>`. Chaque `<fieldset>` peut contenir une légende avec la balise `<legend>`. Regardez cet exemple :

```
<form method="post" action="traitement.php">

  <fieldset>
    <legend>Vos coordonnées</legend> <!-- Titre du fieldset -->

    <label for="nom">Quel est votre nom ?</label>
    <input type="text" name="nom" id="nom" />
    <label for="prenom">Quel est votre prénom ?</label>
    <input type="text" name="prenom" id="prenom" />
    <label for="email">Quel est votre e-mail ?</label>
    <input type="email" name="email" id="email" />

  </fieldset>
```

```

<fieldset>
  <legend>Votre souhait</legend> <!-- Titre du fieldset -->
  <p>
    Faites un souhait que vous voudriez voir exaucé :

    <input type="radio" name="souhait" value="riche"
id="riche" /> <label for="riche">Etre riche</label>
    <input type="radio" name="souhait" value="celebre"
id="celebre" /> <label for="celebre">Etre célèbre</label>
    <input type="radio" name="souhait" value="intelligent"
id="intelligent" /> <label for="intelligent">Etre
<strong>encore</strong> plus intelligent</label>
    <input type="radio" name="souhait" value="autre"
id="autre" /> <label for="autre">Autre...</label>
  </p>
  <p>
    <label for="precisions">Si "Autre", veuillez préciser
:</label>
    <textarea name="precisions" id="precisions" cols="40"
rows="4"></textarea>
  </p>
</fieldset>
</form>

```

## Sélectionner automatiquement un champ

Vous pouvez placer automatiquement le curseur dans l'un des champs de votre formulaire avec l'attribut autofocus. Dès que le visiteur chargera la page, le curseur se placera dans ce champ.

Par exemple, pour que le curseur soit dans le champ "Prénom" par défaut :

```
<input type="text" name="prenom" id="prenom" autofocus />
```

## Rendre un champ obligatoire

Vous pouvez faire en sorte qu'un champ soit obligatoire en lui donnant l'attribut required.

```
<input type="text" name="prenom" id="prenom" required />
```

Le navigateur indiquera alors au visiteur qu'il doit remplir le champ si celui-ci est vide au moment de l'envoi.

Les anciens navigateurs, qui ne reconnaissent pas cet attribut, enverront le formulaire sans vérification. Il sera nécessaire pour eux de compléter les tests avec des scripts Javascript par exemple.

On dispose de pseudo-formats en CSS pour styliser les éléments requis (:required) et invalides (:invalid). Pour ajouter un fond rouge aux éléments requis qui n'auraient pas été saisis, on écrira donc :

```

:required
{
  background-color: red;
}

```

N'oubliez pas aussi que le pseudo-format :focus est disponible pour changer l'apparence d'un champ lorsque le curseur se trouve à l'intérieur.

## Le bouton d'envoi

Il ne nous reste plus qu'à créer le bouton d'envoi. Là encore, la balise `<input />` vient à notre secours. Elle existe en 4 versions :

- `type="submit"` : le principal bouton d'envoi de formulaire. C'est celui que vous utiliserez le plus souvent. Le visiteur sera envoyé à la page indiquée à l'attribut `action` du formulaire.
- `type="reset"` : remise à zéro du formulaire.
- `type="image"` : équivalent du bouton "submit", présenté cette fois sous forme d'image. Rajoutez l'attribut `src` pour indiquer l'URL de l'image.
- `type="button"` : bouton générique, qui n'aura (par défaut) aucun effet. En général, ce bouton est géré en Javascript pour exécuter des actions sur la page. Nous ne l'utiliserons pas ici.

On peut changer le texte affiché à l'intérieur des boutons avec l'attribut `value`.

Pour créer un bouton d'envoi, on écrira donc par exemple :

```
<input type="submit" value="Envoyer" />
```

Lorsque vous cliquez sur le bouton "Envoyer", le formulaire vous amène alors à la page indiquée à l'attribut `action`. Nous avons imaginé une page fictive (`traitement.php`), souvenez-vous.

Le problème, c'est que vous ne pouvez pas créer cette page seulement en HTML. Il est nécessaire d'apprendre un nouveau langage, comme [le PHP](#), pour pouvoir "récupérer" les informations saisies et décider quoi en faire. Ca sera l'objet de la deuxième partie de ce cours l'année prochaine en troisième année de baccalauréat.

## CHAPITRE XII : LA VIDEO ET L'AUDIO

### XII.1 Les formats audio et vidéo

#### Les formats audio

Pour diffuser de la musique ou n'importe quel son, il existe de nombreux formats. La plupart d'entre eux sont compressés (comme le sont les images JPEG, PNG et GIF) ce qui permet de réduire leur poids :



- **MP3** : C'est l'un des plus vieux, mais aussi l'un des plus compatibles (tous les appareils savent lire des MP3), ce qui fait qu'il est toujours très utilisé aujourd'hui.
- **AAC** : utilisé majoritairement par Apple sur iTunes, c'est un format de bonne qualité. Les iPod, iPhone et autres iPad savent les lire sans problème.
- **OGG** : le format Ogg Vorbis est très répandu dans le monde du logiciel libre, notamment sous Linux. Ce format a l'avantage d'être libre, c'est-à-dire qu'il n'est protégé par aucun brevet.
- **WAV** (format non compressé) : évitez de l'utiliser autant que possible, car le son est très volumineux avec ce format. C'est un peu l'équivalent du Bitmap (BMP) pour l'audio.

Aucun navigateur ne gère tous ces formats. Retenez surtout la compatibilité pour les MP3 et OGG :

| Navigateur        | MP3 | OGG |
|-------------------|-----|-----|
| Internet Explorer | Oui | -   |
| Chrome            | Oui | Oui |
| Firefox           | -   | Oui |
| Safari            | Oui | -   |
| Opera             | -   | Oui |

Il n'y a pas de format "idéal" reconnu par tous les navigateurs ?

Eh non ! Heureusement on pourra proposer différents formats aux navigateurs, qui sélectionneront celui qu'ils savent lire.

#### Les formats vidéo

Le stockage de la vidéo est autrement plus complexe. On a besoin de 3 éléments :

- **Un format conteneur** : c'est un peu comme une boîte qui va servir à contenir les deux éléments ci-dessous. On les reconnaît à l'extension du fichier en général : AVI, MP4, MKV...
- **Un codec audio** : c'est le format du son de la vidéo, généralement compressé. Nous venons de les voir, on utilise les mêmes : MP3, AAC, OGG...
- **Un codec vidéo** : c'est le format qui va compresser les images. C'est là que les choses se corsent, car ces formats sont complexes et on ne peut pas toujours les utiliser gratuitement. Les principaux à connaître pour le Web sont :
  - **H.264** : l'un des plus puissants et des plus utilisés aujourd'hui... mais il n'est pas 100% gratuit. En fait, on peut l'utiliser gratuitement dans certains cas (comme la diffusion de

vidéos sur son site web), mais il y a un flou juridique qui fait qu'il est risqué de l'utiliser à tout va.

- **Ogg Theora** : un codec gratuit et libre de droits, mais moins puissant que H.264. Il est bien reconnu sous Linux, mais sous Windows il faut installer des programmes pour pouvoir le lire.
- **WebM** : un autre codec gratuit et libre de droits, plus récent. Proposé par Google, c'est le concurrent le plus sérieux à H.264 à l'heure actuelle.

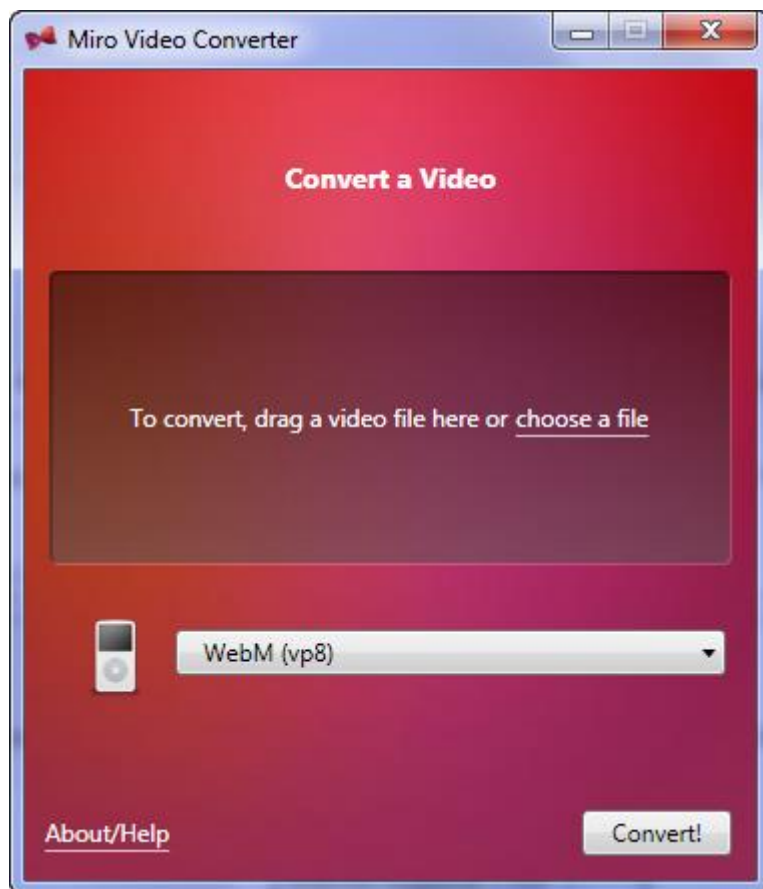
Quelle est la compatibilité des codecs vidéo sur les différents navigateurs ? Là encore, vous allez voir que c'est un joyeux bazar :

| Navigateur        | H.264 | Ogg Theora | WebM  |
|-------------------|-------|------------|-------|
| Internet Explorer | Oui   | -          | Oui * |
| Chrome            | -     | Oui        | Oui   |
| Firefox           | -     | Oui        | Oui   |
| Safari            | Oui   | -          | -     |
| Opera             | Oui   | Oui        | Oui   |

\* WebM pour IE ne fonctionne que si le codec est installé sur l'ordinateur.

Là encore, aucun format ne sort du lot. Il est conseillé de proposer sa vidéo en plusieurs formats pour qu'elle soit lisible sur un maximum de navigateurs.

Pour convertir une vidéo dans ces différents formats, je vous conseille l'excellent logiciel gratuit [Miro Video Converter](#) :



Il vous suffit de faire glisser-déposer votre vidéo dans la fenêtre du programme et de sélectionner le format de sortie souhaité. Cela vous permettra de créer plusieurs versions de votre vidéo !

## XII.2 Insertion d'un élément audio

La balise <audio> que nous allons découvrir est reconnue par tous les navigateurs récents, y compris Internet Explorer à partir de la version 9 (IE9).

En théorie, il suffit d'une simple balise pour jouer un son sur notre page :

```
<audio src="musique.mp3"></audio>
```

En pratique, c'est un peu plus compliqué que ça.

Si vous testez ce code... vous ne verrez rien ! En effet, le navigateur va seulement télécharger les informations générales sur le fichier (on parle de métadonnées), mais il ne se passera rien de particulier.

Vous pouvez compléter la balise des attributs suivants :

- controls : pour ajouter les boutons "Lecture", "Pause" et la barre de défilement. Ca peut sembler indispensable, et vous vous demandez peut-être pourquoi ça n'y est pas par défaut, mais certains sites web préfèrent créer eux-mêmes leurs propres boutons et commander la lecture avec du Javascript.
- width : pour modifier la largeur de l'outil de lecture audio.
- loop : la musique sera jouée en boucle.
- autoplay : la musique sera jouée dès le chargement de la page. Evitez d'en abuser, c'est en général irritant d'arriver sur un site qui joue de la musique tout seul !
- preload : indique si la musique peut être préchargée dès le chargement de la page ou non. Elle peut prendre les valeurs :
  - auto (par défaut) : le navigateur décide s'il doit précharger toute la musique, uniquement les métadonnées ou rien du tout.
  - metadata : charge uniquement les métadonnées (durée, etc.).
  - none : pas de préchargement. Utile si vous ne voulez pas gaspiller de bande passante sur votre site.

On ne peut pas forcer le préchargement de la musique, c'est toujours le navigateur qui décide. Les navigateurs mobiles, par exemple, ne préchargent jamais la musique pour économiser de la bande passante (le temps de chargement étant long sur portable).

Ajoutons les contrôles et ça sera déjà mieux !

```
<audio src="hype_home.mp3" controls></audio>
```

L'apparence du lecteur audio change en fonction du navigateur (ci-dessous l'apparence sous Google Chrome) :



Pourquoi ouvrir la balise pour la refermer immédiatement après ?

Cela vous permet d'afficher un message ou de proposer une solution de secours pour les navigateurs qui ne gèrent pas cette nouvelle balise. Par exemple :

```
<audio src="hype_home.mp3" controls>Veuillez mettre votre navigateur à jour !</audio>
```

Ceux qui ont un navigateur récent ne verront pas le message. Les anciens navigateurs, qui ne comprennent pas la balise, afficheront en revanche le texte qui se trouve à l'intérieur.

On a vu que certains navigateurs ne géraient pas le MP3, comment faire ?

Il faut proposer plusieurs versions du fichier audio. Dans ce cas, on va construire notre balise comme ceci :



```
<audio controls>
  <source src="hype_home.mp3"></source>
  <source src="hype_home.ogg"></source>
</audio>
```

Le navigateur prendra automatiquement le format qu'il reconnaît.

## XII.3 Insertion d'une vidéo

La balise `<video>` que nous allons découvrir est reconnue par tous les navigateurs récents, y compris Internet Explorer à partir de la version 9 (IE9).

Il suffit d'une simple balise `<video>` pour insérer une vidéo sur sa page :

```
<video src="sintel.webm"></video>
```

Mais, là encore, vous risquez d'être déçus si vous utilisez seulement ce code. Aucun contrôle ne permet de lancer la vidéo !

Rajoutons quelques attributs (la plupart sont les mêmes que pour la balise audio) :

- **poster** : image à afficher à la place de la vidéo lorsque celle-ci n'est pas lancée. Par défaut, le navigateur prendra la première image de la vidéo, mais comme il s'agit souvent d'une image noire ou d'une image peu représentative de la vidéo, je vous conseille d'en créer une ! Vous pouvez tout simplement faire une capture d'écran d'un moment de la vidéo.
- **controls** : pour ajouter les boutons "Lecture", "Pause" et la barre de défilement. Ca peut sembler indispensable, mais certains sites web préfèrent créer eux-mêmes leurs propres boutons et commander la lecture avec du Javascript. En ce qui nous concerne, ça sera largement suffisant !
- **width** : pour modifier la largeur de la vidéo.
- **height** : pour modifier la hauteur de la vidéo.
- **loop** : la vidéo sera jouée en boucle.
- **autoplay** : la vidéo sera jouée dès le chargement de la page. Là encore, évitez d'en abuser, c'est en général irritant d'arriver sur un site qui lance quelque chose tout seul !
- **preload** : indique si la vidéo peut être préchargée dès le chargement de la page ou non. Elle peut prendre les valeurs :
  - **auto** (par défaut) : le navigateur décide s'il doit précharger toute la vidéo, uniquement les métadonnées ou rien du tout.
  - **metadata** : charge uniquement les métadonnées (durée, dimensions, etc.).
  - **none** : pas de préchargement. Utile si vous ne voulez pas gaspiller de bande passante sur votre site.

On ne peut pas forcer le préchargement de la vidéo, c'est toujours le navigateur qui décide.

Les proportions de la vidéo sont toujours conservées. Si vous définissez une largeur et une hauteur, le navigateur fera en sorte de ne pas dépasser les dimensions indiquées, mais il conservera les proportions.

Voici un code un peu plus complet :

```
<video src="sintel.webm" controls poster="sintel.jpg"
width="600"></video>
```

Pourquoi ouvrir et refermer immédiatement après la balise ?

Vous utiliserez la balise `<source>` à l'intérieur de la balise `<video>` pour proposer différents formats. Le navigateur prendra celui qu'il reconnaît :

```
<video controls poster="sintel.jpg" width="600">
  <source src="sintel.mp4" />
  <source src="sintel.webm" />
  <source src="sintel.ogv" />
</video>
```

Les iPhone, iPad et iPod ne reconnaissent que le format H.264 à l'heure actuelle (fichier .mp4)... et uniquement si celui-ci est affiché en premier dans la liste ! Je vous recommande donc d'indiquer le format H.264 en premier pour assurer une compatibilité maximale.

## Envoyez votre site sur le web

### *Le nom de domaine*

Savez-vous ce qu'est un **nom de domaine** ?

Il s'agit en fait d'une adresse sur le Web : monsite.com est par exemple un nom de domaine. Un nom de domaine est constitué de 2 parties :

#### **monsite.com**

- **monsite**, le nom de domaine proprement dit. Il s'agit d'un nom que l'on peut généralement choisir librement tant que personne ne l'a réservé avant nous. Il peut contenir des lettres et des chiffres, mais pas de symboles particuliers (comme le ç français, le é, le è, les espaces, etc).
- **.com** ; l'extension (aussi appelée tld). Il existe grosso modo une extension par pays (.fr pour la France, .be pour la Belgique, .ca pour le Canada, .bi pour le Burundi et .cd pour la RDC). Toutefois, il y a aussi des extensions utilisées au niveau international comme .com, .net, .org. Elles étaient au départ réservées aux sites commerciaux, aux organisations, etc... mais cela fait longtemps que tout le monde peut les réserver. D'ailleurs, .com est très probablement l'extension la plus utilisée sur le Web.

En général, un site web voit son adresse précédée par "www", comme par exemple "www.monsite.com". Cela ne fait pas partie du nom de domaine : en fait, "www" est ce qu'on appelle un sous-domaine, et on peut en théorie en créer autant qu'on veut une fois qu'on est propriétaire du nom de domaine.

Le "www" a été adopté par tous les webmasters, c'est une sorte de convention, mais elle n'est absolument pas obligatoire.

### **Réserver un nom de domaine**

En effet, un nom de domaine est payant.

Le prix peut varier en fonction de l'extension. Ainsi, l'extension .info est généralement proposée à plus bas prix et peut s'avérer être une alternative intéressante. Mais si vous voulez une adresse plus "courante", il faudra plutôt viser une extension de type ".com", ou encore ".fr".

Pour réserver un nom de domaine, 2 solutions :

- Passer par un **registrar** spécialisé. C'est un organisme qui sert d'intermédiaire entre l'ICANN (l'organisation qui gère les noms de domaine au niveau international, tel les .com) et vous.
- Encore mieux : vous pouvez commander le nom de domaine en même temps que l'hébergement. Comme ça vous faites d'une pierre deux coups, vu que vous aurez de toute façon besoin de l'hébergement et du nom de domaine.

Dans ce chapitre, nous allons voir comment commander un nom de domaine en même temps que l'hébergement, c'est de loin la solution la plus simple et la moins coûteuse pour vous.

### *L'hébergeur*

Sur Internet, tous les sites web sont stockés sur des ordinateurs particuliers appelés "**Serveurs**". Ce sont des ordinateurs généralement très puissants qui restent tout le temps allumés. Ils contiennent les pages des sites web et les délivrent aux internautes qui les demandent, à toute heure du jour et de la nuit.

### **Le rôle de l'hébergeur**

L'hébergeur est une entreprise qui se charge de gérer des baies de serveurs. Elle s'assure du bon fonctionnement des serveurs 24h/24 7j/7. En effet, si l'un d'eux tombe en panne, tous les sites présents sur la machine deviennent inaccessibles.

Ces baies se situent dans des lieux particuliers appelés **datacenters**. Les datacenters sont donc des "entrepôts à serveurs" en quelque sorte, et leur accès est très protégé.



### Exemple d'un datacenter

Il est aussi possible en théorie d'héberger un site sur son propre ordinateur. Toutefois, c'est complexe, il vaut mieux avoir des connaissances en Linux, l'ordinateur doit être assez puissant, tourner jour et nuit et... surtout... la connexion doit être à très très haut débit (surtout en upload, la vitesse d'envoi des fichiers compte énormément). Les particuliers n'ont en règle générale pas une connexion suffisamment puissante pour héberger des sites, tandis que les datacenters oui : ils sont câblés en fibre.

Bref, gérer un serveur soi-même est complexe, et la plupart du temps les particuliers et les entreprises font appel à un hébergeur dont c'est le métier.

### Trouver un hébergeur

Les hébergeurs, contrairement aux registrars, sont très très nombreux. Il y en a de tous types, à tous les prix. Il y a un vocabulaire à connaître pour vous repérer dans leurs offres :

- **Hébergement mutualisé** : si vous optez pour une offre d'hébergement mutualisée, votre site sera placé sur un serveur gérant plusieurs sites à la fois (peut-être une centaine, peut-être plus). C'est l'offre la moins chère et c'est celle que je vous recommande de viser si vous démarrez votre site web.
- **Hébergement dédié virtuel** : cette fois, le serveur ne gère que très peu de sites (généralement moins d'une dizaine). Cette offre est généralement adaptée aux sites qui ne peuvent plus tenir sur un hébergement mutualisé car ils ont trop de trafic (trop de visiteurs), mais qui ne peuvent pas se payer un hébergement dédié (voir ci-dessous).
- **Hébergement dédié** (on parle aussi de "serveur dédié") : c'est le nec plus ultra. Le serveur gère uniquement votre site et aucun autre. Attention, cela coûte assez cher et il vaut mieux avoir des connaissances en Linux pour administrer le serveur à distance..

## Utiliser un client FTP

### Installer un client FTP

FTP signifie File Transfer Protocol et, pour faire court et simple, c'est le moyen que l'on utilise pour envoyer nos fichiers.

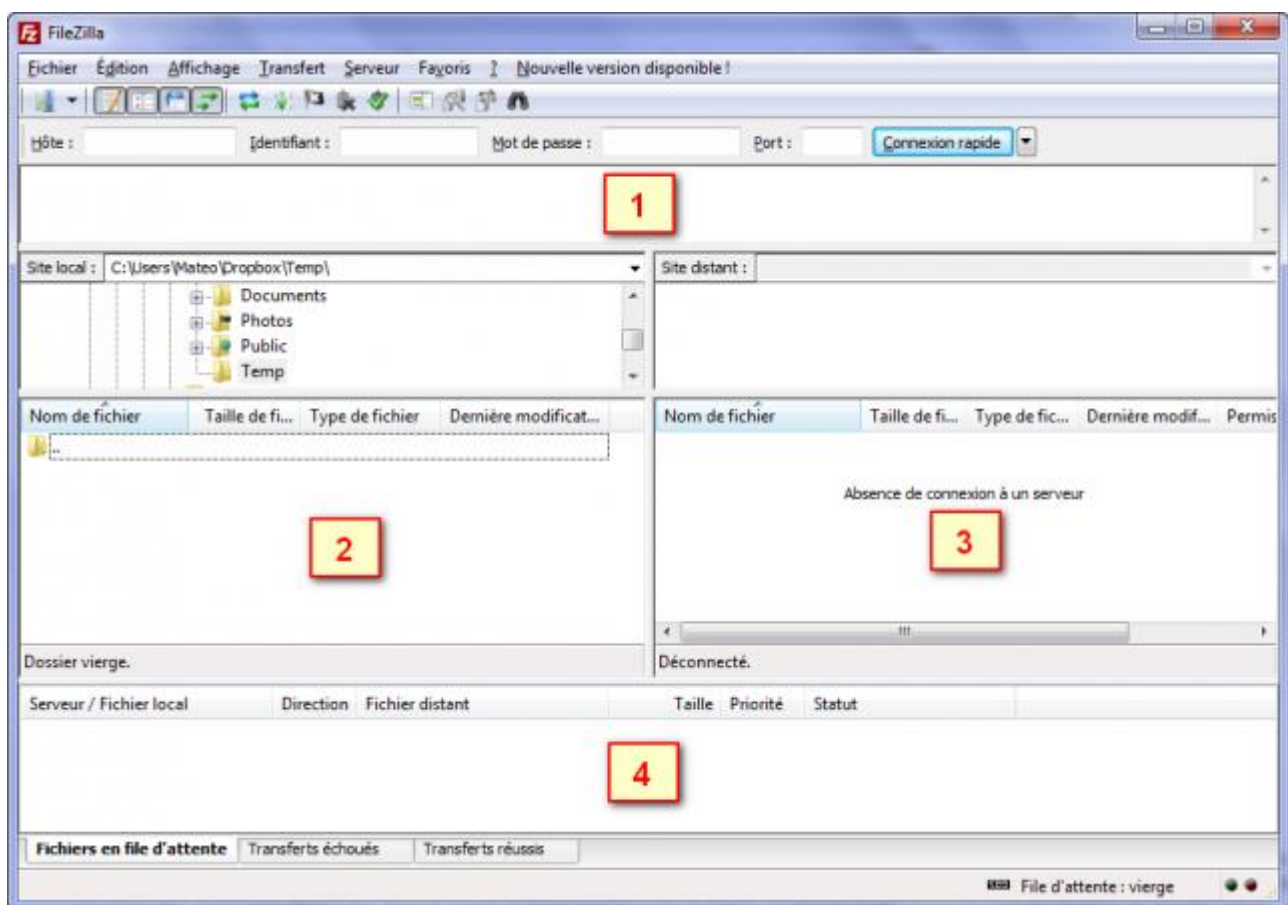
Il existe des logiciels permettant d'utiliser le FTP pour transférer vos fichiers sur Internet.

Bien entendu, des logiciels FTP il en existe des centaines, gratuits, payants, français, anglais etc. je vais vous proposer celui que j'utilise qui est gratuit et en français : **FileZilla**.

Quoiqu'il en soit, je vais vous montrer quelle est la marche à suivre avec FileZilla. Première étape : ... le télécharger bien entendu.

Prenez la version correspondant à votre système d'exploitation (Windows, Mac OS X ou Linux). L'installation est toute simple.

Lancez le logiciel, et vous devriez voir ceci :



A première vue, ça a l'air un peu compliqué (à première vue seulement). En fait, le principe est très simple.

Il y a 4 grandes zones dans la fenêtre à connaître :

1. En haut, vous verrez apparaître les messages qu'envoie et reçoit le logiciel. En général, cette zone ne nous intéresse pas vraiment, sauf s'il y a des messages d'erreur en rouge...
2. A gauche, c'est votre disque dur. Dans la partie du haut vous avez les dossiers, et dans la partie du bas la liste des fichiers du dossier actuel.
3. A droite, c'est la liste des fichiers envoyés sur Internet. Pour le moment il n'y a rien, car on ne s'est pas "connecté", mais ça va venir ne vous en faites pas.
4. Enfin, en bas, vous verrez apparaître les fichiers en cours d'envoi (et le pourcentage d'envoi).

La première étape va être de se "connecter" au serveur de votre hébergeur.

## Configurer le client FTP

Quel que soit l'hébergeur que vous avez choisi, cela fonctionne toujours de la même manière. On va vous fournir **3 informations** qui sont indispensables pour que FileZilla puisse se connecter au serveur :

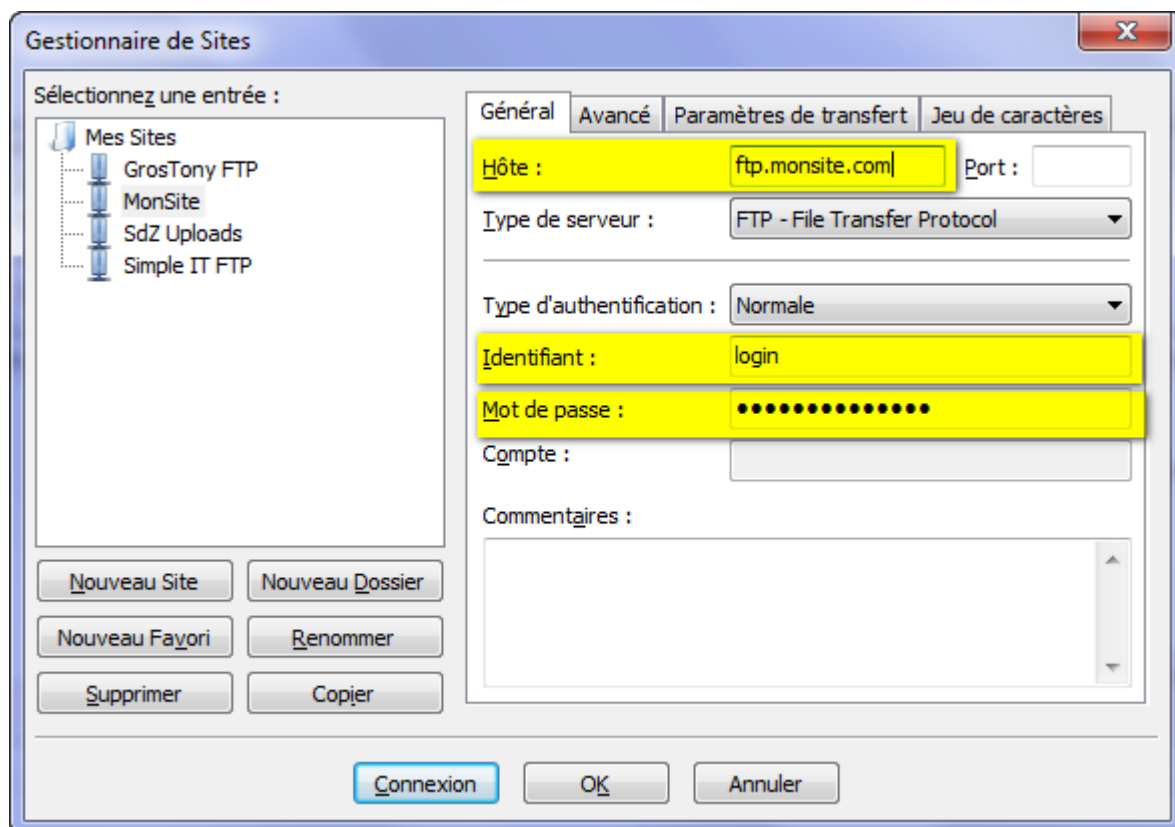
- **L'IP** : c'est "l'adresse" du serveur. Le plus souvent, on vous donnera quelque chose du genre <ftp.mon-site.com>, mais il peut aussi s'agir d'une suite de nombres comme 122.65.203.27
- **Le login** : c'est votre identifiant, on vous l'a probablement demandé. Vous avez peut-être mis votre pseudo, ou le nom de votre site. Mon login pourrait par exemple être mateo21.
- **Le mot de passe** : soit on vous a demandé un mot de passe, soit (c'est plus probable) on vous en a attribué un d'office (un truc imprononçable du genre crf45u7h)

Si vous avez ces 3 informations, vous pouvez continuer. Si vous ne les avez pas, il faut que vous les cherchiez c'est indispensable. On vous les a probablement envoyées par mail. Sinon, n'hésitez pas à les demander à votre hébergeur (IP / Login / Mot de passe).

Maintenant que nous sommes en possession de ces informations, nous allons les donner à FileZilla qui en a besoin pour se connecter au serveur.

Cliquez sur la petite icône en haut à gauche (pas sur la petite flèche à droite, mais sur l'image) : 

Une fenêtre s'ouvre. Cliquez sur "Nouveau site" et donnez-lui le nom que vous voulez. A droite, vous allez devoir indiquer les 3 informations dont je viens de vous parler, comme ceci :



Vous pouvez distinguer en haut l'hôte (c'est là qu'il faut indiquer par exemple <ftp.monsite.com>). Pour pouvoir entrer le login / mot de passe, cochez **Type d'authentification : Normal**

Cliquez sur "Connexion" et le tour est (presque) joué.



## Transférer les fichiers

A ce stade, 2 possibilités :

- Soit la connexion a réussi, et vous voyez apparaître en haut des messages en vert comme "Connecté". Dans ce cas, la zone de droite de la fenêtre de FileZilla devrait s'activer et vous verrez les fichiers qui se trouvent déjà sur le serveur (il se peut qu'il y en ait quelques-uns déjà présents).
- Soit ça a planté, vous avez plein de messages écrits en rouge : vous vous êtes plantés en tapant l'IP ou le login ou le mot de passe. Un de ces éléments est incorrect, veuillez à les redemander à votre hébergeur car s'ils sont bons ça doit marcher.

Si la connexion a réussi, alors ce que vous avez à faire est très simple : dans la partie de gauche, cherchez où se trouvent sur votre disque dur vos fichiers .html et .css (mais aussi vos images .jpg, .png, .gif, etc.).

Double-cliquez sur le fichier que vous voulez transférer à gauche. Au bout de quelques secondes, il apparaîtra à droite, ce qui voudra dire qu'il a été correctement envoyé sur le serveur, donc qu'il est accessible sur Internet !

Vous pouvez envoyer n'importe quel type de fichier. Bien entendu, généralement on envoie des fichiers .php, .html, .css et des images, mais vous pouvez aussi très bien envoyer des .pdf, des programmes, des .zip, etc.

Veuillez noter qu'il faut que votre page d'accueil s'appelle index.html. C'est la page qui sera chargée lorsqu'un nouveau visiteur arrivera sur votre site.

Vous pouvez aussi transférer des dossiers entiers d'un seul coup : il suffit de faire glisser-déplacer le dossier depuis la partie de gauche jusqu'à la partie de droite de la fenêtre.

Une fois configuré, vous pouvez voir que l'envoi de fichiers est très simple.